

VITOR BORGES MONTEIRO
HEAD ORGANIZER

MÉTODOS COMPUTACIONAIS APLICADOS À FINANÇAS: USO DO PROGRAMA R



REVISTA
OBSERVATORIO
DE LA ECONOMÍA
LATINOAMERICANA

2025

Vitor Borges Monteiro
Head Organizer

**Métodos Computacionais
Aplicados à Finanças: Uso do
Programa R**

**Editora Observatório de la
Economía Latino Americano
2025**

Copyright®
Editora Observatório de la Economía
Latino Americano

Copyright do Texto® 2025

O autor

Copyright da Edição® 2025

Observatório de la Economía Latino
Americano Editora

Diagramação

Editora

Edição de Arte

Editora

Revisão

O autor

O conteúdo dos artigos e seus dados em sua forma, correção e confiabilidade são de responsabilidade exclusiva do autor. Permitido o download da obra e o compartilhamento desde que sejam atribuídos créditos a autora, mas sem a possibilidade de alterá-la de nenhuma forma ou utilizá-la para fins comerciais.

Editor Chefe

Prof. Sérgio António Neves Lousada

Conselho Editorial

Axel Etchart

Antonio Rodrigues

Carlos Ruz Saldívar

Benjamín Castillo Osorio

Jorge Enrique Chaparro Medina

Site

www.observatoriolatinoamericano.com

E-mail

editora@observatoriolatinoamericano.com

Dados Internacionais de Catalogação na Publicação (CIP)

Métodos Computacionais Aplicados à Finanças: Uso do Programa R [livro eletrônico] organização Vitor Borges Monteiro. – 1st Edition -- Curitiba, PR: Editora Observatório de la Economía Latino Americano, 2025.

PDF.

Bibliografia.

ISBN: 978-65-83190-19-2

DOI: 10.55905/edobs.978-65-83190-19-2

1. Análise Financeira. 2. Métodos Computacionais. 3. R.
I. Monteiro, Vitor Borges. II. Título.



APRESENTAÇÃO

Este ebook "Métodos Computacionais Aplicados à Finanças: Uso do Programa Estatístico R", oferece uma introdução prática e progressiva à linguagem R voltada para estudantes e profissionais da área financeira. Começa com noções fundamentais, como operações aritméticas, funções matemáticas e manipulação de objetos, vetores e sequências. Ensina também a criação de amostras aleatórias, uso de operadores lógicos e funções estatísticas básicas, essenciais para análise de dados.

A obra avança com foco na visualização de dados, ensinando como construir gráficos de barras, pizza e dispersão com personalização de cores, legendas, títulos e inserção de texto no gráfico. O leitor aprende a abrir janelas com múltiplos gráficos e interpretar relações visuais entre variáveis.

Em seguida, o texto aborda a manipulação de bases de dados com `data.frames`, leitura de arquivos `.txt` e `.xls`, operações com colunas e linhas, uso de `attach`, `subset`, e agregações com `tapply` e `aggregate`. Os exercícios práticos reforçam a compreensão sobre agrupamento por setores, trimestres e média de preços por categorias.

A apostila também cobre matrizes, com operações como transposição, cálculo de determinante, inversa e multiplicação matricial, além de aplicações como o cálculo do estimador beta ($\hat{\beta}$) por regressão linear múltipla. Há ainda a introdução ao uso de funções criadas pelo usuário, estruturas de controle (`for`, `if`, `while`) e simulações de sorteios e jogos como Mega-Sena entre outros.

O conteúdo é finalizado com exemplos de funções financeiras (valor presente, valor futuro, VPL) e uma introdução à análise de séries temporais e análise técnica, utilizando o pacote `quantmod`, com gráficos, médias móveis e, por fim, constrói indicadores de análise técnica de ações e aplica backtesting de estratégias de investimento, teoria da carteira e CAPM. A abordagem didática, combinando teoria e prática, torna o material um guia acessível e valioso para quem deseja aplicar o R na análise financeira real.

SUMÁRIO

CAPÍTULO 01	6
INTRODUÇÃO AO R	
CAPÍTULO 02	9
MANIPULAÇÃO DE DADOS E OBJETOS NO R	
CAPÍTULO 03	22
3GRÁFICOS	
CAPÍTULO 04	32
MATRIZES	
CAPÍTULO 05	39
CRIANDO FUNÇÕES	
CAPÍTULO 06	55
MANIPULAÇÃO DE DADOS	
CAPÍTULO 07	70
SÉRIES TEMPORAIS	
CAPÍTULO 08	104
ESTRATÉGIAS DE INVESTIMENTOS E BACKTESTING	
CAPÍTULO 09	121
INTRODUÇÃO A TEORIA DA CARTEIRA	
CAPÍTULO 10	145
MODELO DE PRECIFICAÇÃO DE ATIVOS DE CAPITAL - CAPM	



CAPÍTULO 01

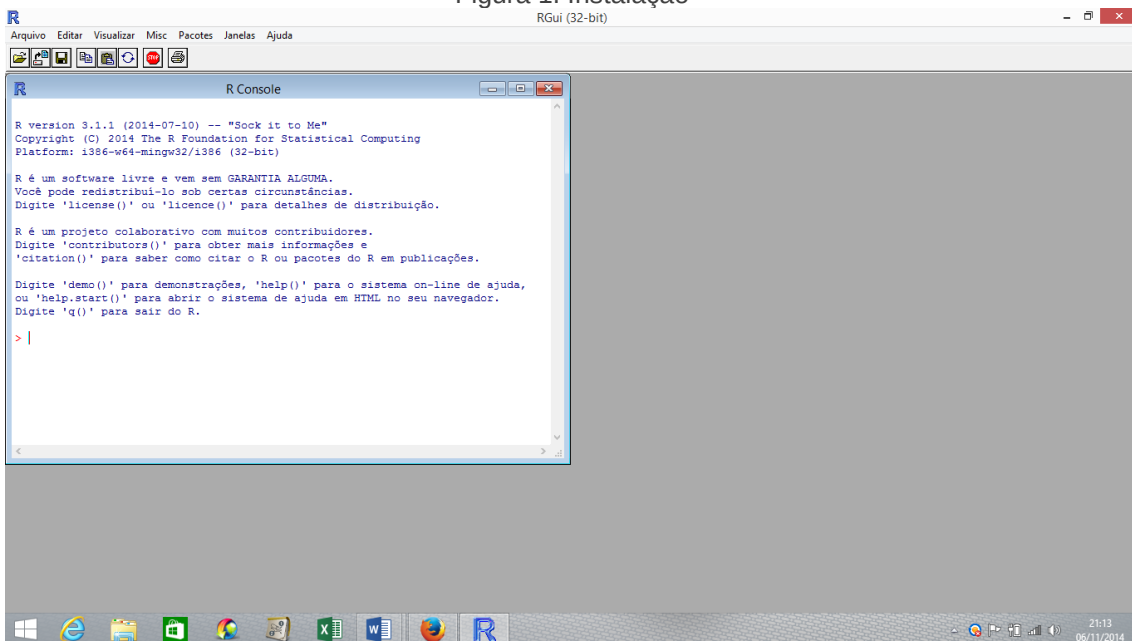
INTRODUÇÃO AO R

O primeiro passo para utilizar o R é realizar sua instalação no computador. Para isso, siga as etapas abaixo:

1. **Acesse o site oficial do R:** www.r-project.org.
2. **Escolha um servidor CRAN:** No menu lateral, clique em "CRAN" e selecione um servidor brasileiro, como cran.fiocruz.br.
3. **Faça o download para Windows:**
 - Clique em "Download R for Windows".
 - Escolha a opção "Base".
 - Baixe o instalador, por exemplo: "Download R 4.4.2 for Windows" (ou a versão mais recente).
4. **Instale o programa:** Execute o arquivo baixado e siga as instruções apresentadas.

Após completar a instalação, o R estará pronto para uso. No próximo tópico, abordaremos os conceitos básicos e o funcionamento do ambiente do R.

Figura 1. Instalação



Fonte: Elaboração própria

1.1 OPERAÇÕES ARITMÉTICAS

As operações aritméticas básicas podem ser realizadas diretamente no **R console**, indicado pelo prompt. Veja alguns exemplos de comandos:

```
2 + 2 # Soma
2 * 3 # Multiplicação
10 / 2 # Divisão
4^2 # Potência
4**2 # Potência (alternativa)
```

É possível incluir comentários explicativos nos comandos utilizando o símbolo `#`. O R ignora o que estiver após o `#`, interpretando como texto informativo. Ao pressionar **Enter** em cada comando, o resultado aparecerá no console. Por exemplo:

Ao pressionar **Enter** em cada comando, o resultado aparecerá no console. Por exemplo:

```
((4 * 10) / 8)^2 # Operação combinada
# Resultado:
[1] 25
```

1.2 FUNÇÕES BÁSICAS

Para organizar melhor o trabalho, recomenda-se utilizar o **Editor R** para editar os comandos e deixar o **R console** apenas para exibição dos resultados. Siga os passos abaixo:

1. Acesse **Arquivo -> Novo script** para abrir o **Editor R**.
2. No Editor R, insira os comandos desejados.
3. Para executar os comandos:
 - Use o botão **Executar linha** na barra de ferramentas; ou
 - Pressione o atalho **Ctrl + R**.
4. Salve o arquivo do Editor R em **Arquivo -> Salvar como** para registrar suas rotinas no computador.

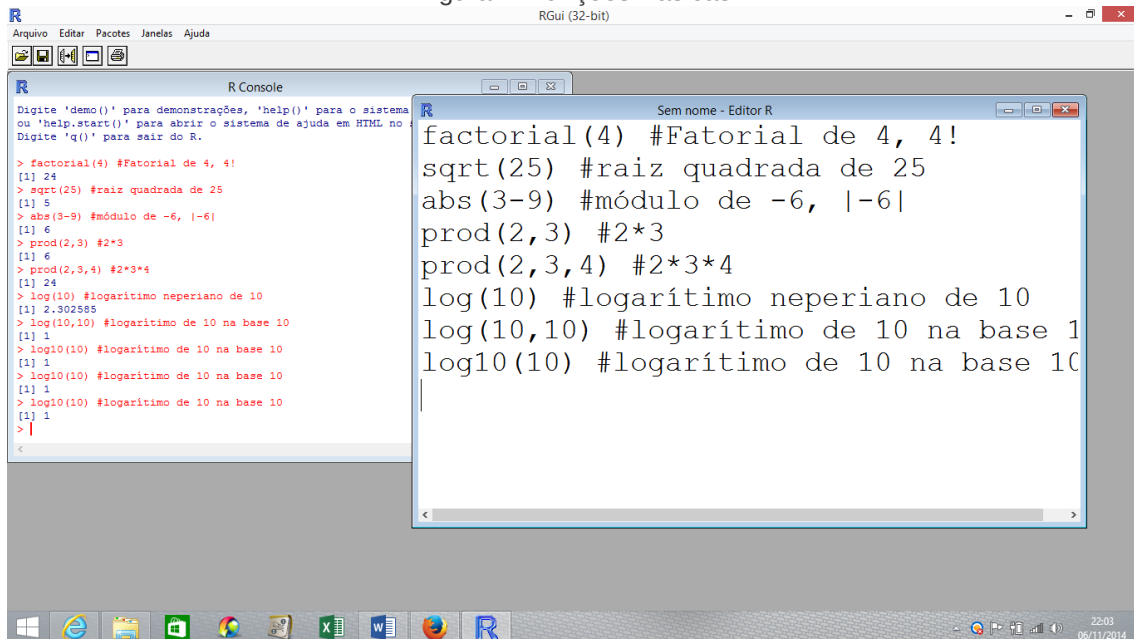
```
factorial(4) # Calcula o fatorial de 4 (4!)
sqrt(25)    # Calcula a raiz quadrada de 25
abs(3 - 9)  # Retorna o valor absoluto de -6 (|-6|)
```

```

prod(2, 3) # Calcula o produto: 2 * 3
prod(2, 3, 4) # Calcula o produto: 2 * 3 * 4
log(10) # Calcula o logaritmo neperiano de 10
log(10, 10) # Calcula o logaritmo de 10 na base 10
log10(10) # Alternativa para calcular o logaritmo de 10 na base 10

```

Figura 2. Funções Básicas



Fonte: Elaboração própria

Na janela acima, o **Editor R** executou todas as funções apresentadas anteriormente. Para acompanhar os resultados, observe o **R console**. Se desejar aumentar o tamanho da fonte no Editor R, mantenha a tecla **Ctrl** pressionada enquanto gira o botão de rolagem do mouse. Esse procedimento é particularmente útil em exposições com projetor.



CAPÍTULO 02

MANIPULAÇÃO DE DADOS E OBJETOS NO R

2.1 OPERAÇÕES BÁSICAS COM OBJETOS

No **Editor R**, crie os objetos x e y:

```
x<-10    #Salva o valor 10 no objeto x, clique em executar na barra de ferramentas
```

```
x <- 10   # Salva o valor 10 no objeto x
```

```
x        # Exibe o objeto x no Console
```

```
y = 15    # Salva o valor 15 no objeto y
```

```
y        # Exibe o objeto y no Console
```

```
list(x, y) # Lista simultaneamente os objetos x e y
```

```
prod(x, y) # Exibe o produto de x e y
```

O R diferencia letras maiúsculas de minúsculas, ou seja, x e X são objetos distintos.

Para visualizar um objeto, você pode simplesmente digitá-lo no Editor R e executá-lo, como nos exemplos acima, ou usar `list(x)` e `list(y)`, `list(x, y)` para exibi-los em conjunto.

O comando `ls()` mostra todos os objetos criados até o momento.

```
rm(x)     # Remove o objeto x
```

```
remove(x) # Remove o objeto x
```

```
remove(x, y) # Remove os objetos x e y
```

Você também pode salvar o resultado de uma função em um objeto:

```
z <- prod(x, y) # Salva o produto de x e y no objeto z
```

```
list(z)        # Exibe o conteúdo do objeto z
```

2.2 CRIANDO OBJETOS COM VÁRIOS ELEMENTOS

Para criar objetos que contenham vários valores, use a função `c()`. Ela permite ao R manipular um conjunto de elementos. Por exemplo, vamos criar o

objeto preço:

```
preço <- c(4.5, 4.8, 5.3, 6.8, 7.1, 8, 8.2, 8.8)
list(preço) # Ou simplesmente digite preço e execute
```

O objeto preço possui 8 elementos. Observe que o ponto (.) representa a casa decimal, enquanto a vírgula separa os elementos. Para consultar elementos de um objeto que possui n valores, utilize objeto[i] para acessar o i-ésimo elemento ou objeto[c(i, ..., j)] para visualizar vários elementos simultaneamente.

Exemplos:

```
preço[1]    # Mostra o primeiro elemento
preço[2]    # Mostra o segundo elemento
preço[6] <- 8.1 # Altera definitivamente o sexto elemento para 8.1
preço[c(1, 3, 6)] # Mostra os elementos 1, 3 e 6
preço[-1]   # Mostra todos os elementos, exceto o primeiro
preço[-2]   # Mostra todos os elementos, exceto o segundo
preço[-8]   # Mostra todos os elementos, exceto o oitavo
preço[c(-1, -2, -3)] # Exibe todos os elementos, exceto os três primeiros
```

Se quiser remover elementos de forma definitiva, crie um novo objeto para salvar as alterações:

```
preço1 <- preço[c(-1, -2, -3)] # Remove os três primeiros elementos e
salva em preço1
```

2.3 FUNÇÕES COM OBJETOS

As principais funções estatísticas possuem comandos específicos para trabalhar com os objetos criados no R. Abaixo, continuamos usando o objeto preço como exemplo:

```
mode(preço)    # Informa se o objeto é numérico ou caractere
length(preço)  # Retorna a quantidade de elementos do objeto preço
sum(preço)     # Soma todos os valores do objeto preço
sum(preço) / length(preço) # Calcula a média dos valores (soma /
quantidade)
media <- sum(preço) / length(preço) # Cria o objeto media com a média
de preço
mean(preço)    # Função para calcular a média
```

```

max(preço)      # Valor máximo
min(preço)      # Valor mínimo
preço^2         # Eleva todos os valores ao quadrado
preço / 10      # Divide todos os valores por 10
log(preço)      # Aplica logaritmo neperiano em cada valor
log(preço, 10)  # Logaritmo na base 10 em cada valor
sqrt(preço)     # Raiz quadrada de cada valor

```

O R também reconhece objetos não numéricos. Como exemplo, crie o objeto letras:

```

letras <- c("a", "b", "c", "d", "e", "f") # Cria o objeto letras
list(letras)      # Lista os elementos
length(letras)    # Quantidade de elementos
mode(letras)      # Indica que o objeto é do tipo caractere
letras[c(2, 4, 6)] # Consulta os elementos 2, 4 e 6

```

Também é possível ter mais de um caractere por elemento:

```

cidades <- c("Manaus", "Belém", "São Luis", "Fortaleza", "Natal", "Recife",
"Salvador")
list(cidades) # Lista os elementos
mode(cidades) # Tipo do objeto (caractere)
length(cidades) # Quantidade de elementos
cidades[5] <- "João Pessoa" # Substitui o 5º elemento (Natal) por João
Pessoa
cidades      # Visualiza a alteração

```

Suponha que você queira adicionar novamente a cidade **Natal**, como último elemento do objeto:

```

cidades1 <- c(cidades, "Natal") # Cria um novo objeto, adicionando 'Natal'
cidades1

```

Para adicionar na ordem correta, entre Fortaleza e João Pessoa, faça:

```

cidades2 <- c(cidades[c(1,2,3,4)], "Natal", cidades[c(5,6,7)]) # Cria um
novo objeto, adicionando 'Natal' na ordem correta
cidades2

```

2.4 GERANDO SEQUÊNCIAS E REPETIÇÕES

Ao trabalhar com dados no R, é comum a necessidade de criar sequências e repetições. Abaixo, apresentamos algumas funções fundamentais para essas tarefas.

```
1:10      # Gera uma sequência de 1 a 10
5:16      # Gera uma sequência de 5 a 16
w <- 5:16  # Armazena a sequência no objeto w
```

A função `sequencia` possui três argumentos: `seq(from,to,by)`.

```
seq(from = 1, to = 10, by = 2) # Sequência de 1 a 10 com intervalos de 2
(1, 3, 5, 7, 9)
```

Poderá ser inserido apenas os valores obedecendo a ordem de entrada dos argumentos.

```
seq(0,100,5) #Gera uma sequência de 0 a 100, intercalada por 5.
```

A função `rep` serve para gerar repetições. Ela possui três argumentos principais: `rep(valor, repetições, each)`

- O primeiro indica o valor (ou vetor) a ser repetido;
- O segundo determina a quantidade de repetições;
- O terceiro (opcional), especificado por `each`, indica uma repetição expandida de cada elemento.

Alguns exemplos:

```
rep(5, 10)      # Repete o número 5, 10 vezes
rep("a", 10)     # Repete a letra 'a', 10 vezes
rep(1:3, 3)      # Repete a sequência de 1 a 3, três vezes (1 2 3 / 1 2 3 / 1
2 3)
```

```
rep(1:3, each = 3) # Expande cada elemento de 1 a 3, 3 vezes consecutivas
```

```
rep(c("a", "b"), 5) # Repete os caracteres 'a' e 'b', 5 vezes alternadas
```

```
rep(c("a", "b"), each = 5) # Repete 'a' 5 vezes e, em seguida, 'b' 5 vezes
```

```
rep(c(1, 2), 5) # Repete os números 1 e 2, 5 vezes alternadas
```

```
rep(c(1, 2), each = 5) # Repete o número 1 cinco vezes e, em seguida, o
2 cinco vezes
```

Se os argumentos da função `rep` forem vetores, ambos devem possuir a mesma dimensão, para associar cada elemento do primeiro argumento com

cada respectiva repetição do segundo argumento, os valores são repetidos pelas suas ordens de entrada.

Exemplo de repetição expandida com vários elementos:

```
rep(c("três", "quatro", "dois", "sete", "quatro"), each = 5) # Cada elemento
é repetido 5 vezes antes de passar ao próximo
```

Caso sejam usados vetores como argumentos em rep, ambos devem ter a mesma dimensão. Dessa forma, cada elemento do primeiro vetor é repetido conforme a posição correspondente do segundo vetor:

```
rep(c("três", "quatro", "dois", "sete", "quatro"), c(1, 2, 7, 4, 2))
# [1] "três" "quatro" "quatro" "dois" "dois" "dois" "dois" "dois"
# [9] "dois" "dois" "sete" "sete" "sete" "sete" "sete" "quatro" "quatro"
rep(c(5, 3), c(7, 4))
# [1] 5 5 5 5 5 5 3 3 3 3
```

2.5 CONJUNTOS

O R oferece comandos básicos para lidar com operações da teoria dos conjuntos. Considere, por exemplo, os conjuntos A e B criados no Editor R:

```
A <- c(1, 2, 3, 4, 5)
B <- c(4, 5, 6)
union(A, B) # A ∪ B
# [1] 1 2 3 4 5 6
intersect(A, B) # A ∩ B
# [1] 4 5
setdiff(A, B) # Elementos em A que não estão em B
# [1] 1 2 3
setdiff(B, A) # Elementos em B que não estão em A
# [1] 6
```

2.6 GERAR DADOS ALEATÓRIOS

Além de gerar dados por meio de sequências e repetições, também é possível criar dados aleatórios a partir de distribuições uniforme e normal.

Uma variável aleatória é uniformemente distribuída em um intervalo [a,b]. Para gerar dados uniformes, utiliza-se o comando `runif`, que possui três argumentos: `runif(n, min = a, max = b)`, onde: **n** é o número de observações que serão geradas; **min** define o limite inferior do intervalo; **max** define o limite superior do intervalo. Veja alguns exemplos:

```
runif(10, max = 0, min = 1) # Gera 10 observações no intervalo [0, 1]
runif(10, 0, 1)             # Equivalente ao exemplo acima
w <- runif(100, 5, 10)      # Cria o objeto w com 100 valores no intervalo [5, 10]
```

A **esperança matemática** de uma variável aleatória X uniformemente distribuída no intervalo [a,b]. $E(X)=(a+b)/2$. No caso do objeto w gerado anteriormente no intervalo [5,10], a esperança matemática esperada seria:

```
(min(w) + max(w)) / 2 # Calcula (a + b)/2 com base nos valores gerados
```

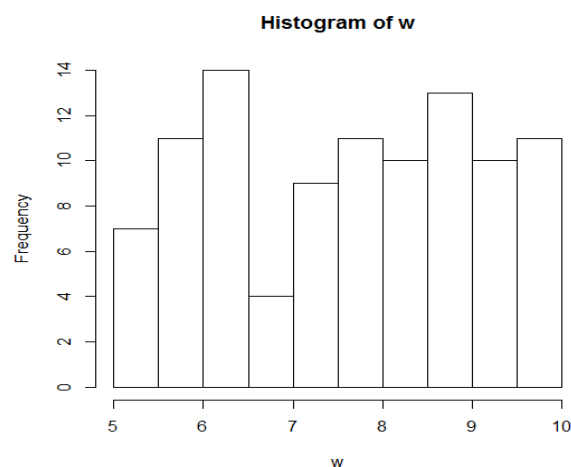
Entretanto, podemos verificar a média empírica das amostras:

```
mean(w) # Retorna a média das 100 observações armazenadas em w
```

Para visualizar a distribuição dos valores gerados, podemos construir um histograma:

```
hist(w) # Exibe o histograma dos dados do objeto w
```

Figura 3. Histogram of w



Fonte: Elaboração própria

Observe que cada computador produzirá dados aleatórios distintos, o que pode resultar em diferentes histogramas para cada execução do código.

Uma variável aleatória X é normal com parâmetros μ , σ , notação $X \sim N(\mu, \sigma)$.

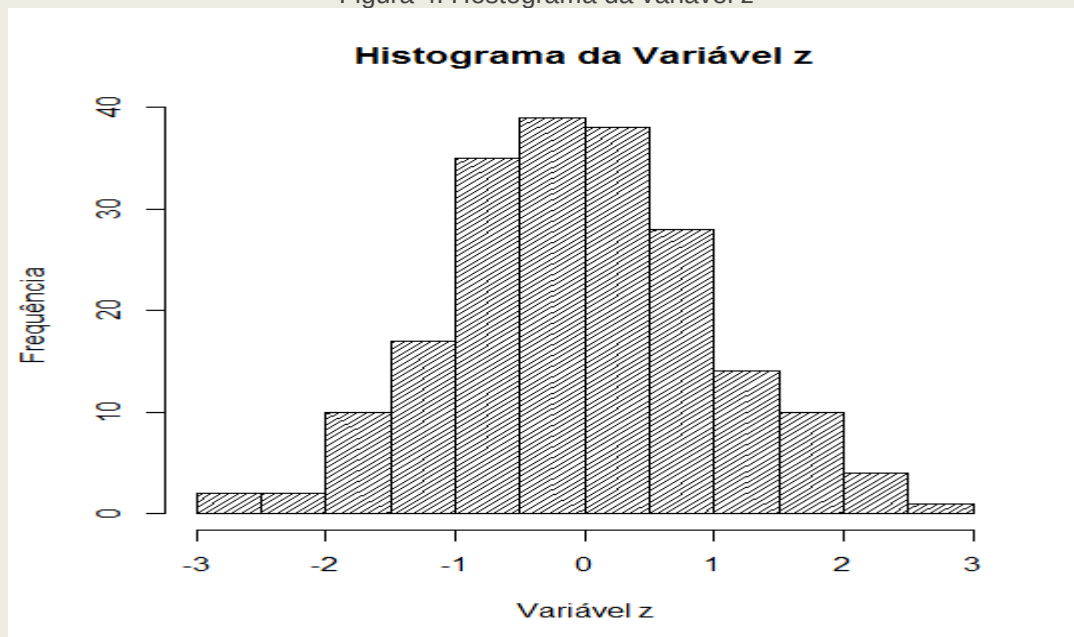
σ^2), onde μ é a média e σ^2 a variância. A função `rnorm` gera dados normalmente distribuídos com três argumentos: `rnorm(n, mean, var)`.

```
rnorm(200, 0, 1) # Gera 200 observações com média 0 e variância 1
rnorm(200)       # Omite média e variância, gerando a Normal(0, 1) por
padrão
z <- rnorm(200, 0, 1) # Salva as 200 observações no objeto z
hist(z)           # Exibe o histograma dos dados no objeto z
```

Para cada nova execução das funções `runif` e `rnorm`, serão gerados novos dados aleatórios, ocasionando variações em seus valores e, conseqüentemente, em seus histogramas. Abaixo, apresentamos o histograma do objeto `z` com algumas opções de personalização na função `hist`:

```
hist(z, main = "Histograma do Objeto z", xlab = "Valores de z", col
= "skyblue",
border = "white", breaks = 10).
```

Figura 4. Histograma da variável `z`



Fonte: Elaboração própria

Nas instruções acima:

- **main** define o título do histograma;
- **xlab** rotula o eixo x;
- **col** define a cor de preenchimento das barras;
- **border** define a cor da borda das barras;

- **breaks** controla o número de barras no histograma.

Ainda utilizando os dados armazenados no objeto `z`, podemos aplicar diversas funções estatísticas adicionais no R, por exemplo:

```
var(z) #toma a variância C.  
sd(z) #toma o desvio padrão dos dados salvos no objeto z  
sd(z)^2 #confirme que o quadrado do desvio padrão é a variância.  
median(z) #toma a mediana dos dados salvos no objeto z  
summary(z) # Fornece um resumo estatístico abrangendo min, quartis,  
mediana, média e max  
fivenum(z) # Retorna 5 estatísticas sumárias (similar ao summary, mas  
sem a média)  
range(z) # Mostra o valor mínimo e o valor máximo do objeto z  
IQR(z) # Retorna a amplitude interquartil (3º quartil - 1º quartil)  
length(z) # Exibe a quantidade de elementos em z
```

A mediana é o valor central dos dados organizados em um rol (dados em ordem crescente). Quando o número de elementos é ímpar, é fácil observar qual a observação central. Como o objeto `z` possui 200 observações (par), a mediana será a média aritmética da observação 100 e 101. Faça o seguinte exercício.

```
# 1. Ordene as observações de z em ordem crescente  
z_ordenado <- sort(z)  
  
# 2. Selecione as observações na posição 100 e 101  
z_ordenado[100]  
z_ordenado[101]  
  
# 3. Calcule a média aritmética dessas duas observações  
mediana_manual <- (z_ordenado[100] + z_ordenado[101]) / 2  
  
# 4. Compare com a mediana obtida pela função median()  
mediana_fun <- median(z)  
  
# Visualize o resultado  
mediana_manual  
mediana_fun
```

2.7 GERAR AMOSTRAS ALEATÓRIAS.

Pode-se gerar amostras aleatórias a partir de um conjunto de dados já estabelecido, a sintaxe é a seguinte: `sample(x, size, replace = TRUE ou FALSE)`, onde `x` é o conjunto de dados original, `size` é o tamanho da amostra que vai ser extraída desse conjunto de dados e `replace` refere-se ao experimento a ser realizado: com reposição (`TRUE`) ou sem reposição (`FALSE`).

```
sample(1:10, 5, FALSE)
# Extrai 5 valores aleatórios da sequência 1:10, sem reposição
sample(1:10, 5)
# Equivalente ao comando anterior; ao omitir replace, o R assume replace
= FALSE
sample(1:10, 15) #Note que não é possível extrair 15 observações de 10,
sem reposição.
sample(1:10, 15, TRUE) # Extrai 15 valores aleatórios da sequência 1:10
com reposição.
```

Para simular o lançamento de uma moeda 10 vezes, utilizamos `sample()` com **replace = TRUE**, pois cada lançamento é independente

```
moeda<-c ("cara","coroa") #Cria um objeto com os elementos caracteres
“cara” e “coroa”
sample(moeda, 10, TRUE) #A moeda será lançada dez vezes, note que
esse experimento só é possível devido o replace com reposição.
table(sample(moeda, 10, TRUE)) #comando table elabora a distribuição
de frequência absoluta.
```

Similarmente, Jogue um dado 20 vezes e observe a distribuição de frequência.

```
dado<-c(1:6)
resultado<-sample(dado,20,TRUE)
table(resultado)
```

Para escolher 6 números (sem reposição) dentre 1 a 60, tal como em uma aposta simples na Megasena:

```
dados<-1:60
sample(dados,6,FALSE) #A surpresinha da Megasena faz esse comando
para você!
```

2.8 ORDENAR OS DADOS

As funções **sort()**, **order()** e **rank()** permitem organizar e manipular dados com base em suas posições.

```
exemplo<-sample(1:100,10) #Cria o objeto exemplo com 10 observações aleatórias extraídas da sequência de 1 a 100, sem reposição.
```

```
list(exemplo)
```

```
> exemplo<-sample(1:100,10)
```

```
> exemplo
```

```
[1] 18 61 15 23 14 62 30 28 71 89
```

```
sort(exemplo) #Coloca os dados em ordem crescente.
```

```
> sort(exemplo)
```

```
[1] 14 15 18 23 28 30 61 62 71 89
```

```
sort(exemplo, TRUE) #Coloca os dados em ordem decrescente.
```

```
> sort(exemplo, TRUE)
```

```
[1] 89 71 62 61 30 28 23 18 15 14
```

```
order(exemplo) #Coloca os dados em ordem crescente pela posição do elemento no objeto exemplo.
```

```
> order(exemplo)
```

```
[1] 5 3 1 4 8 7 2 6 9 10
```

Neste exemplo acima, o primeiro valor na ordem crescente é o 14, que representa o 5º elemento do objeto original exemplo. O segundo valor na ordem crescente é 15, 3º elemento do objeto exemplo, e assim por diante.

```
order(exemplo, decreasing=TRUE) ##Coloca os dados em ordem decrescente pela posição do elemento no objeto exemplo.
```

```
> order(exemplo, decreasing=TRUE)
```

```
[1] 10 9 6 2 7 8 4 1 3 5
```

Note que na ordem decrescente, o primeiro valor é 89, 10ª observação do objeto exemplo. O segundo valor é 71, 9º elemento do objeto exemplo. O terceiro valor é 62, 6º elemento do objeto exemplo, e assim por diante.

```
rank(exemplo) # Diz qual a posição na ordem crescente de cada valor do objeto exemplo.
```

```
> rank(exemplo)
```

```
[1] 3 7 2 4 1 8 6 5 9 10
```

O primeiro valor do objeto exemplo é 18, ele está na 3ª posição na ordem crescente. O segundo valor do objeto exemplo é 61, ele está na 7ª posição na ordem crescente. O terceiro valor do objeto exemplo é 15, ele está na 2ª posição na ordem crescente, e assim por diante.

2.9 COMANDOS DE LÓGICA

As funções de lógica são ferramentas fundamentais em R, utilizadas para realizar testes condicionais, tomar decisões e manipular dados de maneira eficiente. Elas permitem avaliar condições, combinar ou inverter valores lógicos, e retornam valores como TRUE ou FALSE. Essas funções são amplamente utilizadas em estruturas de controle, como loops e condicionais, além de serem indispensáveis em análises de dados. A seguir, exploraremos as principais funções de lógica disponíveis no R, como `ifelse()`, `all()`, `any()`, `is.na()` e operadores lógicos, e aprenderemos como utilizá-las em diferentes cenários.

Os operadores de comparação são utilizados para comparar elementos de vetores ou objetos no R, retornando valores lógicos (TRUE ou FALSE). Abaixo estão alguns exemplos de como eles funcionam:

```
x <- c(1, 2, 9, 4, 5)
y <- c(1, 2, 6, 7, 8)
x > y # Compara elemento por elemento; retorna TRUE para
comparações verdadeiras, FALSE caso contrário.
x >= y # Retorna TRUE para valores de x maiores ou iguais aos de y.
x < y # Retorna TRUE para valores de x menores que os de y.
x == y # Retorna TRUE para elementos de x iguais aos de y.
x != y # Retorna TRUE para elementos de x diferentes de y.
```

É possível utilizar as condições lógicas para filtrar ou selecionar elementos, considere o exemplo a seguir:

```
a <- c(2, 4, 6, 8, 10, 12, 14, 16, 18, 20)
a > 10 # Retorna um vetor lógico com TRUE para valores maiores que 10
e FALSE caso contrário.
which(a > 10) # Identifica as posições dos valores em 'a' que são maiores
que 10.
a[6] # Seleciona o sexto elemento do vetor 'a'.
```

```
a[c(6:10)] # Seleciona os elementos de 'a' que estão entre as posições 6 e 10.
```

```
a[which(a >= 14)] # Retorna os valores de 'a' que são maiores ou iguais a 14.
```

O R oferece funções como `ifelse()` e `switch()` para executar ações com base em condições lógicas. Veja como elas funcionam

```
salarios<-c(1000, 400, 1200, 3500, 380, 3000, 855, 700, 1500, 500)
ifelse(salarios<1000,"pouco","muito") #três argumentos: teste lógico, resposta caso verdade e, resposta caso falso.
```

```
semaforo <- "verde"
switch(semaforo, verde = "continua", amarelo = "acelera", vermelho = "para")
>continua
```

No exemplo acima, dependendo da cor do semáforo, é retornada a ação correspondente: "continua" para verde, "acelera" para amarelo e "para" para vermelho.

O R fornece funções que verificam condições de forma geral sobre um conjunto de valores ou identificam valores ausentes. Aqui exploramos as funções `all()`, `any()` e `is.na()`. A função `all()` verifica se todos os elementos de um vetor ou condição são verdadeiros. Se todos os valores forem TRUE, ela retorna TRUE; caso contrário, retorna FALSE.

```
x <- c(TRUE, TRUE, TRUE)
y <- c(TRUE, FALSE, TRUE)
all(x) # Retorna TRUE porque todos os valores em 'x' são TRUE.
all(y) # Retorna FALSE porque nem todos os valores em 'y' são TRUE.
# Exemplo prático:
idades <- c(20, 25, 30, 18)
all(idades >= 18) # Verifica se todas as idades são maiores ou iguais a 18.
>TRUE
```

A função `any()` verifica se pelo menos um elemento de um vetor ou condição é verdadeiro. Se algum valor for TRUE, ela retorna TRUE; caso contrário, retorna FALSE.

```
x <- c(FALSE, FALSE, TRUE)
```

```
y <- c(FALSE, FALSE, FALSE)
any(x) # Retorna TRUE porque pelo menos um valor em 'x' é TRUE.
any(y) # Retorna FALSE porque nenhum valor em 'y' é TRUE.
# Exemplo prático:
valores <- c(-5, -10, 0, 15, 20)
any(valores > 0) # Verifica se há pelo menos um valor positivo no vetor.
>TRUE
```

A função `is.na()` identifica valores ausentes (NA) em vetores ou objetos. Ela retorna um vetor lógico indicando quais posições contêm NA.

```
x <- c(1, 2, NA, 4, NA)
is.na(x) # Retorna TRUE para as posições onde há NA e FALSE para as
demais.
# Resultado: [FALSE, FALSE, TRUE, FALSE, TRUE]
# Usando em combinação com outras funções:
x[!is.na(x)] # Retorna apenas os valores que não são NA.
>1 2 4
```



CAPÍTULO 03

3 GRÁFICOS

O programa R permite criar diversos tipos de gráficos e possuem particular peculiaridade devido as diversas opções de comandos que permitem editar gráficos. Os principais gráficos são: barplot (gráfico de barras), pie (gráfico de pizza) e plot (gráfico de dispersão).

3.1 GRÁFICO DE BARRAS

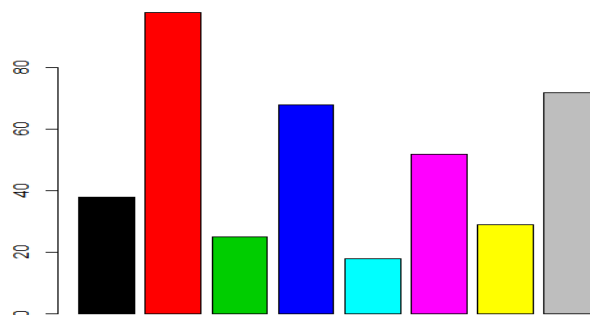
A função `barplot()` cria um gráfico de barras. No exemplo abaixo, geramos aleatoriamente 8 valores entre 10 e 100 e, em seguida, os exibimos em um gráfico de barras:

```
barplot(sample(10:100, 8))
```

Alguns comandos podem ser inseridos na função `barplot`, como opções de personalização do gráfico. O comando `col=1` significa as barras assumirem a cor preta, `col=2` significa cor vermelha, `col=3` cor verde. Para cada uma das 8 barras do exemplo assumir uma cor diferente, e para você observar o valor referente de cada cor, faça:

```
barplot(sample(10:100,8), col=1:8)
```

Figura 5. função barplot



Fonte: Elaboração própria

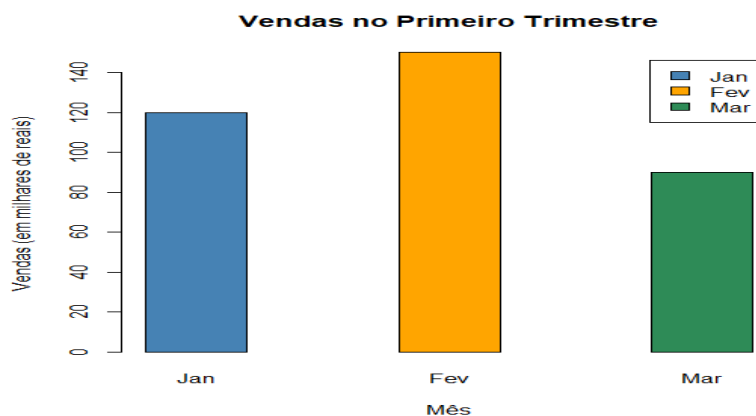
Faça o exercício com uma amostra maior e perceberá que a partir da cor 9 as cores vão se repetir, para criar combinações de cores diferentes, faça:

```
barplot(sample(10:100,20), col = rainbow(20), space= 1.5)) # A opção space aumenta o espaçamento entre as barras.
```

A seguir, um exemplo para ilustrar o uso de **barplot()**, vamos supor que desejamos comparar as vendas mensais de uma empresa em três meses diferentes (por exemplo, no primeiro trimestre). Esse tipo de gráfico de barras é muito comum em análises empresariais.

```
# Vetor com as vendas (em milhares de reais) de uma empresa em três meses
vendas <- c(120, 150, 90)
# Nomeia cada posição do vetor com o respectivo mês
names(vendas) <- c("Jan", "Fev", "Mar")
# Cria o gráfico de barras
barplot(
vendas,
col = c("steelblue", "orange", "seagreen"), # Três cores para as barras
space = 1.5,                               # Espaçamento entre as barras
main = "Vendas no Primeiro Trimestre",     # Título do gráfico
xlab = "Mês",                              # Rótulo do eixo X
ylab = "Vendas (em milhares de reais)",    # Rótulo do eixo Y
legend=names(vendas)                       #Legenda)
```

Figura 6. Vendas no primeiro trimestre



Fonte: Elaboração própria

A seguir, um exemplo com duas categorias. As empresas A e B disputam

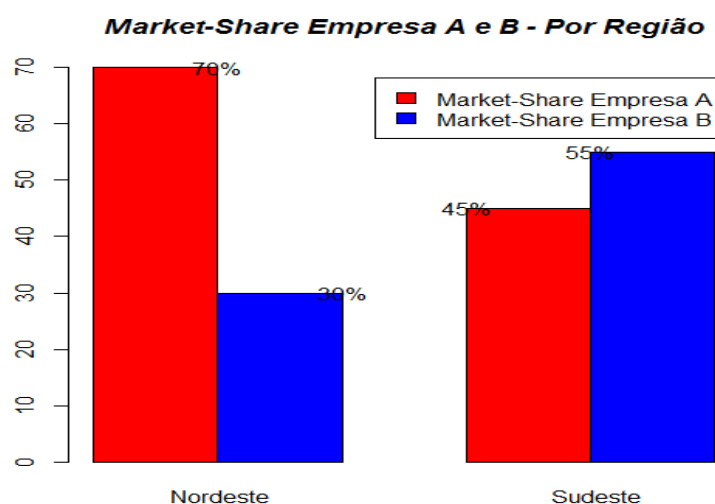
dois mercados. No Nordeste, a empresa A possui 70% do Market-Share, enquanto a empresa B detém 30%. Já no Sudeste, a empresa A possui 45% do Market-Share, enquanto a empresa B detém 55%. Para cada Região, temos as duas mesmas categorias, Market-Share da empresa A e B. Representando graficamente:

```

nordeste<-c(70,30) #Cada objeto será uma torre no barplot quando
nomeá-lo
sudeste<-c(45,55) ##Cada objeto será uma torre no barplot quando
nomeá-lo
names(nordeste)<-c("Nordeste") #Espaços para centralizar
entre as duas colunas
names(sudeste)<-c("Sudeste")
brasil<-c(nordeste,sudeste) #Juntar os dois objetos em um terceiro
chamado brasil. Isso é necessário para criar duas torres dentro de um mesmo
barplot.
barplot(brasil, col=c(2,4,2,4), legend=c("Market-Share Empresa
A","Market-Share Empresa B"),space=c(0,0,1,0)) #Space para juntar e separar
colunas manualmente
text(brasil, c("70%","30%","45%","55%"), col = 1, xpd=TRUE) #Execute
com o gráfico aberto para incluir textos
title(main = "Market-Share Empresa A e B - Por Região", font.main = 4)

```

Figura 7. Market-share Empresa A e B – por Região



Fonte: Elaboração própria

3.2 GRÁFICO DE PIZZA

Vamos agora ilustrar um cenário em que três empresas (A, B e C) competem em um mercado, com participações de 20%, 30% e 50%. Queremos criar um **gráfico de pizza** que apresente cores, legenda e título personalizados.

```
Pie(c(30,20,50)) #Faz o gráfico de pizza
```

Vamos agora ilustrar um cenário em que três empresas (A, B e C) competem em um mercado, com participações de 20%, 30% e 50%. Queremos criar um **gráfico de pizza** que apresente cores, legenda e título personalizados.

```
# Passo 1: Crie e nomeie o vetor de percentuais
percentual <- c(20, 30, 50)
names(percentual) <- c("Empresa A", "Empresa B", "Empresa C")

# Passo 2: Construa o gráfico de pizza
pie(
  percentual,
  col = c(2, 5, 7),    # Define as cores (2 = vermelho, 5 = magenta, 7 = cinza
                        # claro, etc.)
  density = 10,        # Ajusta a densidade das linhas (efeito de hachura)
  labels = c("20%", "30%", "50%") # Rótulos exibidos diretamente nas
                                # fatias)

# Passo 3: Adicione uma legenda
legend(
  1, 1,                # Posição da legenda (x = 1, y = 1), canto superior direito
  names(percentual),    # Texto da legenda (nomes atribuídos ao vetor)
  col = c(2, 5, 7),     # Mesmas cores usadas no gráfico
  pch = 1              # Define o estilo dos símbolos na legenda)

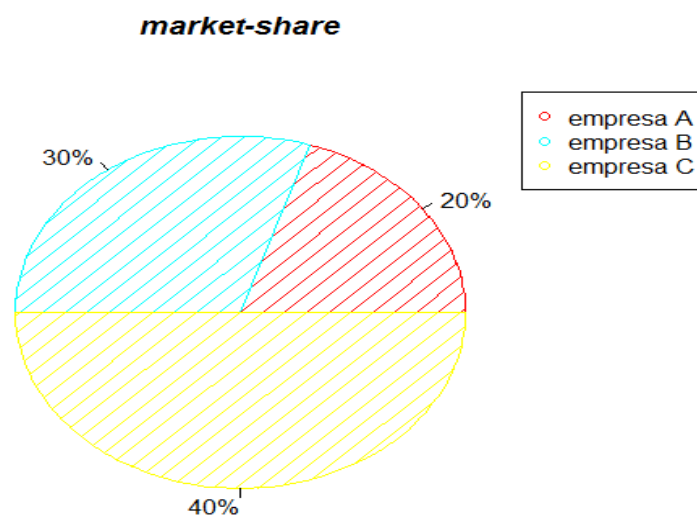
# Passo 4: Insira um título para o gráfico
title(main = "Participação de Mercado", font.main = 4)
```

3.2.1 Observações importantes

1. **names(percentual)**: Permite atribuir nomes a cada fatia da pizza. Se desejarmos que o gráfico use automaticamente esses nomes como rótulos, poderíamos omitir labels em pie().

2. **col**: Define as cores das fatias. No R, números baixos (como 1, 2, 3 etc.) se referem a uma paleta básica (1 = preto, 2 = vermelho, 3 = verde, 4 = azul, 5 = magenta, 7 = cinza claro, etc.). Para uma paleta mais rica, você pode usar nomes como "red", "blue", "green" ou códigos hexadecimais "#RRGGBB".
3. **density**: Controla a densidade das linhas de hachura. Se preferir fatias "sólidas", você pode remover esse argumento ou usar `density = NULL`.
4. **labels**: Texto exibido **dentro** das fatias. Se omitir este argumento, o R usa os nomes (se houver) ou índices do vetor.
5. **legend(1, 1, ...)**: Controla a posição da legenda no gráfico. As coordenadas (1, 1) posicionam a legenda na parte superior direita. Já coordenadas negativas, como (-1, 1), a moveriam para o canto superior esquerdo.
6. **pch**: Define o estilo dos símbolos na legenda. Existem vários tipos de símbolos, de 0 a 25 (e além). Por exemplo, `pch = 16` produz círculos preenchidos.
7. **title()**: Adiciona ou modifica o título do gráfico já existente. O argumento `font.main` controla o estilo da fonte.

Figura 8. Market-share



Fonte: Elaboração própria

Caso o objeto que você queira inserir possua dados com valores absolutos, a função `pie` calcula automaticamente os valores relativos. Por exemplo:

```
pie(c(100,200,150)) #Execute e note que a função pie atribui uma fatia referente ao valor percentual de cada valor total.
```

3.3 GRÁFICO DE DISPERSÃO

Gráficos de dispersão também podem ser realizados com bastante facilidade no programa R, através do comando `plot(y,x)`. Inicialmente, criamos dois objetos de mesma dimensão.

```
x<-c(110,90,115,70,100,90,60,85,50,75,70,45)
y<-1:12
list(x,y)
plot(x,y)
plot(x,y, xlab="Variável Explicativa", ylab="Variável Dependente",
main="Gráfico de Dispersão", pch=1)
```

As opções `xlab`, `ylab` renomeiam os eixos `x` e `y` respectivamente, a opção `main` insere o título do gráfico, a opção `pch` refere-se ao tipo de ponto para cada par ordenado, teste outras numerações para `pch`.

Com o gráfico de dispersão aberto, insira linhas na média de cada variável, com cores diferentes.

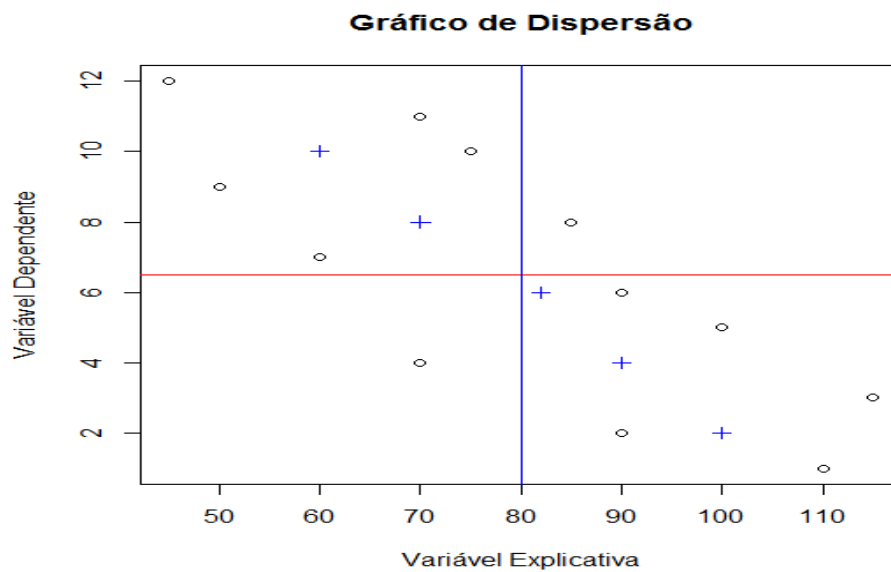
```
abline(h=mean(y), v=mean(x), col=c(2,4)) # h=mean(y) será uma linha horizontal na média de y e v=mean(x) será uma linha vertical na média de x.
```

Para inserir pontos neste gráfico, ainda com o gráfico aberto, crie um novo conjunto de pares ordenados e posteriormente use a função `points`.

```
v<-c(60,70,90,85,100)
w<-c(10,8,4,6,2)
points(v,w, col="blue", pch=3)
```

Observe a ordem de entrada de `v` e `w`, obedecendo a ordem de entrada de `x` e `y` na função `plot`. Ao final de todas as rotinas, deverá obter o seguinte gráfico.

Figura 9. Gráfico de Dispersão

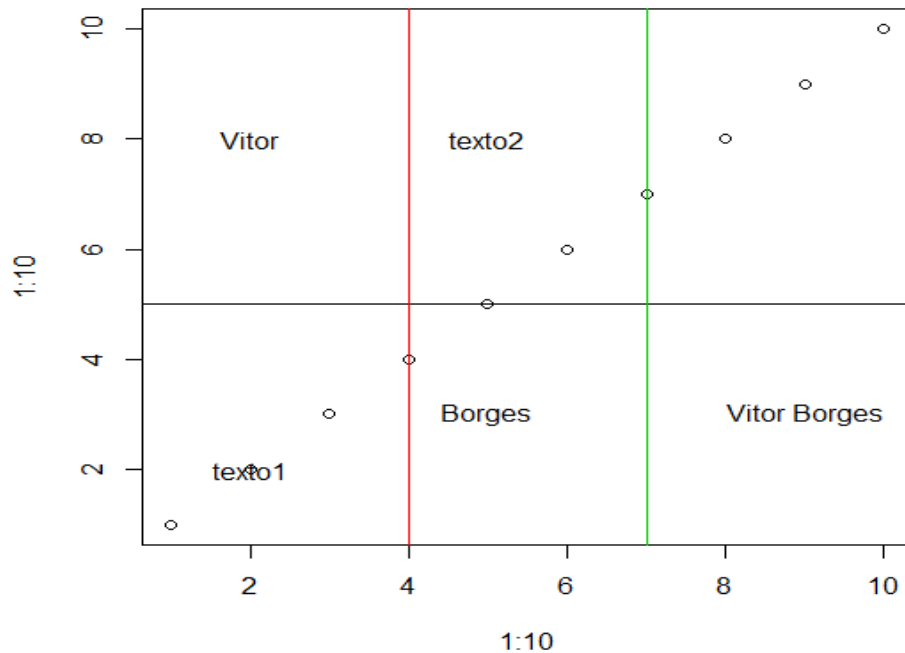


Fonte: Elaboração própria

Para inserir texto no gráfico use a função `text(coox,cooy,"texto")`. Os dois primeiros argumentos da função indicam as coordenadas x e y, respectivamente, no terceiro argumento o texto que deseja inserir. A função `text` deverá ser executada com o gráfico aberto. Faça o seguinte exercício.

```
plot(1:10,1:10) #Dispersão linear
abline(v=c(4,7),h=5, col=1:3) #Cria linhas verticais nas posições 4 e 7 do
eixo x e cria linha horizontal na posição 5 do eixo y.
text(2,8, "Vitor") #Na coox=2 e cooy=8 insere o nome Vitor.
text(5,3, "Borges") #Na coox=5 e cooy=3 insere o nome Borges.
text(9,3, paste("Vitor","Borges")) #função paste cria nome composto na
posição indicada.
text(c(2,5), c(2,8), c("texto1","texto2")) #Note neste comando que os pares
ordenados (x,y) são (2,2) e (5,8).
```

Figura 10. função text



Fonte: Elaboração própria

3.4 CRIANDO JANELAS NO GRÁFICO

Para dividir um mesmo gráfico em janelas, use o comando: `par(mfrow=c(nl,nc))`. Onde, `nl` e `nc` indicam o número de linhas e número de colunas, respectivamente. Se o interesse é criar dois gráficos, um do lado do outro, então, torna-se necessário inserir uma linha e duas colunas neste argumento. Para visualizar, execute as rotinas:

```
par(mfrow=c(1,2))
plot(1)
plot(2)
par(mfrow=c(2,2)) #Divide a janela em 4 gráficos, de duas linhas e duas
colunas
plot(1)
plot(2)
plot(3)
plot(4)
```

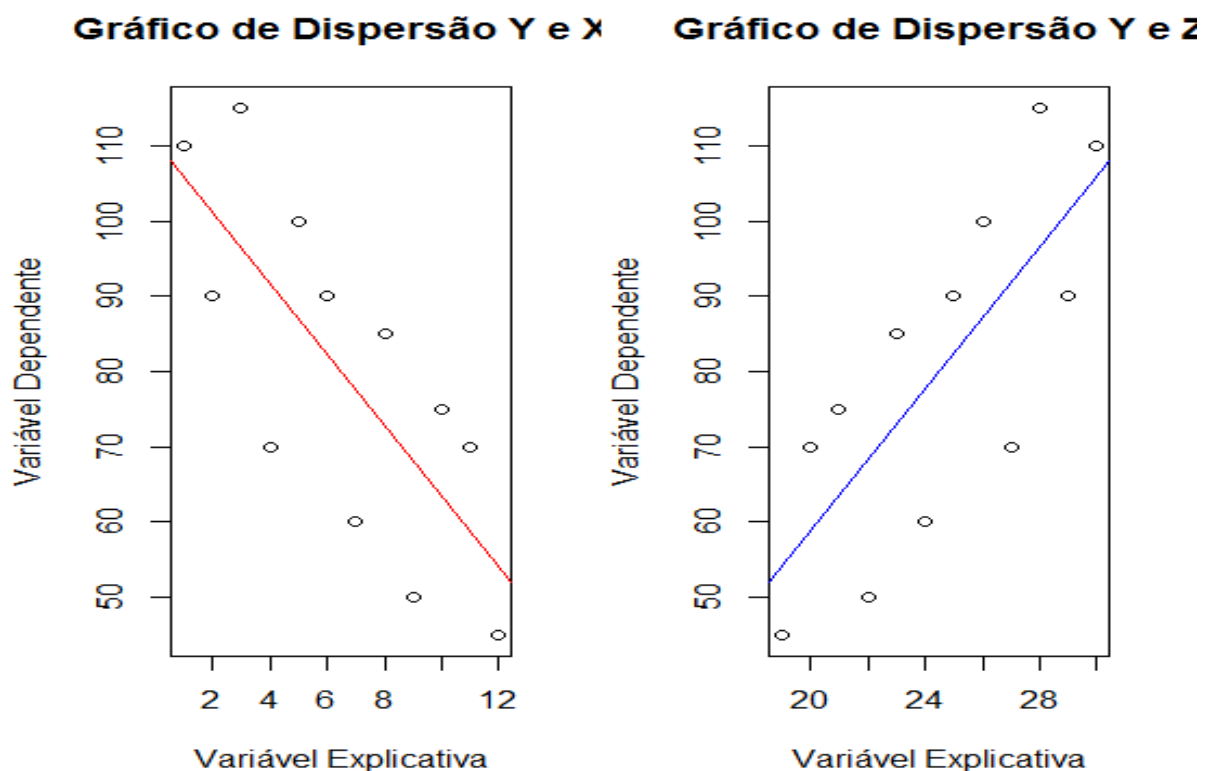
Considere o seguinte exercício:

```

y<-c(110,90,115,70,100,90,60,85,50,75,70,45)
x<-1:12
z<-30:19
par(mfrow=c(1,2))
plot(x,y, xlab="Variável Explicativa", ylab="Variável Dependente",
main="Gráfico de Dispersão Y e X", pch=1)
resultado1<-lm(y~x)
summary(resultado1)
abline(resultado1, col=2)
plot(z,y, xlab="Variável Explicativa", ylab="Variável Dependente",
main="Gráfico de Dispersão Y e Z", pch=1)
resultado2<-lm(y~z)
summary(resultado2)
abline(resultado2, col=4)

```

Figura 11. Gráfico de dispersão Y e X



Fonte: Elaboração própria

Vale ressaltar algumas observações acerca do exercício acima:

1. Os objetos x , y e z foram criados com o mesmo número de elementos.
2. O comando `lm(y~x)` executa a regressão simples $y_i = b_0 + b_1x_i + e_i$.
3. Observe que a ordem de entrada da função `plot` é diferente da ordem de entrada da função `lm`.
4. Você pode salvar o resultado de uma regressão em um objeto para posteriormente executar o comando do sumário econométrico da regressão sobre o objeto. Caso não queira salvar o sumário econométrico, pode realizar de forma direta `summary(lm(y~x))`.
5. Entretanto, foi interessante salvar as regressões nos objetos chamados `resultado1` e `resultado2`, para posteriormente criar a reta de regressão no gráfico, através da função `abline`.
6. Observe no sumário econométrico a concordância do sinal do β com o gráfico gerado.
7. A regressão múltipla seria `lm(y~x+z)`, faça novo `summary` sobre a regressão múltipla.



CAPÍTULO 04

MATRIZES

As matrizes são estruturas de dados bidimensionais amplamente utilizadas em R para armazenar elementos de forma organizada em linhas e colunas. Elas são ideais para operações matemáticas e algébricas, pois permitem realizar cálculos eficientes com múltiplos elementos simultaneamente. No R, as matrizes podem ser criadas usando a função `matrix()`, `dim()` ou convertendo vetores para o formato matricial com os comandos `cbind()` ou `rbind()`. Além disso, é possível realizar operações como transposição, multiplicação e extração de elementos com facilidade. Nesta seção, exploraremos os conceitos básicos de matrizes e as principais funções associadas a elas.

4.1 CRIANDO MATRIZES COM `CBIND()` E `RBIND()`

Crie os objetos e construa a matriz a partir de vetores:

```
a <- c(1, 3, 5, 4)
b <- c(11, 15, 13, 17)
c <- c(0, 2, 0, 1)
d <- c(5, 2, 0, 1)
# Criando uma matriz unindo vetores por colunas
w1 <- cbind(a, b, c, d)
# Criando uma matriz unindo vetores por linhas
w2 <- rbind(a, b, c, d)
w1 # Exibe a matriz criada por colunas
w2 # Exibe a matriz criada por linhas
```

4.2 OPERAÇÕES COM MATRIZES

A seguir, as principais funções para operações de matrizes

```
det(w1) #determinante
```

```

solve(w1) #inversa
ncol(w1) #nº de colunas
nrow(w1) #nº de linhas
w1%*%w2 #multiplicação de matrizes w1(4x4) x w2(4x4)
t(w1) # transposta de w1
I10 <- diag(rep(1,10)) #Matriz Identidade
I10

```

4.3 CRIANDO MATRIZES COM A FUNÇÃO DIM() E MATRIX()

No R, podemos criar e manipular matrizes de diferentes maneiras. Vamos explorar essas abordagens utilizando as funções `dim()` e `matrix()`, além de aprender como realizar operações matemáticas e consultar elementos específicos. Vamos criar o objeto `vendas` com 12 elementos:

```

vendas <- c(45, 23, 66, 77, 33, 44, 56, 12, 78, 23, 78, 90)
vendas
dim(vendas)<-c(3,4) #Organiza o objeto vendas em uma matrix 3x4,
preenchimento por coluna.
vendas #Consultar Matriz
rm(vendas) #Remove vendas

```

Já função `matrix()` permite criar matrizes diretamente, especificando os dados, o número de linhas (`nrow`) e o número de colunas (`ncol`). A função `matrix()` tem a seguinte ordem de argumentos: `matrix(dados, nrow, ncol)`. O preenchimento da matriz é por colunas, de cima pra baixo.

```

vendas <- matrix(c(45, 23, 66, 77, 33, 44, 56, 12, 78, 23, 78, 90), 3, 4)
vendas #recriou a mesma matriz vendas anterior

```

Outros exemplos de matrizes:

```

matrix(1,4,4) #matriz de 1s 4x4
matrix(1:16,4,4) #Observar preenchimento vertical das colunas na matriz
4x4
t(matrix(1:16,4,4)) #Transposta
matrix(1:16,4,4,byrow=T) #Transposta
det(t(matrix(1:16,4,4)))

```

4.4 GRÁFICOS COM MATRIZES

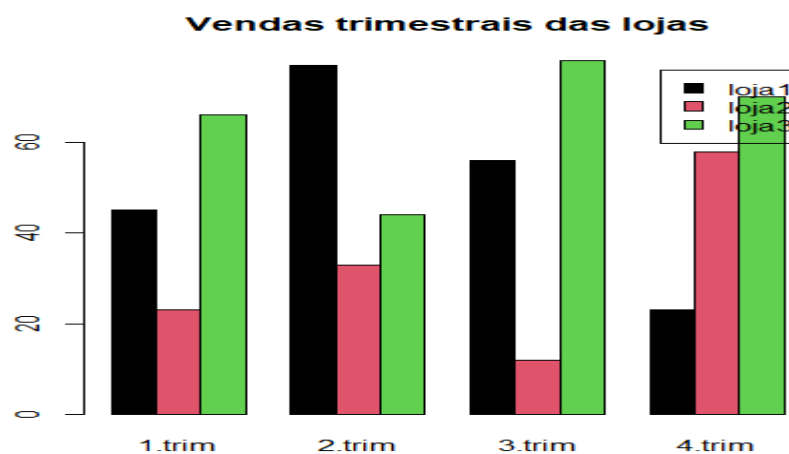
Importante inserir nomes nas linhas e colunas da matriz, isso facilita na apresentação gráfica.

```
rownames(vendas) <- c("loja1", "loja2", "loja3")
colnames(vendas) <- c("1.trim", "2.trim", "3.trim", "4.trim")
vendas #Matriz com os nomes das linhas e colunas
>
1.trim 2.trim 3.trim 4.trim
loja1   45   77   56   23
loja2   23   33   12   78
loja3   66   44   78   90
```

Observe agora como a linguagem matricial facilita na apresentação gráfica do `barplot()`.

```
barplot(vendas, beside=TRUE, col=1:3, legend=rownames(vendas),
main="Vendas trimestrais das lojas")
```

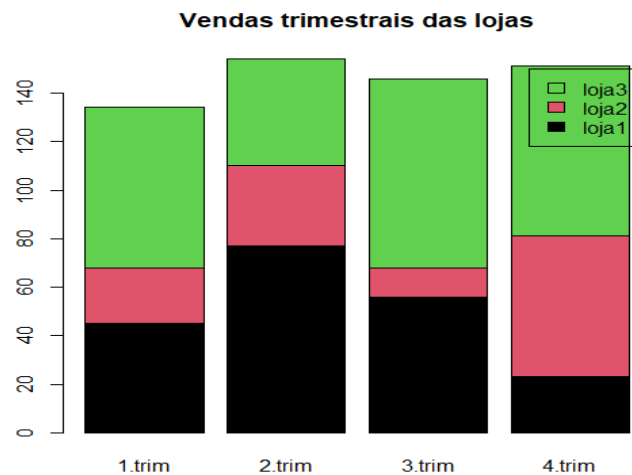
Figura 12. Vendas trimestrais das lojas



Fonte: Elaboração própria

```
barplot(vendas, beside=FALSE, col=1:3, legend=rownames(vendas),
main="Vendas trimestrais das lojas")
```

Figura 13. Vendas trimestrais das lojas



Fonte: Elaboração própria

4.5 CONSULTANDO ELEMENTOS DA MATRIZ

Para fazer consulta ou recortes de matrizes usa-se o colchete []. Veja exemplos:

```
vendas[2,2] #Consulta a segunda linha e segunda coluna
vendas[1:2,] #consulta linha 1 e 2 de todas as colunas
vendas[,3:4] #consulta todas as linhas das colunas 3 e 4
vendas["loja2", "2.trim"]
vendas[-2,2] #Mostra a segunda coluna exceto segunda linha
vendas[, -4] #Mostra todas as linhas exceto a quarta coluna.
vendas[1, -c(2, 4)] #Mostra primeira linha exceto segunda e quarta
coluna.
```

4.6 EXERCÍCIOS PROPOSTOS

1. Dada a matriz:

```
m1 <- matrix(c(45, 23, 66, 77, 33, 44, 56, 12, 78, 23), 2, 5)
```

Multiplique todos os elementos de m1 pelo escalar 3. Qual será o valor do elemento na posição (2,4) após a multiplicação?

2. Crie uma matriz m2 com 10 linhas e 5 colunas, preenchida apenas com o número 10, utilizando a função matrix()

3. Multiplique a matriz m1 pela transposta da m2.

4. Use a função `rbind()` para unir a primeira linha de `m1` e a quinta linha de `m2` em uma nova matriz chamada `m3`. Qual será o valor do elemento na posição (2,4) de `m3`?
5. Utilize as matrizes `m1` e `m3` criadas anteriormente e realize uma operação de soma de matrizes. É possível?
6. Considerando um sistema linear $AX = B$, usando a função `solve()`, encontre o valor do vetor `X`

```
A <- matrix(c(2, 0.4, 5, -0.9), 2, 2)
```

```
B <- c(4, 5)
```

4.7 ESTIMADOR BETA CHAPÉU

O estimador $\hat{\beta}$ é a base para a obtenção dos coeficientes em uma regressão linear. Ele é calculado a partir da fórmula matricial $\hat{\beta} = (X'X)^{-1}X'Y$, onde `X` representa a matriz de regressores (variáveis independentes e intercepto) e `Y` é a matriz da variável dependente. Esse método minimiza a soma dos quadrados dos resíduos, fornecendo os valores que melhor ajustam o modelo aos dados. Nesta seção, exploramos o cálculo manual do $\hat{\beta}$ usando operações matriciais no R, comparando os resultados com a função automatizada `lm()`. Considere os seguintes vetores com 14 elementos cada.

```
y<-c(120, 150,145, 170,165, 190, 210, 240, 220, 250, 245, 270, 320, 350)
x<-c(320, 300, 180, 280, 250, 190, 280, 170, 230, 200, 150, 120, 130, 100)
z<-rep(c(50,100,150), c(5,5,4))
```

Construindo as matrizes `X` e `Y`

```
co<-rep(1,14) #Cria um objeto para representar (posteriormente) a coluna
de 1's da matriz X, referente ao intercepto da regressão.
```

```
co
```

```
X<-cbind(co,x,z) #Cria a matriz X de regressores
```

```
X
```

```
Y<-matrix(y,14,1) #Cria a matriz Y representando a variável dependente
da regressão.
```

```
Y
```

Para calcular o Beta Chapéu

```
beta<-solve(t(X)%*%X)%*%t(X)%*%Y
```

Para conferir o resultado, utilize o comando lm

```
resultado<-lm(y~x+z)
```

```
summary(resultado)
```

Para encontrar os desvios padrão dos parâmetros, precisamos encontrar a Matriz de Variância e Covariância, e extrair a raiz quadrada dos termos da diagonal principal.

```
erro<-Y-X%*%beta #e=Y-X*beta
```

```
s2<-(t(erro)%*%erro)/11 # s2=e'e/(n-k)
```

```
s2
```

```
mvc<-s2[1,1]* solve(t(X)%*%X) #Matriz variância-covariância s2(X'X)-1
```

```
mvc
```

#Para extrair da matriz MVC o erro-padrão de cada estimador, faça:

```
ep1<-mvc[1,1]^0.5
```

```
ep2<-mvc[2,2]^0.5
```

```
ep3<-mvc[3,3]^0.5
```

```
ep1
```

```
ep2
```

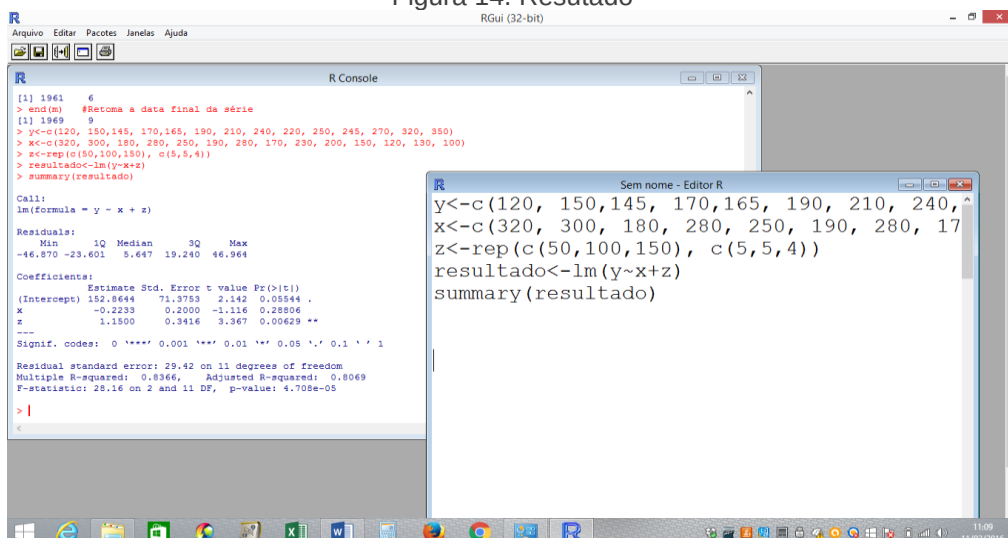
```
ep3
```

Para conferir o resultado da regressão múltipla, faça:

```
resultado<-lm(y~x+z)
```

```
summary(resultado)
```

Figura 14. Resultado



Fonte: Elaboração própria

Desafio!

Encontre o valor de $R^2 = \frac{\hat{\beta}'X'Y' - n\bar{Y}^2}{Y'Y - n\bar{Y}^2}$



CAPÍTULO 05

CRIANDO FUNÇÕES

No R, as funções e estruturas de controle desempenham papéis fundamentais para organizar e automatizar processos analíticos. As **funções**, criadas com `function()`, permitem encapsular blocos de código em unidades reutilizáveis, otimizando tarefas repetitivas ou específicas. Já as **estruturas de controle**, como o `for()` e a condicional `if()`, são ferramentas indispensáveis para gerenciar fluxos de execução. O `for()` é utilizado para controlar a iteração sobre observações, enquanto o `if()` avalia condições lógicas para tomar decisões dinâmicas durante a execução do código. Nesta seção, exploraremos como usar essas ferramentas para criar scripts eficientes e bem estruturados.

5.1 COMANDO FUNCTION

O comando **function**, para gerar funções no R, possui a seguinte lógica de construção:

nome da função <- function(lista de argumentos) {Instruções}

Exemplos:

Exemplo 1: Criar uma função simples, apenas para executar uma operação aritmética

```
valor<-function(x){  
  9/5*x+32  
}  
valor(2)  
>35.6  
valor(5)  
>41  
valor(10)  
>50  
valor(c(2,5,10))  
>35.6 41 50
```

Exemplo 2: Objetos criados no corpo da instrução, se a resposta ficar salva em um objeto, usar a função `return()` para a função retornar o valor pretendido.

```
f<-function(x){  
  z<-56.3  
  y<-34  
  a<-(x/4)*z*y  
  return(a)  
}  
f(21)  
> 10049.55
```

Exemplo 3: Dois argumentos na lista de argumentos da função, usando a função `sample`.

```
jogar.moeda<-function(x,n){  
  sample(x,n,TRUE) #TRUE com reposição  
}  
jogar.moeda(1:30,10)  
moeda<-c("cara","coroa")  
jogar.moeda(moeda,10)  
jogar.moeda(moeda,100)  
jogar.moeda(moeda,1000)  
table(jogar.moeda(moeda,1000)) #table é a distribuição de frequência  
dado<-1:6  
jogar.moeda(dado,20)  
table(jogar.moeda(dado,20))
```

#Exercício de classe

#Criar a função média

```
dados<-rnorm(100,50,10) #100obs com média 50 e variância 10  
mean(dados) # A função a ser criada tem que conferir com esse resultado  
média<-function(x){  
  sum(x)/length(x)  
}  
média(dados)  
média<-function(x){
```

```

    d<-sum(x)/length(x)  #Salvando em um objeto para chamar a função
return()
    return(d)
}
média(dados)

```

5.2 CONTROLE DE OBSERVAÇÕES: COMANDO FOR

O laço `for()` é uma das estruturas de repetição mais usadas no R para iterar sobre sequências de valores, como vetores, listas ou matrizes. Ele permite executar um bloco de código para cada elemento de uma sequência de maneira controlada e ordenada. Cada iteração do laço processa um elemento da sequência, atribuindo-o a uma variável temporária que pode ser usada dentro do bloco de execução.

A estrutura básica do `for()` é:

`for(i in 1:n) {Código a ser executado em cada iteração}`

Exemplo 1:

```

resu<-numeric(5) #objeto vazio para ser preenchido com for()
resu
for(i in 1:5){
  resu[i]<-i^2
}
Resu # Retorna os quadrados dos números de 1 a 5

```

Exemplo 2: Sequência de Fibonacci: 1 1 2 3 5 8 13 21 34 55 89 144

```

fibonacci<-numeric(12)
fibonacci
fibonacci[1:2]<-1
fibonacci
for(i in 3:12){
  fibonacci[i]<-fibonacci[i-1]+fibonacci[i-2]
}
fibonacci

```

Exemplo 3: Preenchimento de uma matriz, fazer n jogos “surpresinha” da megasena

```

megasena<-function(njogos){
  numeros<-matrix(0,6,njogos) #matriz 6xn
  for(i in 1:njogos){
    numeros[,i]<-sample(1:60,6,FALSE)
  }
  return(numeros)
}
megasena(10) #retoma 10 jogos surpresinha, cada jogo em uma coluna
megasena(100) #retoma 100 jogos surpresinha, cada jogo em uma coluna

```

Exemplo 4: Fazer o Valor Presente Líquido (VPL) do seguinte Fluxo de Caixa

```

fluxo.caixa<-c(-1000,200,250,300,350,500)
r<-0.1 #taxa de desconto
vpl<-function(x,r){
  y<-x[-1] #isolar o fluxo de benefícios no objeto y
  z<-numeric(5) #criar um objeto para salvar os benefícios descontados
  for(i in 1:5){
    z[i]<-y[i]/((1+r)^i)
  }
  vp<-sum(z) #somar os benefícios descontados
  vp+x[1] #somar os benefícios descontados com o investimento, VPL>0
  projeto viável
}
vpl(fluxo.caixa,0.1)

```

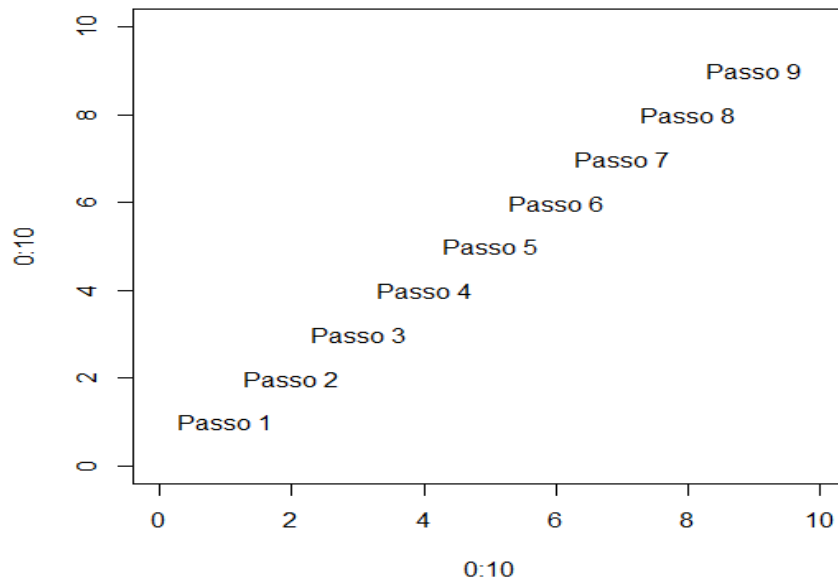
Exemplo 5: Usar comando for em uma estrutura de plot

```

plot(0:10,0:10, type="n")
for(i in 1:9){
  text(i,i, paste("Passo", i))
}
plot(0:10,0:10, type="n")
for(i in 1:9){
  text(i,i, paste("Passo", i))
  Sys.sleep(1) ## retarda os passos em 1 segundo
}

```

Figura 15. Comando For



Fonte: Elaboração própria

Exercício de Classe: Fazer njogos da Lotofácil

```
lotofacil<-function(njogos){  
  numeros<-matrix(NA,15,njogos)  
  for(i in 1:njogos){  
    numeros[,i]<-sample(1:25,15)}  
  return(numeros)}  
lotofacil(100) #Resultado de 100 jogos no console
```

5.3 COMANDOS IF, ELSE, IFELSE, WHILE E CONDIÇÕES && E ||

Os comandos if, else e ifelse no R são estruturas de controle fundamentais para a tomada de decisões em códigos. Eles permitem que diferentes blocos de código sejam executados com base em condições lógicas.

- O comando if avalia uma condição e executa o código associado apenas se a condição for verdadeira (TRUE).
- O comando else pode ser usado em conjunto com if para especificar o que fazer quando a condição for falsa (FALSE).

- Já o ifelse é uma função vetorizada que aplica condições de forma eficiente a cada elemento de um vetor, retornando resultados diferentes para casos verdadeiros e falsos.
- Esses comandos são essenciais para criar fluxos dinâmicos no código, adaptando a execução às características dos dados ou parâmetros de entrada.

Exemplo 1:

```
x=10
if(x>5){
  print(x)
} #Como x=10 é maior que 5, o comando vai exibir o 10
>10

rm(x) #remover x para novo exemplo
```

Exemplo 2: IF combinado com ELSE faz o teste lógico com resposta caso verdade e caso falso

```
x=4
y=6
if (x>y) {
  paste("o maior numero é:",x)
}else{
  paste("o maior numero é:",y)
}
> "o maior numero é: 6"
```

Exemplo 3: Similarmente, usando IFELSE

```
a=4
b=6
ifelse (a>b , print (paste("O maior número é:",a)) , print (paste("O maior
número é:",b)))
```

```
rm(x) #remover x para novo exemplo
rm(y) #remover y para novo exemplo
```

Exemplo 4: Preenchimento de um objeto vazio, com base em funções lógicas interregido com objetos

```
result<-numeric(6)
x=c(4,6,3,2,7,4)
```

```

y=c(5,7,2,3,6,5)
for(i in 1:6){
  if (x[i]>y[i]) {
    result[i]<-1
  }else{
    result[i]<-0
  }
}
result
> 0 0 1 0 1 0

```

rm(result) #remover objeto para novo exemplo

Exemplo 5: Preenchimento de um objeto vazio, com base em funções lógicas interagido com objetos

```

result<-numeric(14)
result
x=c(4,6,3,2,7,4,5,7,2,3,6,5,7,8,5)
for(i in 2:15){
  if (x[i]>x[i-1]) {
    result[i-1]<-1
  }else{
    result[i-1]<-0
  }
}
result
> 1 0 0 1 0 1 1 0 1 1 0 1 1 0

```

Observe que no corpo da instrução tem x[i-1], então, precisamos começar o controle de observações no índice 2. O objeto original tem 15 observações mas o resultado da interação só poderá gerar 14 resultados.

Exemplo 6: Acrescentando a condições && (E)

```

result<-numeric(13)
result
x=c(4,6,3,2,7,4,5,7,2,3,6,5,7,8,5)
for(i in 2:14){
  if ((x[i]>x[i-1])&&(x[i]>x[i+1])) {

```

```

result[i-1]<-1
}else{
result[i-1]<-0
}
}
result
> 1 0 0 1 0 0 1 0 0 1 0 0 1
cbind(x,c(NA,result,NA)) #visualizar entrada e saída em uma matriz

```

Observe agora que no corpo da instrução tem $x[i-1]$ e $x[i+1]$, então, precisamos começar o controle de observações no índice 2 e terminar no índice 14. O objeto original tem 15 observações mas o resultado da interação só poderá gerar uma Janela de 13 respostas. O comando `cbind` é importante para visualizar os dados originais e gerados pela interação.

Exemplo 7: Acrescentando a condições `||` (OU)

```

result<-numeric(13)
result
x=c(4,6,3,2,7,4,5,7,2,3,6,5,7,8,5)
for(i in 2:(length(x)-1)){
if ((x[i]>x[i-1])||(x[i]>x[i+1])) {
result[i-1]<-1
}else{
result[i-1]<-0
}
}
result
> 1 1 0 1 0 1 1 0 1 1 0 1 1

```

Observe que o exemplo 7 apenas substituiu o `&&` por `||` em relação ao exemplo 6. Porém, substituímos o 14 no controle de observações FOR por `length(x)-1`, que nesse exemplo será também 14, porém é importante construir comandos amplos que possam se adaptar a novos conjuntos de dados.

Exemplo 8: Vejamos agora um exemplo com duas condições na função IF.

```

a=1:10
b=10:20

```



```

if ( length (a) == length (b) && a[1]!=0 ) {
  c = a*b
} else {
  c = 1:100
}

```

Temos aqui dois vetores a e b, com comprimentos diferentes e o primeiro elemento de a é diferente de 0. A condição dada é que o comprimento (length) dos vetores seja igual e o primeiro elemento do vetor a não seja nulo. Como a primeira condição não é verdadeira, a (condição 1 e (&&) condição 2) é falsa, então o vetor c irá receber uma sequência entre 1 e 100.

Exemplo 9: Comando While

A tradução desta função nos ajuda a entendê-la mais: while significa enquanto. Então podemos ler essa função como: Enquanto alguma condição for verdadeira, o código abaixo será repetido. A sintaxe da função while (condição){comandos a serem repetidos}.

```

a=1
while (a <=10 ) {
  print (a)
  a=a+1
}

```

Exemplo 10: Comando While

```

contador <- 5
while (contador > 0) {
  print(paste("Faltam", contador, "segundos..."))
  contador <- contador - 1 # Decrementa o contador
}
print("Tempo esgotado!")

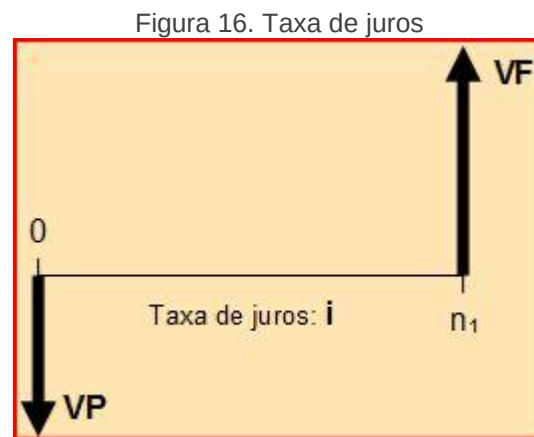
```

5.4 CRIANDO FUNÇÕES FINANCEIRAS

As funções financeiras são ferramentas essenciais para a análise de investimentos, auxiliando na tomada de decisões e na avaliação da viabilidade de projetos. A seguir, são apresentados exemplos implementados em R para calcular Valor Presente (PV), Valor Futuro (FV), Valor Presente Líquido (VPL) e

Taxa Interna de Retorno (TIR).

Um fluxo de caixa representa os desembolsos financeiros ao longo do tempo. O dinheiro tem diferentes valores no tempo, a taxa de juros i é a medida de unificação de valores medidos na mesma unidade temporal. Valor Presente é o tempo 0, valor futuro é o tempo n .



Fonte: Elaboração própria

$$PV = \frac{FV}{(1 + i)^n} \quad (1)$$

$$FV = PV \cdot (1 + i)^n \quad (2)$$

5.4.1 Valor presente (PV)

O Valor Presente (PV) representa o valor atual de um fluxo de caixa futuro, descontado a uma taxa de retorno específica. Ele é calculado com a fórmula:

$$PV = \frac{FV}{(1+i)^n} \quad (3)$$

Onde:

PV: Valor Presente

FV: Valor Futuro

i: Taxa de desconto

n: prazo

Exemplo: Valor Presente

```
pv_f<-function(fv,r,n){fv/(1+r)^n}  
pv_f(100,0.1,5)  
> 62.09213
```

5.4.2 Valor futuro (PV)

O Valor Futuro (FV) indica quanto um investimento valerá em um período futuro, considerando uma taxa de juros composta. A fórmula utilizada é:

$$FV = PV \times (1 + i)^n \quad (4)$$

Exemplo: Valor Futuro

```
fv_f<-function(pv,r,n) {pv*(1+r)^n}  
fv_f(100,0.1,5)  
>161.051
```

5.4.3 Valor presente líquido (VPL)

Valor Presente Líquido (VPL) é a soma dos fluxos de caixa futuros descontados para o presente, subtraindo o investimento inicial. O VPL é um indicador da viabilidade de um projeto de investimento. Se **o VPL for positivo**, o investimento é considerado viável.

$$VPL = \sum_{t=1}^n \frac{FC_t}{(1+i)^n} + FC_0 \quad (5)$$

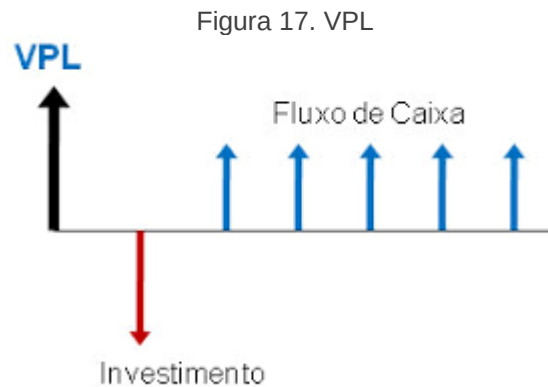
Onde:

FCt é o fluxo de caixa do período t

O Fluxo de Caixa Descontado (VPL) - é usado quando os desembolsos

financeiros (débitos e créditos) são realizados ao longo de todo fluxo de caixa. Todos os fluxos, individualmente, precisam ser descontados ao tempo 0.

VPL > 0 Significa que o projeto é viável, pois os benefícios gerados, superam o investimento.



Fonte: Elaboração própria

$$VPL = FC_0 + \frac{FC_1}{(1+TMA)^1} + \frac{FC_2}{(1+TMA)^2} + \dots + \frac{FC_n}{(1+TMA)^n} \quad (6)$$

Exemplo: Fluxo de caixa descontado – VPL – Investimento único em t=0

```
fluxo.caixa<-c(-1000,200,250,300,350,500) #Investimento no t=0
vpl<-function(x,tx){
y<-x[-1] #y será o fluxo de benefícios a ser descontado
coef<-numeric(5)
for(i in 1:5)
coef[i]<-y[i]/((1+tx)^i)
vp<-sum(coef)
vp+x[1] #x[1] é o investimento inicial a ser somando com a soma do fluxo
de benefícios descontado
}
vpl(fluxo.caixa,0.1)
>163.3396
ifelse(vpl(fluxo.caixa,0.1)>0, "projeto viável", "projeto inviável")
```

Exemplo: Fluxo de caixa descontado – VPL - Investimento em t=0 e t=1

```
fluxo.caixa<-c(-1000,-1000,200,250,300,350,500,700,800) #fluxo com
```

```

investimento no t=0 e t=1
vpl<-function(x,tx){
y<-x[-2]
coef<-numeric(7)
for(i in 2:8)
coef[i-1]<-y[i-1]/((1+tx)^i)
vp<-sum(coef)
vp+x[1]+x[2]/1.1
}
vpl(fluxo.caixa,0.1)
> -1447.544

```

5.4.3 Taxa interna de retorno (TIR)

A Taxa Interna de Retorno (TIR) é a taxa de desconto que faz com que o **VPL** seja igual a **zero**, ou seja: $VPL(TIR) = 0$.

Exemplo: Taxa Interna de Retorno - TIR

```

cf <- c(-10000, 1300, -1200, 12000)
npv <- function(i, cf, t=seq(along=cf)) sum(cf/(1+i)^t)
irr <- function(cf) { uniroot(npv, c(0,1), cf=cf)$root }
irr(cf)
>0.06860643

```

5.5 A DINÂMICA DA DÍVIDA PÚBLICA

Para esta seção, recomenda-se a leitura do capítulo 9 do livro de Finanças Públicas do Fábio Giambiagi, denominado “A dinâmica da Dívida Pública”.

A evolução da Dívida Pública.

$$D_t = D_{t-1} + J_t - SP_t - S_t \quad (7)$$

Onde:

D_t : Dívida no tempo t

D_{t-1} : Dívida no tempo $t-1$

J_t : Despesa com Juros da Dívida

SP_t : Superávit (ou Déficit) Primário

S_t : Senhoriagem

i : Taxa de juros nominal

$$J_t = D_{t-1} \times i \quad (8)$$

(2) em (1) = (3)

$$D_t = D_{t-1}(1+i) - SP_t - S_t \quad (9)$$

Dividindo pelo PIB, tem-se os indicadores em letra minúscula em (4), representadas como proporções do PIB.

$$d_t = d_{t-1}(1+i)/(1+q)(1+\pi) - h_t - s_t \quad (10)$$

Ao dividir pelo PIB, deve-se descontar da taxa de crescimento nominal da dívida a taxa de crescimento nominal do PIB: $(1+q)(1+\pi)$. Onde q é a taxa de crescimento real e π a inflação.

A equação (4) permite entender claramente que adiar o ajustamento, como a rolagem da dívida ou desequilíbrios no lado fiscal da economia ou ambos, tende a tornar este ajustamento cada vez mais difícil, pois a postergação das medidas de austeridade implicam a persistência de déficit, que por sua vez geram um aumento da dívida e inicia-se um ciclo vicioso.

Além disso, a equação (4) expressa uma equação diferencial de primeira ordem, reorganizando os termos e resolvendo, encontra-se a sua solução geral:

$$d_t = [d_0 - d_p](b)^t + d_p \quad (11)$$

Onde:

dt = relação dívida/PIB

do = dívida inicial

dp = dívida potencial

$b = (1+i)/(1+q)(1+\pi)$.

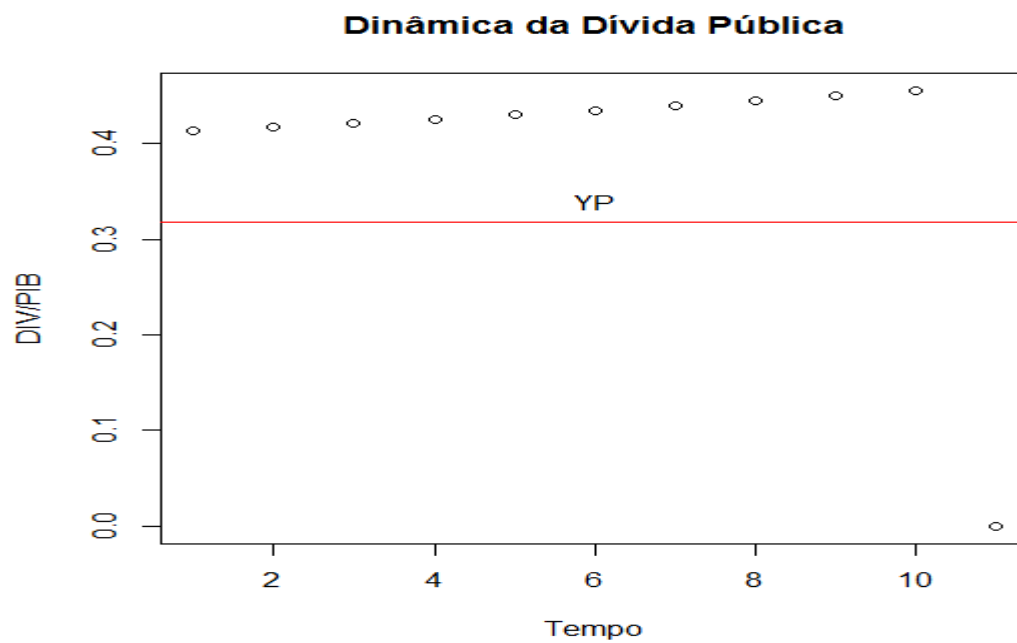
t = tempo

Se $|b| > 1$, dinâmica explosiva, $|b| < 1$, dinâmica convergente para a dívida potencial. A dívida potencial é aquela onde a combinação de parâmetros da economia mantém a relação Dívida/PIB constante ao longo do tempo

Encontre a trajetória da dívida pública para uma economia com os seguintes parâmetros: $d_0=0,41$, $h=0,013$, $q=0,003$, $\pi=0,068$ e $tx=0,115$. (Brasil 2014).

```
divida<-function(d0,h,q,pi,tx){
  yp<- (-1)*h*(((1+q)*(1+pi))/((1+q)*(1+pi)-(1+tx)))
  b<-(1+tx)/((1+q)*(1+pi))
  return(c(yp,b))
}
divida(0.41,0.013,0.003,0.068,0.115)
> 0.3179663 1.0408848
divida<-function(d0,h,q,pi,tx){
  yp<- (-1)*h*(((1+q)*(1+pi))/((1+q)*(1+pi)-(1+tx)))
  b<-(1+tx)/((1+q)*(1+pi))
  div<-numeric(11)
  for(i in 0:10)
    div[i]<-(d0-yp)*((b)^i)+yp
  plot(div, xlab="Tempo", ylab="DIV/PIB", main="Dinâmica da Dívida
Pública")
  abline(h=yp, col=2)
  text(6,0.34, "YP")
  div
}
divida(0.41,0.013,0.003,0.068,0.115)
```

Figura 18. Dinâmica da Dívida Pública



Fonte: Elaboração própria

A trajetória acima é explosiva. $|b| > 1$

Em quanto tempo a dívida atingirá 45,53% do PIB? Resp. 10 anos.

Resolve-se a solução geral, equação (5), para t .

```
divida<-function(d0,h,q,pi,tx){
  yp<- (-1)*h*(((1+q)*(1+pi))/((1+q)*(1+pi)-(1+tx)))
  b<-(1+tx)/((1+q)*(1+pi))
  d<-0.4553622
  a<-d0-yp
  t<-(log(d-yp)-log(a))/log(b)
  return(t)
}
divida(0.41,0.013,0.003,0.068,0.115)
>10
```




CAPÍTULO 06

MANIPULAÇÃO DE DADOS

6.1 LEITURA DE DADOS

A partir de agora estaremos interessados na leitura de dados. O comando `scan` é útil para scanear dados, possivelmente o pesquisador poderá utilizar este comando quando sua informação estiver em um site na internet não disponível para download, e sua única opção seja fazer um `ctrl+c` dos dados e `ctrl+v` no Editor do R, então:

```
x <- scan(n=5) #Scanear 5 elementos e salvar no objeto x
```

```
45 21.4 56 66.54 45.8787
```

```
list(x)
```

`y<-scan(what=character())` #Scanea e salva no objeto y todos os elementos que estiveram após esse comando, sem limitar sua quantidade.

```
"erer" "fdf" 233 44.5
```

Observe que para parar o scanear você deverá clicar duas vezes no console + ESC

```
list(y)
```

Para transformar informações textuais de datas para a linguagem de compreensão de datas no R, use a função `as.Date`, como a seguir.

```
dates1 <- c("02/27/92", "02/27/92", "01/14/92", "02/28/92", "02/01/92")
```

```
as.Date(dates1, "%m/%d/%y")
```

```
dates1
```

6.2 IMPORTANDO DADOS TXT

Digite no excel as informações abaixo e depois salve em formato txt, como *exemplo.txt*.

Tabela 1. Importando Dados TXT

Empresa	Setor	Preço	DY	VPA
bbas3	Bancário	23	10%	35
sanb11	Bancário	14	12%	20
ltsa4	Bancário	9	8%	15
oibr4	Telecomunicações	1,5	17%	8
timp3	Telecomunicações	5	8%	4
vivt4	Telecomunicações	6	5%	5

Fonte: Elaboração própria

Os dados também podem ser acessados pelo link:

<https://github.com/vitorbmonteiro/apostila-R-datasets/blob/main/exemplo.txt>

O Comando para buscar uma uma base de dados TXT no Editor R é:

```
exemplo<-read.table(file.choose(), header=T) #Chamar o arquivo txt e salvar no objeto exemplo, o comando header=T indica que a base de dados possui cabeçalho, caso não tenha use FALSE. Se a base estiver com “virgula” na formatação da casa decimal, use a opção dec=“,”.
```

```
exemplo #Execute o objeto para visualiza-lo no console
```

A base de dados armazenada no objeto exemplo contém tanto variáveis numéricas quanto variáveis categóricas. Dessa forma, essa estrutura é denominada dataframe.

Inicialmente, o objetivo é explorar as formas de consulta de variáveis e elementos dentro do dataframe. Para isso, podemos utilizar uma abordagem matricial para acessar linhas e colunas, seguindo a sintaxe: exemplo[linhas,colunas]

Os colchetes `[]` são utilizados para acessar elementos dentro de um objeto. No caso de um dataframe, é necessário incluir uma vírgula `,` para separar as referências às linhas e colunas. Por exemplo:

```
exemplo[1,1] #Consulta primeira linha e primeira coluna, no caso, bbas3.
```

```
exemplo[,2] #Consulta todas as linha da segunda coluna, no caso, a variável setor.
```

Observe no R console que a variável setor é categórica e, portanto, apresenta dois níveis, Bancário e Telecomunicações.

```
exemplo[1,] #Consulta primeira linha de todas as coluna.
```

```
exemplo[3,3] #Consulta terceira linha e terceira coluna.
```

```
exemplo[1,3] #Consulta primeira linha e terceira coluna.
```

```
exemplo[c(1:5),c(2,3)] #Consulta linhas 1 a 5 das colunas 2 e 3, ou simplesmente,
```

```
exemplo[1:5,c(2,3)]
```

Existem outras três maneiras de extrair dados de um DATAFRAME. Uma é usando a função `attach()`, que torna as variáveis disponíveis apenas digitando o nome do cabeçalho no Editor do R.

```
attach(exemplo)
```

```
setor #Similar a exemplo[,2]
```

A segunda é utilizando o símbolo `$` combinando o nome do objeto e a respectiva variável.

```
exemplo$Setor #Similar a exemplo[,2]
```

```
exemplo$VPA #Similar a exemplo[,5]
```

E a terceira é colocando o nome da variável dentro da consulta do objeto entre aspas.

```
exemplo[, "Setor"] #Similar a exemplo$Setor ou exemplo[,2]
```

6.3 OPERAÇÕES BÁSICAS E GRÁFICOS USANDO DATAFRAMES

Os dataframes são estruturas fundamentais no R para trabalhar com dados organizados em formato de tabela, ou seja, linhas e colunas. Após a importação ou criação de um dataframe, é comum realizarmos operações como ordenação, cálculo de estatísticas descritivas e aplicação de funções a colunas ou linhas específicas. Nesta seção, apresentamos algumas dessas operações básicas utilizando o dataframe `exemplo`, criado anteriormente.

```
exemplo[order(exemplo$Preço),] #Classifica de A a Z com base no Preço.
```

Esse comando reorganiza todas as linhas do dataframe `exemplo` em ordem crescente da variável `Preço`, considerando todas as colunas. É similar ao procedimento Classificar de A a Z do Excel. Caso queira ordenar em ordem decrescente, utilize o argumento `decreasing = TRUE` dentro da função.

```
exemplo[order(exemplo$Preço, decreasing = TRUE),]
```

Para calcular a média dos preços das ações, podemos utilizar diferentes abordagens, todas essas formas a seguir acessam a coluna de preços e retornam sua média.

```
mean(exemplo[, "Preço"])
```

```
mean(exemplo[,3])  
mean(exemplo$Preço)
```

As funções `colSums()` e `rowSums()` são utilizadas para calcular a soma das colunas e das linhas, respectivamente. Da mesma forma, `colMeans()` e `rowMeans()` retornam a média das colunas e das linhas. Para que essas funções funcionem corretamente, é necessário que o dataframe tenha mais de uma linha ou coluna — ou seja, mais de uma dimensão na direção em que a função será aplicada. No exemplo a seguir, utilizamos as colunas 3 a 5 com a função `colMeans()`.

```
colMeans(exemplo[,3:5]) #obrigatório mais de uma entrada de coluna  
ou  
colSums(exemplo[,3:5])/length(exemplo[,1])
```

Para fazer a média de preço por setor:

```
mean(exemplo[1:3,3]) #Média de preço setor Bancário  
mean(exemplo[4:6,3]) #Média de preço setor Telecomunicações
```

Para mais de uma coluna:

```
colMeans(exemplo[1:3,3:5]) #Média de preço setor Bancário  
colMeans(exemplo[4:6,3:5]) #Média de preço setor Telecomunicações
```

Para combinar variáveis numéricas com variáveis categóricas, utilizar o comando `TAPPLY`, que possui três argumentos: `tapply(variável numérica, variável categórica, função)`.

```
tapply(exemplo$Preço, exemplo$Setor, mean) #Esse comando fará a  
média de preço das ações por setor.
```

Caso deseje aplicar uma função como a média a **mais de uma variável numérica** simultaneamente, em combinação com uma variável categórica, o comando mais adequado é o `aggregate()`:

```
aggregate(exemplo[,3:5], list(exemplo[,2]), mean)
```

Obtém-se a média de Preço, DY e VPA das ações dos setores bancário e telecomunicações.

Para fazer o gráfico do `aggregate`, siga os passos:

```
resu<-aggregate(exemplo[,3:5], list(exemplo[,2]), mean)
```

Transformar apenas as colunas numéricas em matriz

```
matriz_medias <- as.matrix(resu[, -1]) # remove a coluna Group.1
```

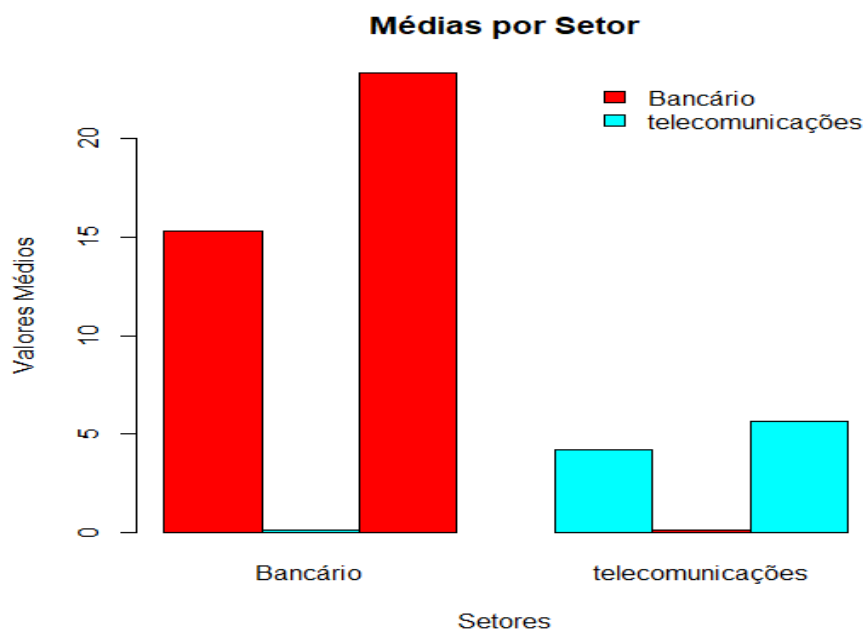
Adicionar os nomes dos setores como rótulos de coluna

```
rownames(matriz_medias) <- resu$Group.1
```

Gerar o gráfico de barras

```
barplot(t(matriz_medias),           # transposta para agrupar por setor
beside = TRUE,                     # barras lado a lado
col = rainbow(nrow(matriz_medias)), # cores diferentes por variável
main = "Médias por Setor",
xlab = "Setores",
ylab = "Valores Médios",
legend.text = rownames(matriz_medias),
args.legend = list(x = "topright", bty = "n"))
```

Figura 19. Médias por setor



Fonte: Elaboração própria

Uma alternativa para fazer o gráfico da base *exemplo.txt* acima, é criar manualmente as torres com conjunto de barras com as médias de Preço, VPA e DY, para os setores Bancário e Telecomunicações.

```
bancário<-colMeans(exemplo[1:3,3:5])
```

```
tele<-colMeans(exemplo[4:6,3:5])
```

Juntar as torres em um novo objeto e nomeá-los.

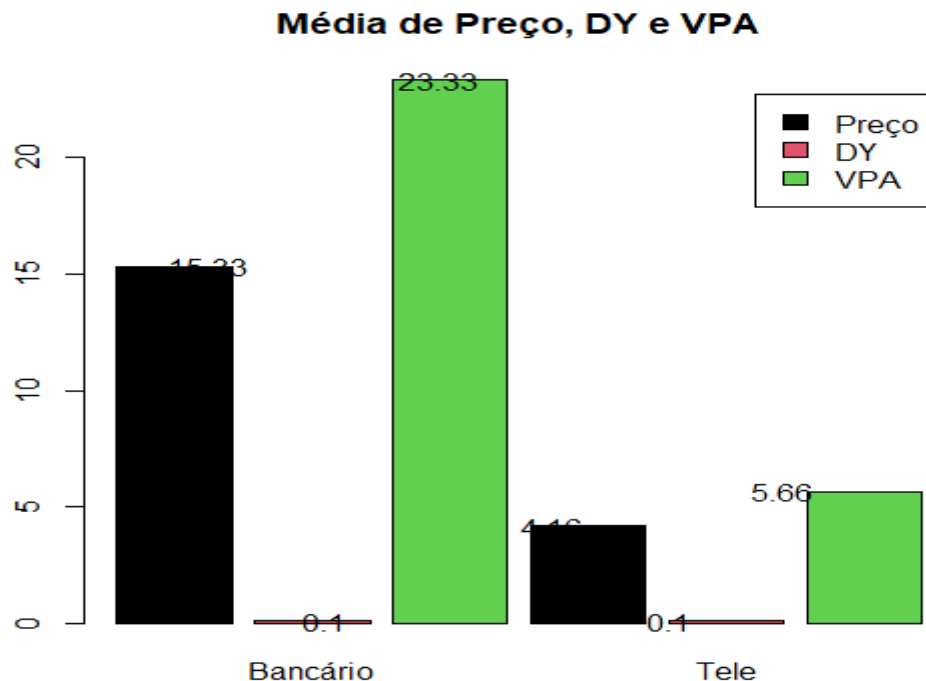
```
carteira<-c(bancário,tele)
```

```

carteira
names(bancário)<-c("", "Bancário")
names(tele)<-c("", "Tele")
carteira<-c(bancário,tele)
barplot(carteira)
barplot(carteira, col=c(1,2,3,1,2,3), legend=c("Preço","DY","VPA"),
main="Média de Preço, DY e VPA")
text(carteira, c("15.33","0.1","23.33","4.16","0.1","5.66"), col = 1,
xpd=TRUE)

```

Figura 20. Média de preço DY e VPA



Fonte: Elaboração própria

Em uma nova perspectiva com a base *exemplo.txt*, faremos um gráfico de barras para Preço, VPA e DY, fragmentado pelas ações, com uso do argumento `beside=TRUE` na função `barplot`, mas para isso o dataframe precisa ser configurado para uma matriz.

```

exemplo1<-cbind(exemplo[,3],exemplo[,4]*100,exemplo[,5]) #

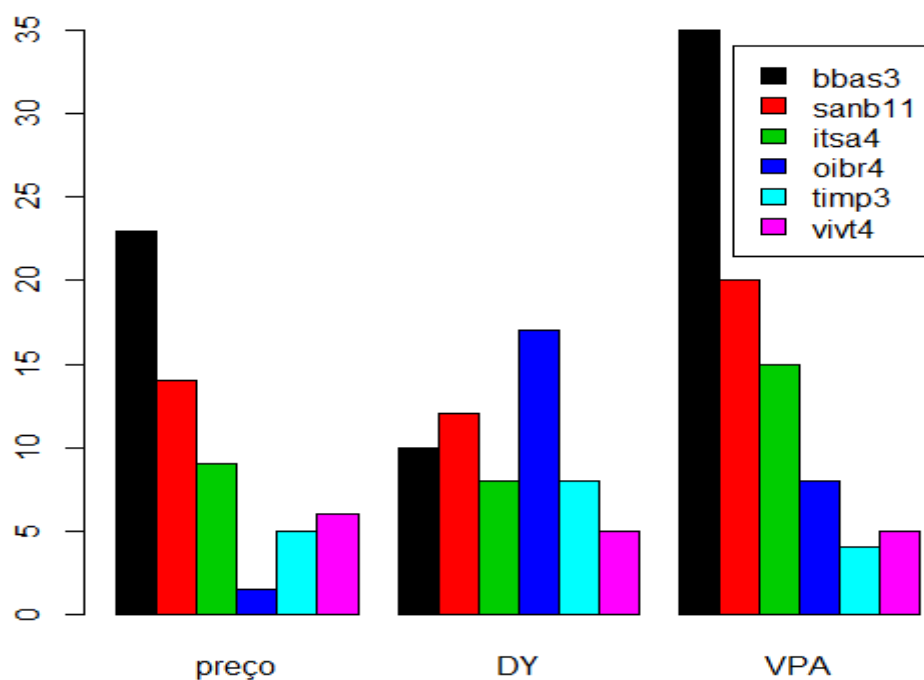
```

Fragmentação e nova junção das variáveis para aumentar a escala da variável

DY, que ficou inexpressiva nos gráficos anteriores.

```
dimnames(exemplo1)=list(c("bbas3", "sanb11", "itsa4",  
"oiabr4", "timp3", "vivr4"), c("preço", "DY", "VPA"))  
barplot(exemplo1)  
barplot(exemplo1, beside=TRUE, col=rep(1:6,3),  
legend.text=rownames(exemplo1))
```

Figura 21. Gráfico de barras para Preço, VPA e DY



Fonte: Elaboração própria

6.4 IMPORTANDO DADOS DO EXCEL, ANÁLISE GRÁFICA E FUNÇÕES LÓGICAS NO DATAFRAME

Para aprofundar os conceitos de manipulação de dados no DATAFRAME, suponha o exemplo a seguir. Digite os dados no excel e salve como *bovespa.xls* ou *bovespa.xlsx*

Tabela 2. dados no DATAFRAME

mês	bbas3	bbdc4	itsa4	csna3	vale3	goau4	lbovespa	Trimestre
jan/15	24	34	7	6	23	12.5	46900	1T
fev/15	25	34.5	8	6.5	22.5	12	51500	1T
mar/15	27	35	9	7	24	13	51150	1T
abr/15	26	34	9	7.5	23.5	12.5	56230	2T
mai/15	22	33	8	9	27	12	52760	2T
jun/15	20	32	7.5	7.5	25	11	53080	2T
jul/15	18.5	28	7.5	6	22	9	50860	3T
ago/15	18	26	7	2.5	18	7	46600	3T
set/15	17	24	7	4	16	5	45050	3T
out/15	16.5	22	6.5	5	17	3	45800	4T
nov/15	16	20	6	4	13	2	45045	4T
dez/15	15	19	6	4.5	12	1.5	43390	4T

Fonte: Elaboração própria

Os dados podem ser acessados pelo link:

<https://github.com/vitorbmonteiro/apostila-R-datasets/blob/main/bovespa.xlsx>

Instalar o pacote para importar dados do Excel.

```
install.packages("readxl")
```

```
library(readxl) # Habilita o pacote
```

Crie um objeto denominado bov para salvar seus dados

```
bov<- data.frame(read_excel(file.choose())) #Buscar no diretório
```

```
bov #Chamar os dados
```

```
bov
```

```
names(bov) #nome de todas as variáveis
```

```
preço<-bov[,2:7] #Salva no objeto preço apenas os preços das ações
```

Vamos aos gráficos que mostram as relações entre as variáveis, com gráficos de dispersão.

```
plot(preço[, "bbas3"],preço[, "itsa4"]) #relação positiva
```

```
plot(preço[, "goau4"],preço[, "vale3"]) #relação positiva
```

```
pairs(preço) #Faz a dispersão de todas as variáveis dois a dois
```

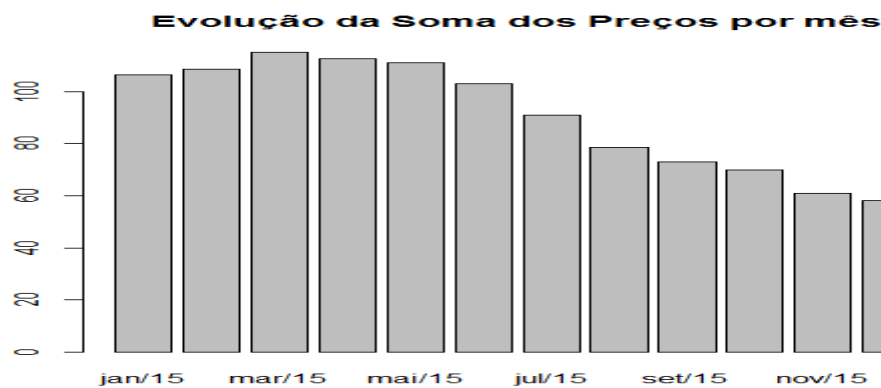
Fazer gráfico da soma dos preços

```
rowSums(preço) #Soma do preço das ações por mês
```

```
barplot(rowSums(preço), main="Evolução da Soma dos Preços por mês")
```

#Gráfico de barras do passo anterior

Figura 22. Evolução da Soma dos preços por mês



Fonte: Elaboração própria

```
order(bov[, "Ibovespa"]) #Ordem crescente das linhas da variável
Ibovespa
```

```
[1] 12 11 9 10 8 1 7 3 2 5 6 4
```

Observe no console que os valores indicam o número da ordem crescente dos valores da variável Ibovespa.

```
bov[order(bov[, "Ibovespa"]),] #DATAFRAME reordenado por ordem
crescente da variável Ibovespa
```

```
aggregate(preço, list(bov[, "Trimestre"]), mean) #Média de preço por
Trimestre
```

Group.1	bbas3	bbdc4	itsa4	csna3	vale3	goau4
1	1T 25.33333	34.50000	8.000000	6.500000	23.16667	12.50000
2	2T 22.66667	33.00000	8.166667	8.000000	25.16667	11.83333
3	3T 17.83333	26.00000	7.166667	4.166667	18.66667	7.000000
4	4T 15.83333	20.33333	6.166667	4.500000	14.00000	2.166667

Agora vamos construir um gráfico apresentando a evolução trimestral de cada ação.

```
aggregate(preço, list(bov[, "Trimestre"]), mean) #Média de preço por
Trimestre
```

```
resu1 <- aggregate(preço, list(bov[, "Trimestre"]), mean) #Média de preço
por Trimestre
```

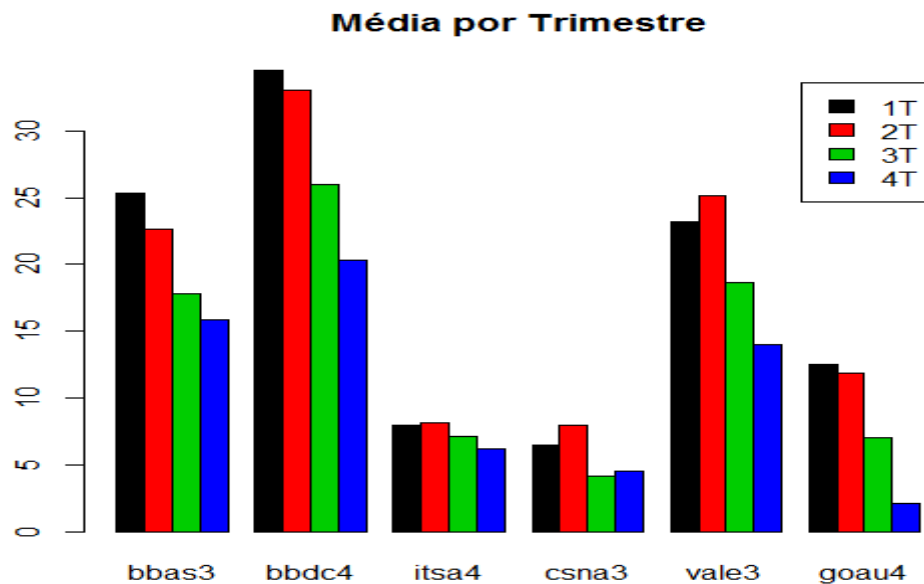
```
resu1
```

```
resu2 <- cbind(resu1[, 2], resu1[, 3], resu1[, 4], resu1[, 5], resu1[, 6], resu1[, 7])
```

```
#linguagem matricial do dataframe. Cada variável será uma coluna no beside=T
dimnames(resu2) = list(c("1T", "2T", "3T", "4T"), c("bbas3", "bbdc4", "itsa4", "cs
```

```
na3","vale3","goau4"))
resu2
barplot(resu2,beside=T,legend.text=rownames(resu2), col=1:4,
main="Média por Trimestre")
```

Figura 23. Média por trimestre



Fonte: Elaboração própria

Para manipular os dados com funções lógicas, veja os exemplos a seguir:

```
bov[which(bov$Ibovespa>50000),] #Recorta a base de dados para todas
as linhas cujo os valores da variável Ibovespa acima de 50.000
```

```
bov[which(bov$Trimestre=="1T"),] #Recorta a base de dados para valores
da variável Trimestre igual a 1T
```

```
bov[which(bov$Trimestre=="1T" & mol$Ibovespa>50000),] #Recorta a
base de dados para valores da variável Ibovespa acima de 50.000 e Trimestre
igual a 1T.
```

```
colMeans(bov[which(bov$Trimestre=="1T" & bov$Ibovespa>50000), 1:7])
```

Agora construir um gráfico com as médias das ações quando seus preços estão acima e abaixo da média do Ibovespa.

```
colMeans(bov[which(bov$Ibovespa>=mean(bov$Ibovespa)), 1:6])
```

```
colMeans(bov[which(bov$Ibovespa<mean(bov$Ibovespa)), 1:6])
```

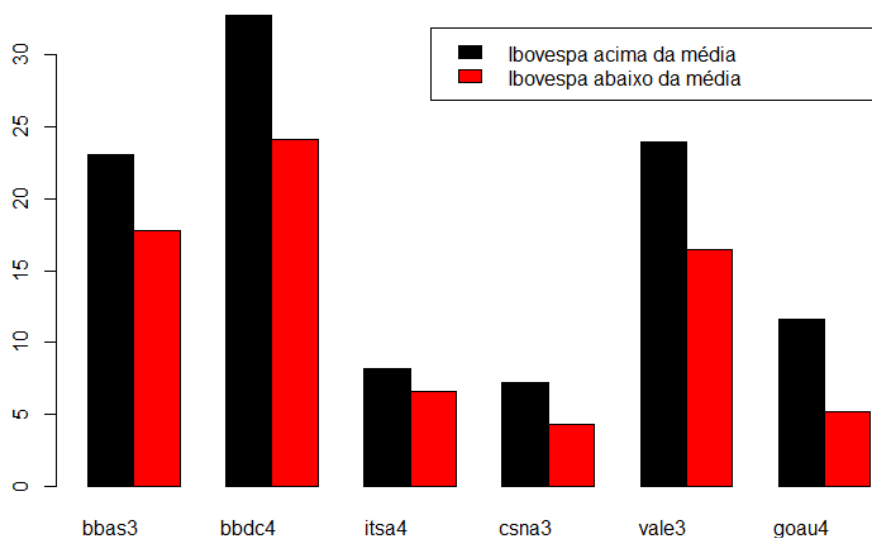
```

maior<-colMeans(bov[which(bov$Ibovespa>=mean(bov$Ibovespa)), 1:6])
menor<-colMeans(bov[which(bov$Ibovespa<mean(bov$Ibovespa)), 1:6])
pares1<-c(maior[1],menor[1])
pares2<-c(maior[2],menor[2])
pares3<-c(maior[3],menor[3])
pares4<-c(maior[4],menor[4])
pares5<-c(maior[5],menor[5])
pares6<-c(maior[6],menor[6])
names(pares1)<-c("bbas3")
names(pares2)<-c("bbdc4")
names(pares3)<-c("itsa4")
names(pares4)<-c("csna3")
names(pares5)<-c("vale3")
names(pares6)<-c("goau4")
grafico<-c(pares1,pares2,pares3,pares4,pares5,pares6)
barplot(grafico, col=1:2, space=c(0,0,1,0,1,0,1,0,1,0,1),
legend=c("Ibovespa acima da média","Ibovespa abaixo da média" ))
title(main="Preço das ações com Ibovespa abaixo e acima da média do
próprio Ibovespa")

```

Figura 24. Preço das ações com Ibovespa abaixo e acima da média do próprio Ibovespa

Preço das ações com Ibovespa abaixo e acima da média do próprio Ibovespa



Fonte: Elaboração própria

6.5 CONSTRUINDO COLUNA DE TRIMESTRES

No exercício anterior, a variável trimestre foi construída no *Excel* e posteriormente importada para o *dataframe* no R. Neste exercício, o procedimento será inteiramente realizado no R. A base de dados utilizada será o arquivo `portfolio01.txt`, disponível no seguinte endereço:

<https://github.com/vitorbmonteiro/apostila-R-datasets/blob/main/portfolio01.txt>

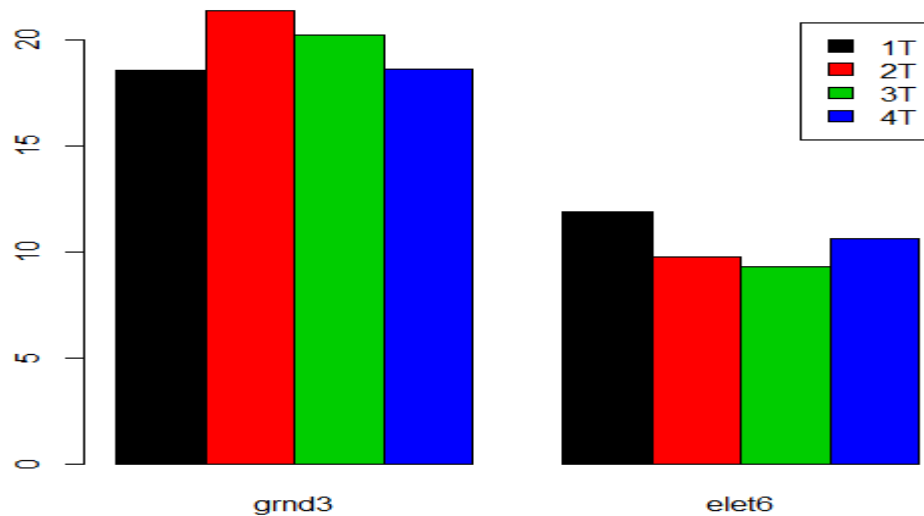
Essa base em formato `.txt`, havia sido previamente tratada no *Excel*, aplicando-se a função `=MÊS()` sobre as datas diárias para extrair a informação de mês.

```
port01<-read.table(file.choose(), header=T) #Chamar os dados salvo em
seu diretório
port01 #Visualizar os dados no console. Note que existe a variável mês,
o objetivo é gerar as médias por trimestres.
resu<-aggregate(port01[,3:7],list(port01[,2]),mean)
resu # Média de preço das ações por mês
```

A coluna de mês está na ordem alfabética e não na ordem cronológica, reorganizar as linhas do *dataframe* manualmente

```
resu1<-resu[c(5,4,9,1,8,7,6,2,12,11,10,3),]
resu1
trimestre<-rep(c("1T","2T","3T","4T"), each=3)
trimestre #ver objeto
resu2<-data.frame(trimestre,resu1)
resu2
resu3<-aggregate(resu2[,3:7],list(resu2[,1]),mean)
resu3
resu4<-cbind(resu3[,2],resu3[,3])
dimnames(resu4)=list(c("1T","2T","3T","4T"),c("grnd3","elet6"))
barplot(resu4,beside=T,legend.text=rownames(resu4), col=1:4)
```

Figura 25. Dataframe manualmente



Fonte: Elaboração própria

6.6 FINANÇAS COMPORTAMENTAIS

A teoria do prospecto, proposta por Kahneman e Tversky (1979), transformou profundamente a forma como entendemos as decisões econômicas sob risco. Em 2002, Daniel Kahneman recebeu o Prêmio Nobel de Economia em reconhecimento às suas contribuições ao aplicar a psicologia à economia, demonstrando que os indivíduos frequentemente se desviam das previsões da teoria da utilidade esperada. Esse marco consolidou o campo que posteriormente ficaria conhecido como economia comportamental ou finanças comportamentais.

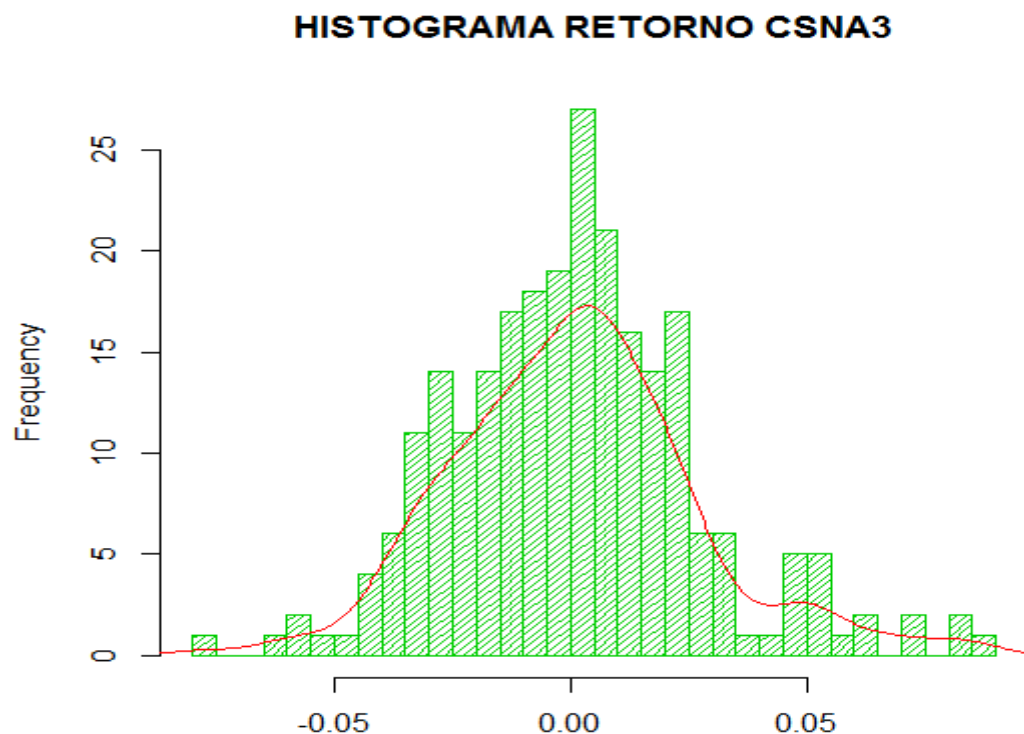
A tradição neoclássica sustentava que os consumidores, ao atribuírem utilidade aos bens, poderiam tomar decisões probabilísticas sempre respeitando axiomas de racionalidade. No entanto, Kahneman e Tversky mostraram que não é tão simples assim: indivíduos apresentam vieses cognitivos que distorcem escolhas, fazendo com que a forma como a alternativa é apresentada (framing), o contexto de risco ou mesmo o momento em que a decisão é tomada influenciem fortemente o resultado. Um exemplo marcante é a aversão à perda: a dor psicológica de perder R\$ 100 é maior do que a satisfação de ganhar a mesma quantia. Isso reflete uma avaliação assimétrica entre ganhos e perdas,

contrariando a teoria da utilidade esperada.

Mostre a assimetria positiva dos retornos das ações, use CSNA3 do *portfolio01.txt* da seção anterior

```
csna3<-ts(port01[,7]) #chamar a variável
rcsna3<-diff(csna3)/lag(csna3,-1) #retorno
hist(rcsna3, nclass=30, main="HISTOGRAMA RETORNO CSNA3",
density=30, col=3, xlab="")
lines(density(rcsna3), col=2)
```

Figura 26. Hisgrama retorno CSNA3



Fonte: Elaboração própria

Assimetria da distribuição:

$$\frac{n}{(n-1)(n-2)} \sum \left(\frac{x_i - \bar{x}}{s} \right)^3 \quad (12)$$

```
length(rcsna3)/((length(rcsna3)-1)*(length(rcsna3)-2))*(sum((rcsna3-
```

```
mean(rcsna3))^3)/sd(rcsna3)) #Note que o resultado é assimetria positiva
```

```
[1] 0.0002785931
```

Para Kahneman e Tversky (1979), os investidores sentem muito mais a dor da perda do que o prazer obtido por um ganho de igual valor. A literatura em séries financeiras já aceita a idéia da assimetria dos retornos das ações. As volatilidades do retorno indicam fortes sinais de persistência, os choques levam a um longo tempo para dissipar-se, onde choques positivos e negativos têm impacto distinto sobre a volatilidade. Os mercados são mais sensíveis a notícias negativas, isso, de forma geral, leva a uma assimetria positiva, pois a distribuição fica mais densa à esquerda. Em geral, modelos como Value at Risk modelam os retornos financeiros com distribuição log normal para captar a assimetria da distribuição.



CAPÍTULO 07

SÉRIES TEMPORAIS

O estudo de séries temporais constitui uma etapa fundamental na análise de dados econômicos e financeiros, pois permite compreender a evolução de variáveis ao longo do tempo e identificar padrões sazonais, tendências e ciclos. No R, para que o programa reconheça um conjunto de dados como série temporal, é necessário convertê-lo utilizando a função `ts()`. Essa função organiza os dados de acordo com sua periodicidade, possibilitando a aplicação de métodos e rotinas próprias da análise temporal.

A função `ts()` possui três principais argumentos:

- **dados**: o vetor ou conjunto de valores a ser transformado em série temporal;
- **start**: define o ponto inicial da série (ano, ou ano e mês/trimestre, dependendo da frequência);
- **frequency**: especifica a periodicidade da série, sendo 1 para anual, 12 para mensal, 4 para trimestral, entre outras possibilidades.

```
x<-ts(rnorm(10), start = 2000, frequency = 1)
x
```

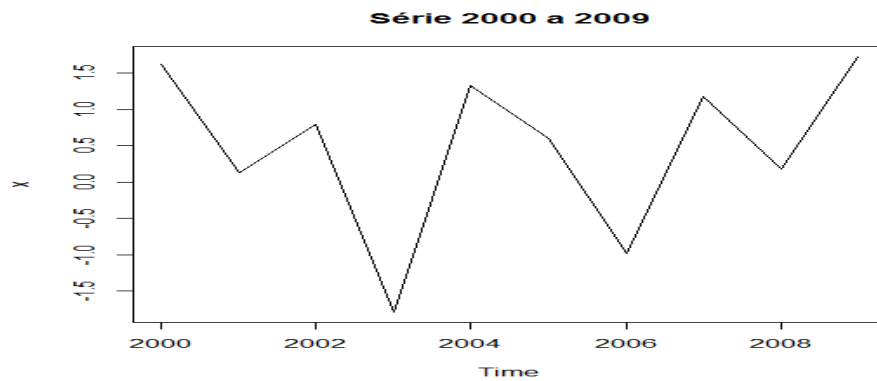
No exemplo acima, foi gerada uma série temporal composta por 10 observações aleatórias provenientes de uma distribuição normal padrão, isto é, com média zero e variância igual a 1. A série tem início no ano de 2000 e é incrementada em unidades anuais, dado que o argumento `frequency = 1` define a periodicidade como anual.

7.1 GRÁFICOS DAS SÉRIES

A série salva no objeto `x` possui dados gerados aleatoriamente, sob uma distribuição normal com média zero e variância um, com início em 2000 e término em 2009, dados anuais.

```
plot(x, main="Série 2000 a 2009") # Gráfico da Série
```


Figura 27. Série 2000 a 2009



Fonte: Elaboração própria

Segue exemplo com dados trimestrais e mensais. Frequency = 4 são dados trimestrais e Frequency = 12 são dados mensais. O comando `rnorm(10)` vai gerar dez observações aleatórias normalmente distribuídas com média zero e variância um.

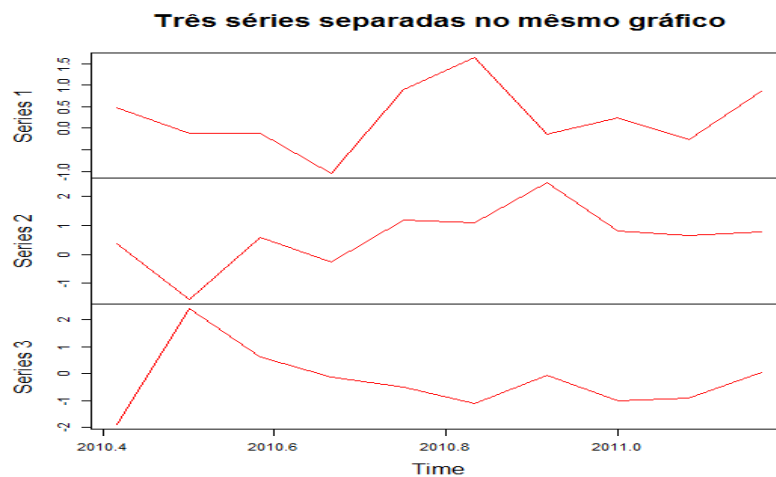
```
ts(rnorm(10), frequency = 4, start = c(2010, 2)) # Início no segundo trimestre de 2010
```

```
ts(rnorm(10), frequency = 12, start = c(2010, 2)) # Início em fevereiro de 2010
```

Podemos salvar a série em um objeto. Quando a série for uma matriz, cada coluna representa uma série.

```
m <- ts(matrix(rnorm(30), 10, 3), start = c(2010, 6), frequency = 12)
m
plot(m, main="Três séries separadas no mesmo gráfico", col="red")
```

Figura 28. Três series separadas no mesmo gráfico



Fonte: Elaboração própria

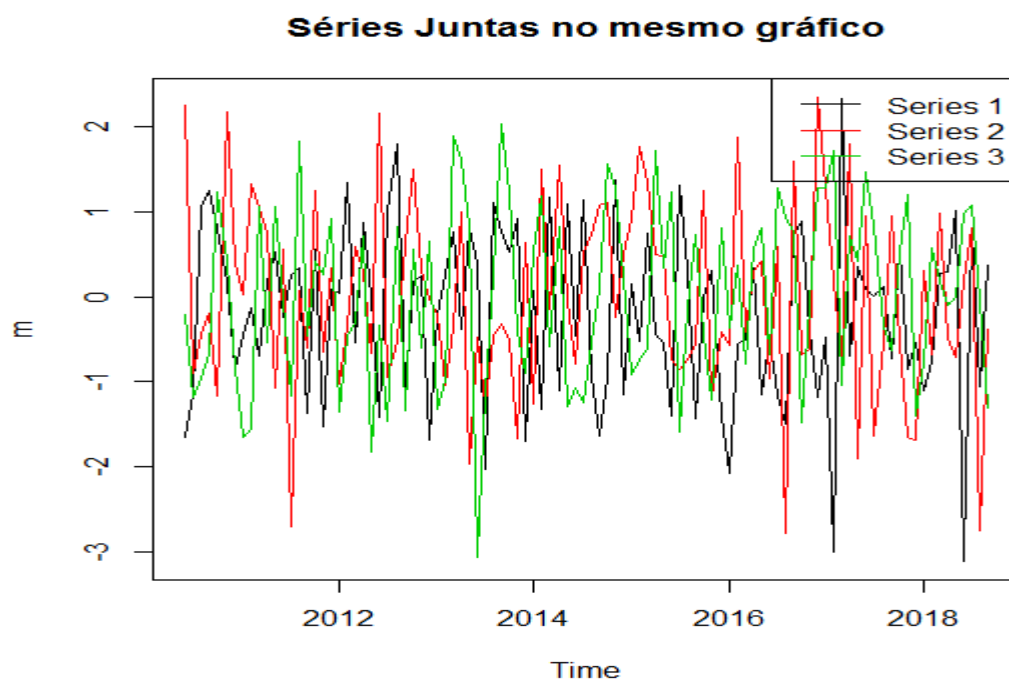
```

m <- ts(matrix(rnorm(300), 100, 3), start = c(2010, 6), frequency = 12)
m
plot(m)
plot(m, main="Séries Juntas no mesmo gráfico", plot.type = "single", col
=1:3)
legend("topright", legend = colnames(m), col = 1:3, lty = 1)

```

Usa-se o commando `plot.type = "single"` como argumento da função `plot` para as series ficarem juntas no mesmo gráfico.

Figura 29. Séries juntas no mesmo gráfico



Fonte: Elaboração própria

7.2 CONSTRUINDO JANELAS

A análise de séries temporais frequentemente exige a delimitação de intervalos específicos dentro da série completa, seja para realizar comparações entre períodos distintos, seja para estimar modelos em subconjuntos de dados. Essa técnica é conhecida como construção de janelas (*rolling windows*).

No R, o comando `window()` permite extrair partes da série temporal armazenada em um objeto, definindo datas de início (`start`) e de término (`end`). Assim, é possível trabalhar com períodos delimitados da série original, sem

necessidade de criar manualmente novas bases. Essa abordagem é útil, por exemplo, quando se deseja analisar apenas uma fase de expansão ou recessão econômica, ou quando se busca estimar indicadores em intervalos de tempo específicos.

```
start(m) #Retoma a data inicial da série  
[1] 2010 6  
end(m) #Retoma a data final da série  
[1] 2018 9
```

O objeto `m` tem início em junho de 2010 e término em setembro de 2018. Para quebrar as séries e criar subconjuntos de dados a partir das séries originais usa-se o comando `windows`.

```
J1<-window(m, start = c(2017, 1)) #Quebra as séries salvas no objeto m  
com início em janeiro de 2017 e término em setembro de 2018.
```

```
J2<-window(m, end = c(2013,7)) #Quebra as séries salvas no objeto m  
com início em junho de 2010 e término em julho de 2013.
```

```
J3<-window(m, start = c(2013, 7), end = c(2017,1)) #Quebra as séries  
salvas no objeto m com início em julho de 2013 e término em janeiro de 2017.
```

Note que a matriz “`m`” possui três séries, ao usar `m[,1]` estamos trabalhando apenas com a primeira série da matriz, ou seja, a primeira coluna.

Usar o comando `cbind` irá juntar as séries em uma matriz. O comando `cbind` cria matriz juntando colunas.

```
cbind(m[,1],j1,j2,j3)
```

Note que ao juntar as quatro séries `m[,1]`, `j1`, `j2` e `j3`, a série original encontra-se na primeira coluna, e aparece NA nas observações excluídas após os recortes nas demais séries.

7.3 DEFASAGENS E DIFERENÇAS

A análise de séries temporais frequentemente requer a manipulação dos dados em termos de defasagens (*lags*) e diferenças (*differences*). Esses recursos permitem investigar relações dinâmicas entre períodos distintos, bem como estabilizar séries não estacionárias.

O operador `lag()` desloca os valores da série no tempo, possibilitando comparações entre o período atual e períodos passados ou futuros. Já a função

diff() (que será apresentada posteriormente) calcula a variação entre observações consecutivas, sendo útil para remover tendências ou tornar a série estacionária.

Inicialmente carregamos o objeto “x” conforme tabela abaixo.

Tabela 3. Objeto “x”

Período	Xt	ΔX_t	Lag(X, -1)	Lag(X, 1)
Dezembro				2,5
Janeiro	2,5			2,7
Fevereiro	2,7	0,2	2,5	2,8
Março	2,8	0,1	2,7	2,7
Abril	2,7	-0,1	2,8	2,9
Maio	2,9	0,2	2,7	
Junho			2,9	

Fonte: Elaboração própria

```
x<-ts(c(2.5,2.7,2.8,2.7,2.9),start=c(2022,1), frequency=12)
```

```
list(x)
```

```
diff(x)
```

```
lag(x)
```

```
lag(x,-1)
```

Note que a diferença pode ser calculada como $X_t - \text{Lag}(X_t, -1)$

```
x-lag(x,-1)
```

Remova o objeto x e crie uma nova série temporal, com início no Segundo trimestre de 2010, com dez observações aleatórias normalmente distribuídas com media zero e variância um.

```
x <- ts(rnorm(10), frequency = 4, start = c(2010, 2))
```

```
x
```

Observe no console o deslocamento da série com as seguintes defasagens:

```
lag(x) #As informações do tempo t são reconduzidas para o tempo t-1  
(Defasa 1 período)
```

```
lag(x,-1) #As informações do tempo t são reconduzidas para o tempo t+1  
(Adianta 1 período)
```

```
lag(x,4) #As informações do tempo t são reconduzidas para o tempo t-4
```

Figura 30. R console

```

>
> x <- ts(rnorm(10), frequency = 4, start = c(2010, 2))
> x
      Qtr1      Qtr2      Qtr3      Qtr4
2010      0.53292038 -0.52829231  0.74031844
2011  0.56008061 -1.11440949 -1.07545186 -0.23067252
2012  1.42948235 -0.40651123 -0.03716719
> lag(x,1) # Ou apenas lag(x). As informações do tempo t são reconduzidas para o tempo t-1 (Defasa 1 p$
      Qtr1      Qtr2      Qtr3      Qtr4
2010  0.53292038 -0.52829231  0.74031844  0.56008061
2011 -1.11440949 -1.07545186 -0.23067252  1.42948235
2012 -0.40651123 -0.03716719
> lag(x,-1) #As informações do tempo t são reconduzidas para o tempo t+1 (Adianta 1 período)
      Qtr1      Qtr2      Qtr3      Qtr4
2010      0.53292038 -0.52829231
2011  0.74031844  0.56008061 -1.11440949 -1.07545186
2012 -0.23067252  1.42948235 -0.40651123 -0.03716719
> lag(x,4) #As informações do tempo t são reconduzidas para o tempo t-4
      Qtr1      Qtr2      Qtr3      Qtr4
2009      0.53292038 -0.52829231  0.74031844
2010  0.56008061 -1.11440949 -1.07545186 -0.23067252
2011  1.42948235 -0.40651123 -0.03716719
>

```

Fonte: Elaboração própria

Note que no console acima o comando `lag(x)` fez com que a série inicie agora no primeiro trimestre de 2010 ($t-1$). O comando `lag(x,-1)` fez com que a série inicie agora no terceiro trimestre de 2010 ($t+1$) e o comando `lag(x,4)` defasou um ano, iniciando agora no Segundo trimestre de 2009.

O comando `diff(x)` ou `diff(x, 1)` faz a diferença entre o tempo $t+1$ e t , `diff(x,2)` ou `diff(x, lag = 2)` faz a diferença entre o tempo $t+2$ e t . O comando `diff(x, differences= 2)` faz a diferença da primeira diferença.

`diff(x)` #faz a diferença entre $t+1$ e t , logo perde a primeira observação
`diff(x, differences = 2)` #faz a diferença da primeira diferença, logo perde as duas primeiras observações.

`diff(x, lag = 2)` #faz a diferença entre $t+2$ e t

Figura 31. R console

```

>
>
> diff(x) #faz a diferença entre t+1 e t, logo perde a primeira observação
      Qtr1      Qtr2      Qtr3      Qtr4
2010          2.03121591 -0.42096633
2011 -0.66524075 -0.56832267 -0.09056172  2.31232464
2012 -1.92715823 -0.44764394  1.38472787
> diff(x, differences = 2) #faz a diferença da primeira diferença, logo perde as duas primeiras observaç
      Qtr1      Qtr2      Qtr3      Qtr4
2010          -2.45218224
2011 -0.24427442  0.09691808  0.47776094  2.40288636
2012 -4.23948287  1.47951430  1.83237181
> diff(x, lag = 2) #faz a diferença entre t+2 e t
      Qtr1      Qtr2      Qtr3      Qtr4
2010          1.6102496
2011 -1.0862071 -1.2335634 -0.6588844  2.2217629
2012  0.3851664 -2.3748022  0.9370839
>
>
>
>
> |

```

Fonte: Elaboração própria

Caso as séries não tenham o mesmo início e fim, como no caso das series transformadas por diferenças e defasagens. Algumas operações matemáticas ficam inviáveis pela presenças de NAs nas séries.

por exemplo:

```

rm(x)
x <- ts(rnorm(10), frequency = 4, start = c(2010, 2))
y <- ts(rnorm(10), frequency = 4, start = c(2011, 1))
w<-cbind(x,y)
w
na.omit(w)

```

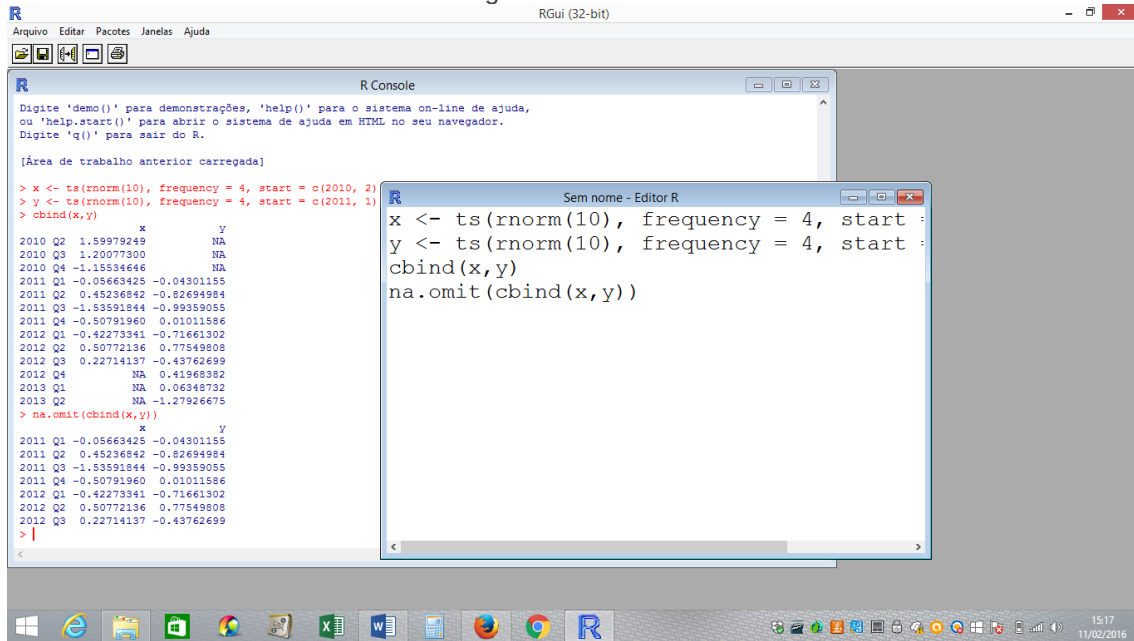
Ao junta-las com o comando `cbind(x, y)`, percebe-se que aparece NA nas informações inexistentes. O comando `na.omit(cbind(x,y))` faz a interseção dos períodos comuns das duas séries.

```

sum(w[,1])
mean(w[,1])
sum(na.omit(w[,1]))
mean(na.omit(w[,1]))

```

Figura 32. Editor R



Fonte: Elaboração própria

7.4 EXEMPLO COM DADOS REAIS

Os dados podem ser acessados pelo link:

<https://github.com/vitorbmonteiro/apostila-R-datasets/blob/main/portfolio01.txt>

Baixar a base de dados chamada *portfolio01.txt* com informações diárias de cinco ações negociadas na Bovespa, de 05/12/2012 a 04/12/2013. Baixe esses dados e carregue no R, no objeto port01.

```
port01<-read.table(file.choose(), header=T) #Carregar os dados
port01 #Visualizar os dados
```

Caso a sua base não tenha cabeçalho, você deve carregar os dados com a informação `header=FALSE` e nomear as variáveis de seu Dataframe.

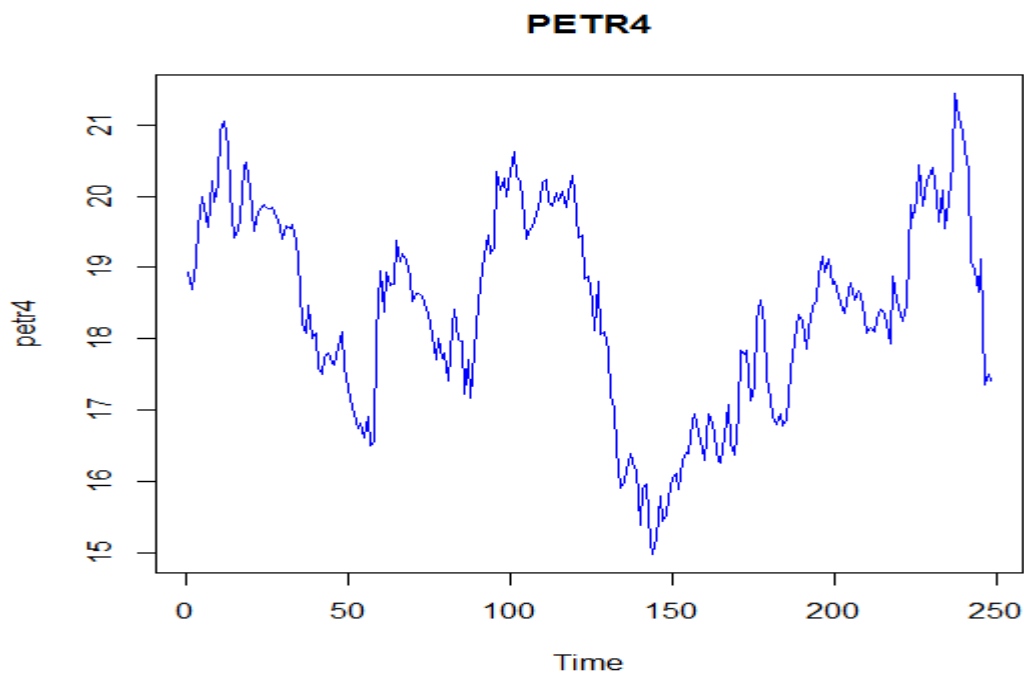
```
colnames(port01)<-c("data", "mes", "grnd3", "elet6", "bbas3", "petr4", "csna3")
port01 #visualizar no console
```

Observe que a Petrobrás e o Banco do Brasil são as colunas 6 e 5 do Dataframe respectivamente. Para salvar essas séries em objetos separados, crie os objetos `petr4` e `bbas3` e utilize os comandos abaixo para buscar os dados da respectiva coluna do Dataframe que corresponde a variável.

```
petr4<-ts(port01[,6], start=1, frequency=1)
```

```
petr4  
plot(petr4, main="PETR4", col="blue")
```

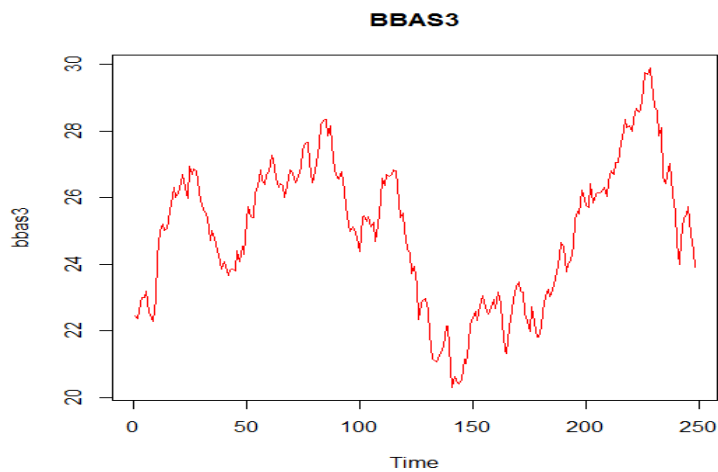
Figura 33. PETR4



Fonte: Elaboração própria

```
bbas3<-ts(port01[, "bbas3"], start=1, frequency=1)  
bbas3  
plot(bbas3, main="BBAS3", col="red")
```

Figura 34. BBAS3

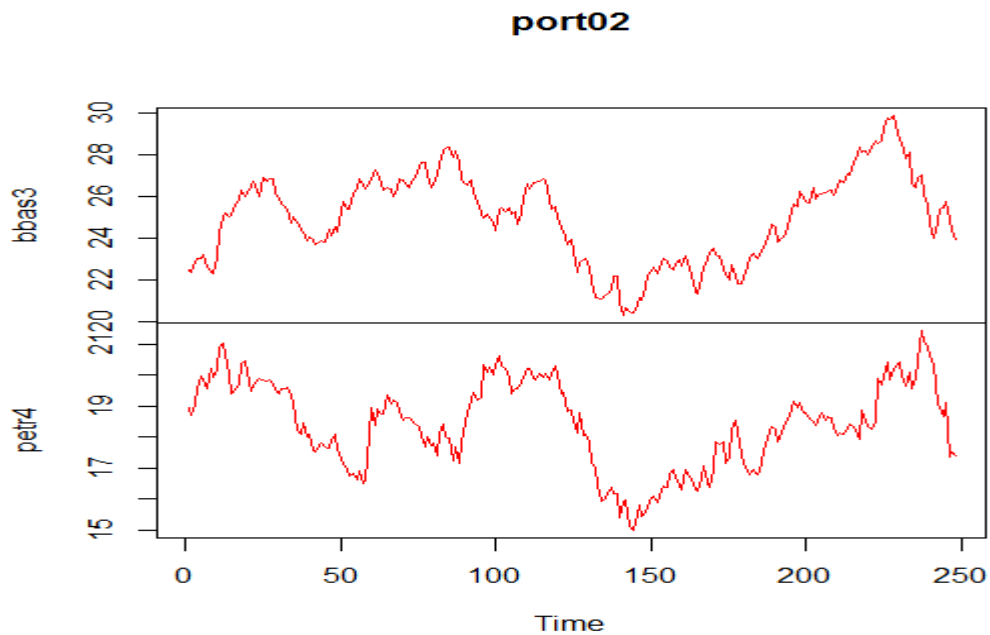


Fonte: Elaboração própria

Agora o objetivo consiste em criar um objeto denominado port02 com as duas séries no mesmo objeto. Execute os comandos a seguir.

```
port02<-ts(port01[,5:6])  
plot(port02, col=2) #Faz o gráfico das duas séries separadas
```

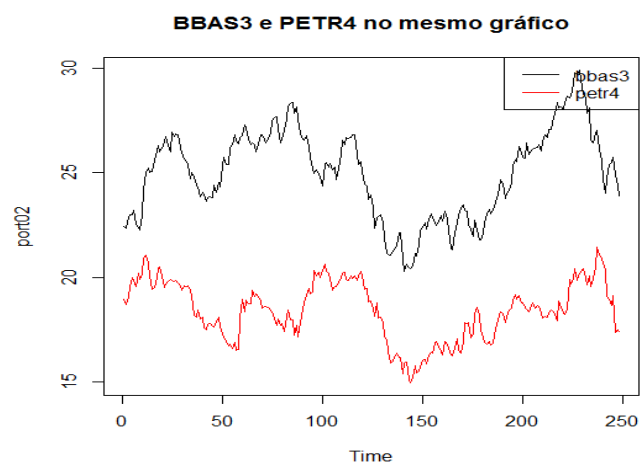
Figura 35. Port02



Fonte: Elaboração própria

```
plot(port02, plot.type = "single")  
plot(port02, plot.type = "single", col=1:2, main="BBAS3 e PETR4 no  
mesmo gráfico") legend("topright", legend = colnames(port02), col = 1:2, lty = 1)
```

Figura 36. BBAS3 e PETR4 no mesmo gráfico

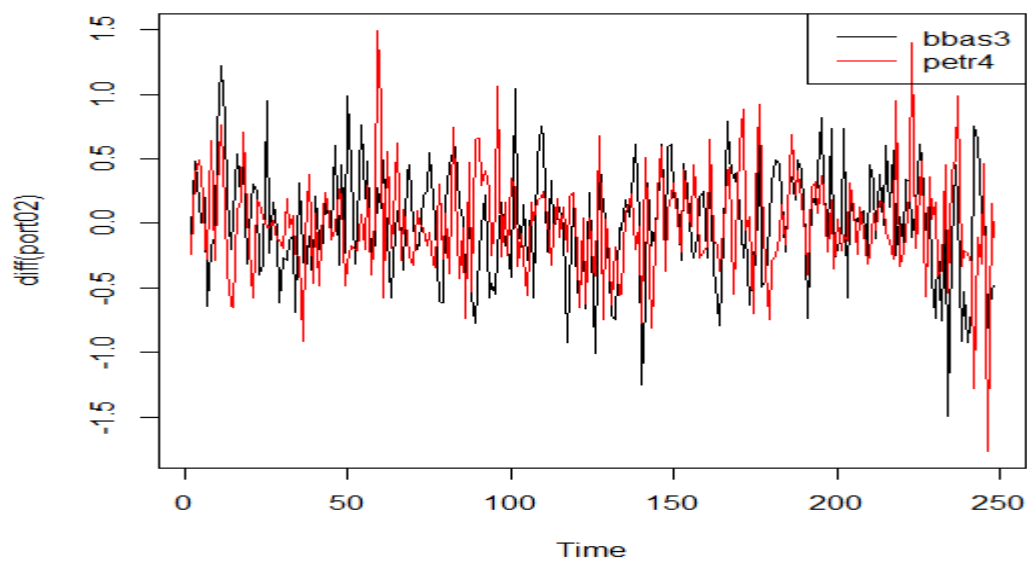


Fonte: Elaboração própria

Para fazer a primeira diferença da série de preços use o comando `diff(port02)`, essa informação reflete a taxa de variação dos preços das ações. Para coloca-las no gráfico, faça:

```
plot(diff(port02), plot.type = "single", col=1:2)
legend("topright", legend = colnames(port02), col = 1:2, lty = 1)
```

Figura 37. Comando `diff(port02)`

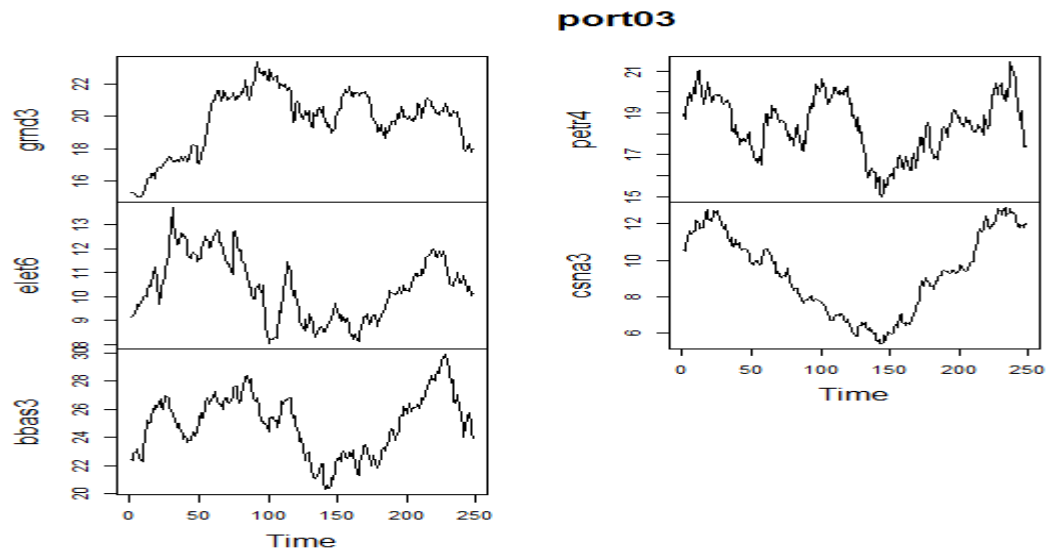


Fonte: Elaboração própria

Agora com todas as séries, crie o objeto `port03`

```
port03<-ts(port01[,3:7])
plot(port03) #Gráficos separados
```

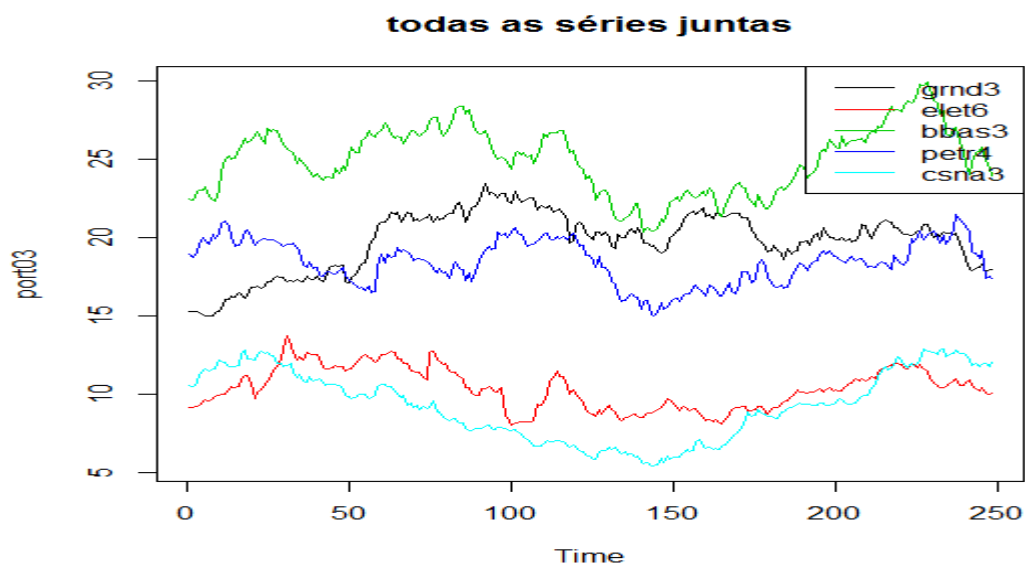
Figura 38. Port03



Fonte: Elaboração própria

```
plot(port03, plot.type = "single", col=1:5, main="todas as séries juntas")
legend("topright", legend = colnames(port03), col = 1:5, lty = 1)
```

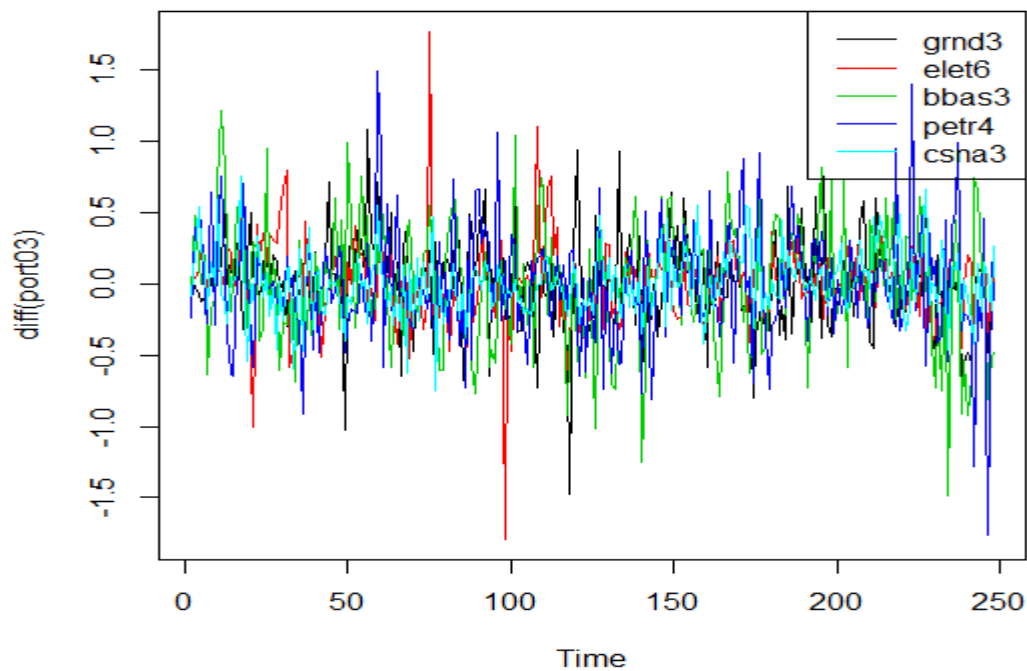
Figura 39. Todas as séries juntas



Fonte: Elaboração própria

```
diff(port03)
plot(diff(port03), plot.type = "single", col=1:5)
legend("topright", legend = colnames(port03), col = 1:5, lty = 1)
```

Figura 40. diff(port03)



Fonte: Elaboração própria

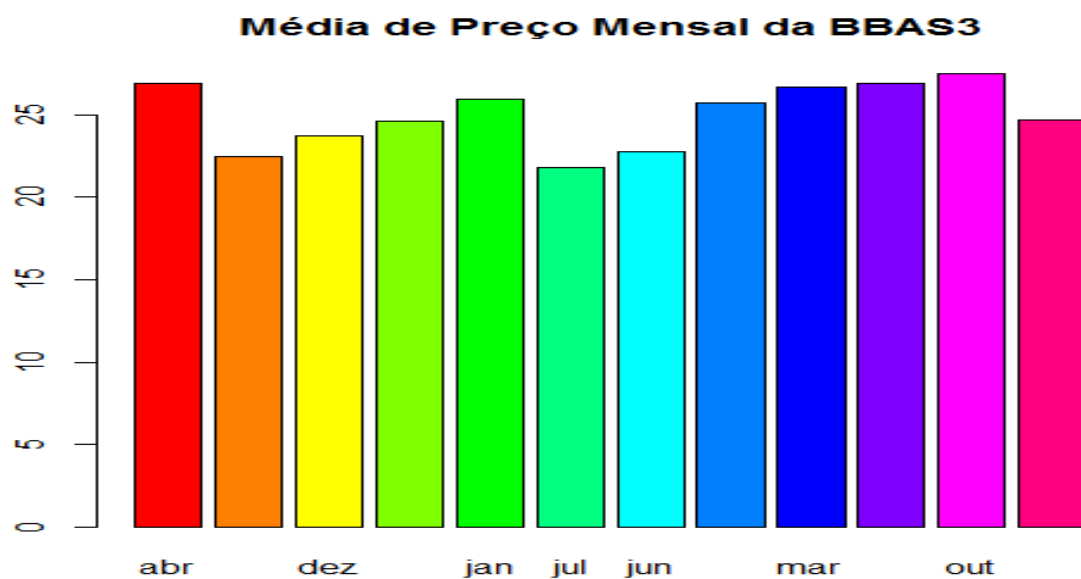
Agora voltando ao Dataframe port01, observe que as duas primeiras colunas são variáveis do tipo caractere, ou seja, não numéricas. Note também que a segunda coluna, referente ao mês, é uma variável categórica e, portanto, podemos manipular o banco de dados cruzando variáveis numéricas com categóricas. Os comandos `tapply` e `aggregate` fazem essas interações.

```
port01<-read.table(file.choose(),header=T)
port01
port01[,2] #Variável categórica referente ao mês
tapply(port01$bbas3,port01[,2], mean) # Média de preços da ação BBAS3
por mês
```

Para colocar no gráfico

```
barplot(tapply(port01$bbas3,port01[,2], mean), col = rainbow(12),
main="Média de Preço Mensal da BBAS3")
```

Figura 41. Média de preço mensal da BBAS3



Fonte: Elaboração própria

```
aggregate(port01[,3:7], list(port01[,2]), mean) #Média de preço mensal de  
todas as ações em estudo.
```

Após manipulação das séries com `tapply` e `aggregate`, construindo gráficos de médias mensais. Agora, retomaremos as séries de preços das ações PETR4 (Petrobras) e BBAS3 (Banco do Brasil), de modo a explorar novas formas de manipulação e filtragem dos dados.

O objetivo é identificar os dias em que os preços dessas ações se situaram acima de suas próprias médias históricas, bem como examinar os efeitos desse comportamento sobre as demais ações presentes no *dataframe* `port01`. Para tanto, serão empregados comandos como `which()` (para localizar índices que atendem a determinadas condições), recortes condicionais do *dataframe* e a função `colMeans()` (para calcular as médias por coluna).

Esses procedimentos permitem, por exemplo, verificar se o desempenho positivo de determinados ativos se associa a aumentos generalizados nos preços de outros papéis da carteira analisada, oferecendo uma perspectiva prática de como variáveis individuais podem influenciar o comportamento agregado do portfólio.

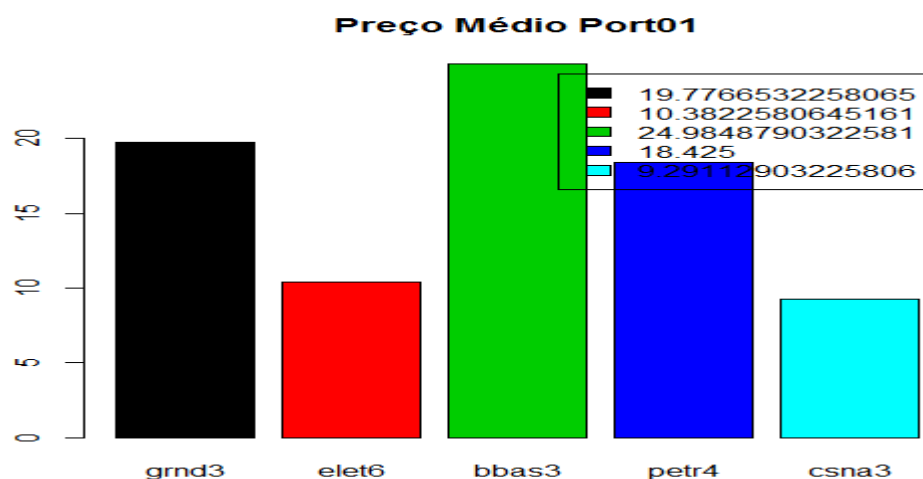
Agora, retome as atenções para as séries PETR4 e BBAS3 para proceder outras manipulações

```

petr4<-port01$petr4
petr4
bbas3<-port01$bbas3
which(port01$petr4>=mean(petr4)) #Diz quais as linhas o preço da ação
da Petrobrás operou acima da sua própria média.
port01[which(port01$petr4>=mean(petr4)),] #Recorta o Dataframe port01
para apenas aquelas linhas indicadas no passo anterior.
port01[which(port01$petr4>=mean(petr4)&port01$bbas3>=mean(bbas3))
,] #Recorta o Dataframe port01 para todas as linhas (representada pelos dias)
nas quais tanto a Petrobrás quanto o Banco do Brasil operaram acima das suas
próprias médias.
colMeans(port01[,3:7]) #Média de preço das ações
colMeans(port01[which(port01$petr4>=mean(petr4)),3:7]) #Média de
preço de todas as ações quando a Petrobrás operou acima da sua média.
colMeans(port01[which(port01$petr4>=mean(petr4)&port01$bbas3>=me
an(bbas3)),3:7]) #Média de preço de todas as ações quando tanto a Petrobrás
quanto o Banco do Brasil operaram acima das suas próprias médias.
media<-colMeans(port01[,3:7])
barplot(media, col=1:5, main="Preço Medio Port01",legend=c(media))

```

Figura 42. Preço médio Port01



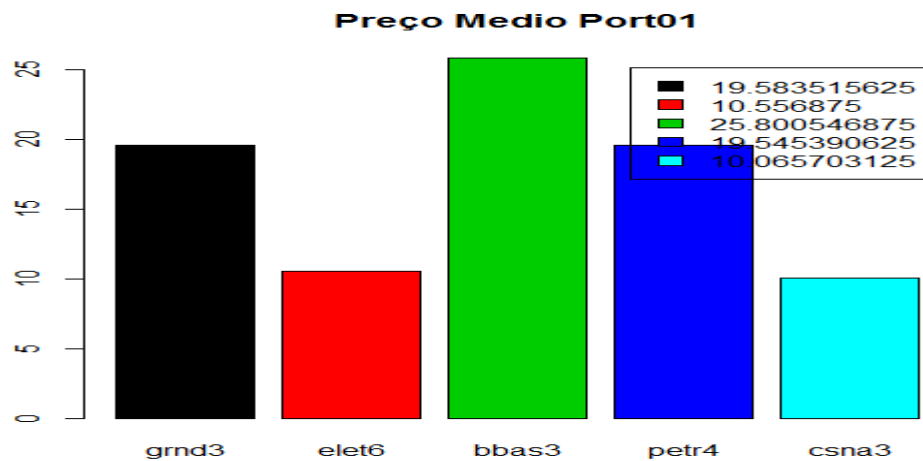
Fonte: Elaboração própria

```

media2<- colMeans(port01[which(port01$petr4>=mean(petr4)),3:7])
barplot(media2, col=1:5, main="Preço Medio Port01", legend=c(media2))

```

Figura 43. Preço medio Port01



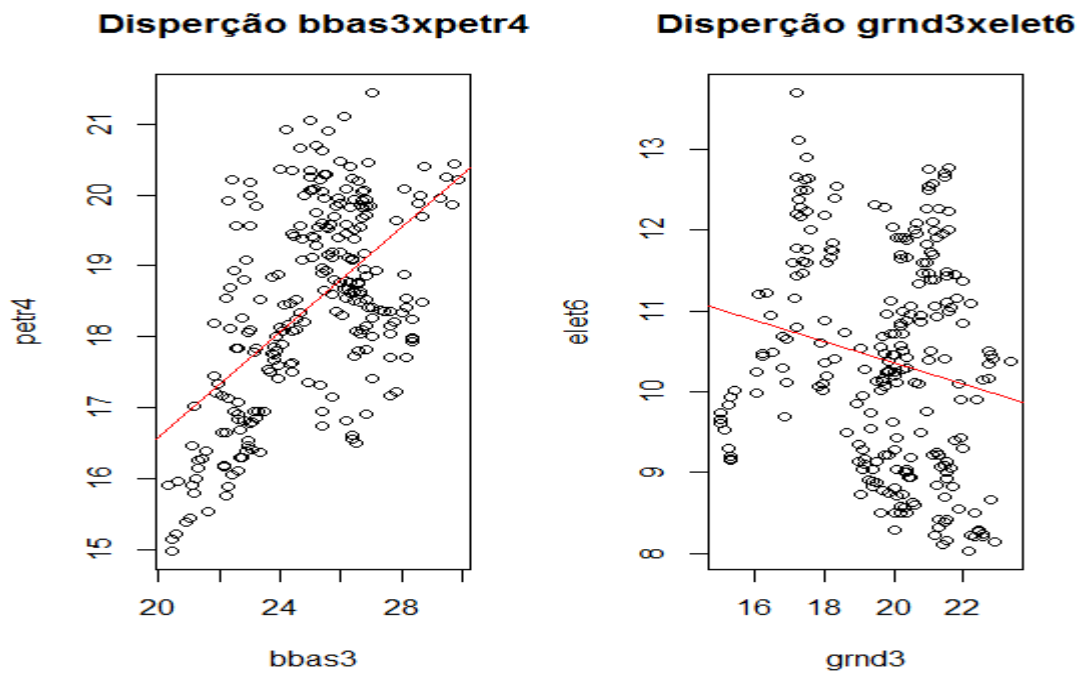
Fonte: Elaboração própria

Perceba que com exceção da Grendene (grnd3), quando a Petrobras opera acima da sua média todas as outras ações possuem uma média de preço maior que o período completo.

Agora, um gráfico de dispersão usando as ações do dataframe port01

```
attach(port01)
par(mfrow=c(1,2))
plot(bbas3,petr4, xlab="bbas3", ylab="petr4", main="Disperção
bbas3xpetr4")
abline(lm(petr4~bbas3), col=2)
plot(port01[,3],port01[,4], xlab="grnd3", ylab="elet6", main="Disperção
grnd3xelet6")
abline(lm(port01[,4]~port01[,3]), col=2)
```

Figura 44. Dispersão



Fonte: Elaboração própria

7.5 RETORNOS DAS AÇÕES

Para fazer os retorno de uma ação, lembre-se da fórmula do retorno discreto, dada por

$$R_t = \frac{P_t - P_{t-1} + D_t}{P_{t-1}} \quad (13)$$

Onde:

- R_t : Retorno das ações
- P_t : Preço no tempo t
- P_{t-1} : Preço no tempo $t-1$
- D_t : Dividendos no tempo t

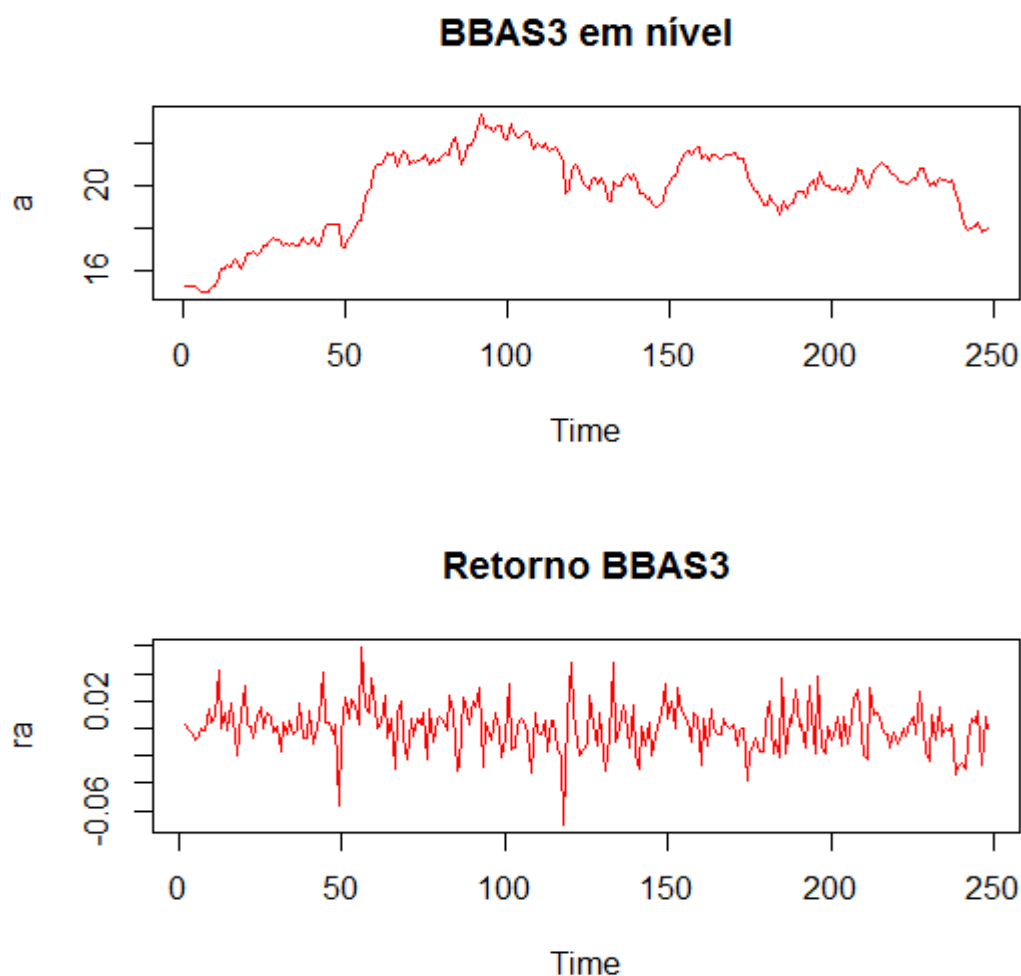
No exemplo a seguir, não será incorporado os dividendos, para fazer o retorno no numerador usa a função `diff` e no denominador a função `lag`.

```
port02<-port01[,3:7] #salvar só os preços das ações em um novo objeto
a<-ts(port02[,1])
b<-ts(port02[,2])
```



```
c<-ts(port02[,3])
d<-ts(port02[,4])
e<-ts(port02[,5])
ra<-diff(a)/lag(a,-1)
rb<-diff(b)/lag(b,-1)
rc<-diff(c)/lag(c,-1)
rd<-diff(d)/lag(d,-1)
re<-diff(e)/lag(e,-1)
retorno<-cbind(ra,rb,rc,rd,re) #agrupar os retornos em uma matriz
retorno
cor(retorno) #Correlação dos retornos
colMeans(retorno) #Média dos retornos
par(mfrow=c(2,1))
plot(a, main="BBAS3 em nível", col=2)
plot(ra, main="Retorno BBAS3", col=2)
```

Figura 45. BBAS3 em nível e retorno



Fonte: Elaboração própria

7.6 EXEMPLO COM ANÁLISE TÉCNICA DE AÇÕES

A análise técnica de ações estuda os movimentos passados dos preços e volumes para identificar padrões e tendências que possam orientar decisões de investimento. A Teoria de Dow, considerada a base da análise técnica, estabelece que os preços se movem em tendências (primárias, secundárias e terciárias) e que o mercado desconta todas as informações disponíveis. Segundo essa teoria, a confirmação do movimento por diferentes índices e o volume negociado são elementos centrais para validar uma tendência. Os pontos básicos da Teoria de Dow são:

1. Os índices descontam tudo: todos os possíveis fatores que afetam a cotação dos preços dos ativos (ações) são descontados por esses índices

que consideram todas as notícias, resultados contábeis e financeiros, acidentes e etc.

2. Os mercados se movem em tendências: as tendências podem ser de alta ou de baixa. Por sua vez, as tendências podem ser primárias, secundárias e terciárias, segundo sua duração. A tendência primária é a principal, representa o movimento mais longo do mercado e pode durar meses ou anos. As tendências secundárias são as correções e reações do mercado podendo retomar 1/3 a 2/3 do movimento anterior e duram semanas ou meses. Já as tendências terciárias são as correções e reações menores que duram algumas semanas.
3. Princípio de confirmação: para confirmar uma tendência é necessário que os índices Dow Jones Industrial Average (DJIA) e Dow Jones Transportation Average (DJTA) coincidam com a tendência.
4. Volume convergente: quando o mercado mudar a tendência haverá um aumento expressivo no volume das negociações. Dow acreditava que volumes menores eram apenas algum vendedor ou comprador agressivo, e que para mudar uma tendência precisava que um bom volume confirmasse. Enquanto não houver um volume alto confirmando a mudança de tendência, vale a tendência anterior.
5. Utiliza as cotações de fechamento para o cálculo das médias: não leva em conta os máximos e mínimos para o cálculo de seus índices.
6. A tendência é vigente até que seja substituída por outra oposta: até que os índices se confirmem, considera-se que a tendência antiga segue em vigor, apesar dos sinais aparentes de mudança. Este princípio procura evitar a prematura troca de posição (comprada ou vendida) .

A análise técnica é uma ferramenta amplamente utilizada no mercado financeiro para avaliar o comportamento passado dos preços e identificar padrões que possam auxiliar na previsão de movimentos futuros. Entre os conceitos fundamentais dessa abordagem **estão a tendência, a resistência e o suporte:**

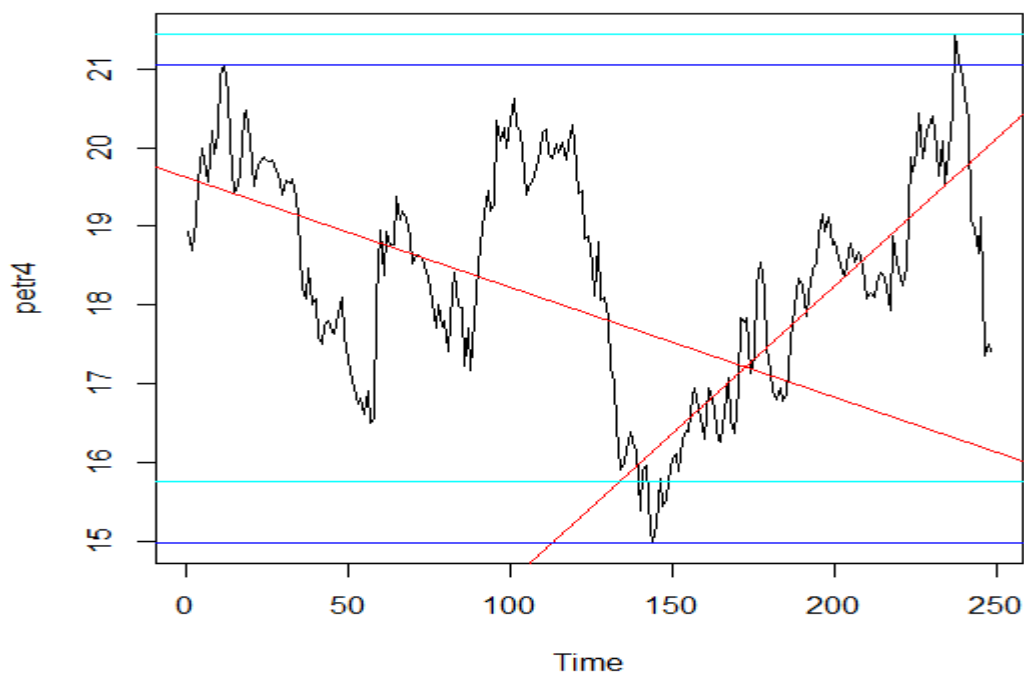
- **Tendência:** direção predominante dos preços ao longo de um período (ascendente, descendente ou lateral).
- **Resistência:** nível de preço que historicamente atua como barreira superior, dificultando a continuidade da alta.

- **Suporte:** nível de preço que funciona como piso, impedindo ou retardando a queda.

No exemplo a seguir, utilizaremos a série de preços da ação PETR4 para ilustrar esses conceitos. A série é dividida em dois períodos, permitindo a estimação de tendências lineares distintas por meio da função `lm()`. Em seguida, traçam-se linhas de suporte e resistência globais e específicas para cada subperíodo. Essa aplicação demonstra como a análise técnica pode ser operacionalizada no R para fins de visualização e interpretação do comportamento de preços.

```
port01<-read.table(file.choose(),header=T) #baixar o arquivo portfolio.txt
port01
petr4<-ts(port01$petr4)
length(petr4)
plot(petr4) #Observe que há uma quebra estrutural na observação 148
y1<-window(petr4, start=1, end=148)
y2<-window(petr4, start=149, end=248)
x1<-1:148
x2<-149:248
lm(y1~x1)
lm(y2~x2)
plot(petr4)
abline(lm(y1~x1), col=2) #tendência primeiro período
abline(lm(y2~x2), col=2) #tendência segundo período
abline(h=max(petr4), col=3) #Resistência global
abline(h=min(petr4), col=3) #Suporte global
abline(h=max(y1), col=4) #Resistência primeiro período
abline(h=min(y1), col=4) #Suporte primeiro período
abline(h=max(y2), col=5) #Resistência segundo período
abline(h=min(y2), col=5) #Suporte segundo período
```

Figura 46. PETR4



Fonte: Elaboração própria

Ao longo de 2013, a Petrobras esteve fortemente exposta a fatores políticos e macroeconômicos que impactaram o desempenho de suas ações. No primeiro semestre, a companhia sofreu com a política de controle de preços dos combustíveis, mantida pelo governo federal como instrumento de combate à inflação. A prática obrigava a estatal a importar derivados a preços elevados e revendê-los no mercado interno a valores artificialmente baixos, comprometendo sua rentabilidade e aumentando a percepção de risco entre investidores.

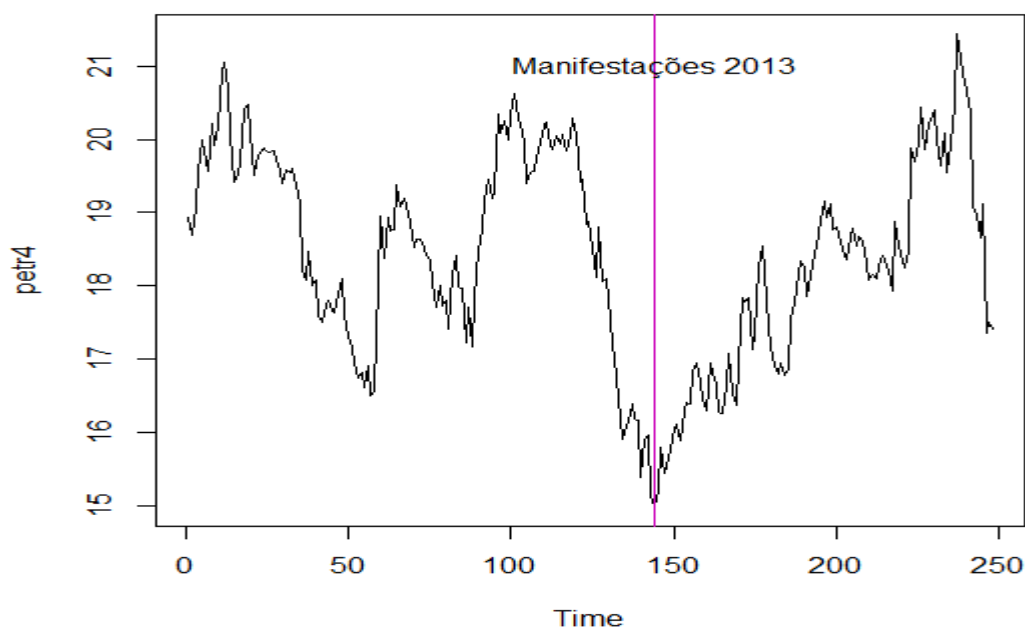
Esse cenário de fragilidade coincidiu com o ápice das manifestações populares de junho de 2013, quando, no dia 20, mais de um milhão de pessoas foram às ruas em todo o país, ampliando a percepção de instabilidade política e fiscal. O movimento, que havia começado com reivindicações em torno das tarifas de transporte público, incorporou críticas mais amplas ao modelo de gestão pública e aos gastos governamentais para a Copa do Mundo Fifa 2014..

Após esse período de forte pessimismo e queda nas cotações, as ações da Petrobras encontraram um fundo em meados de 2013, iniciando uma trajetória de recuperação no segundo semestre. Esse movimento foi sustentado tanto pela expectativa de ajustes graduais na política de preços de combustíveis

quanto pela elevação das cotações internacionais do petróleo, que favoreceram a receita da companhia.

```
plot(petr4) #Observe que há uma quebra estrutural na observação 148  
abline(v=144, col=6) #Suporte segundo período  
text(144,21, paste("Manifestações","2013"))
```

Figura 47. PETR4 – manifestações 2013



Fonte: Elaboração própria

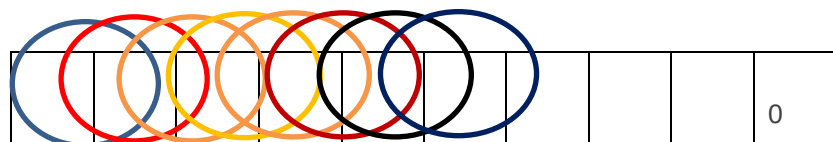
7.7 MÉDIAS MÓVEIS ARITMÉTICAS

As médias móveis constituem um dos instrumentos mais utilizados na análise de séries temporais e, em especial, na análise técnica de ações. Sua principal finalidade é suavizar flutuações de curto prazo, permitindo evidenciar o comportamento tendencial da série.

Uma média, como o próprio nome diz, mostra o valor médio de uma amostra de determinado dado. Uma média **móvel aritmética (MMA)** é uma extensão desse conceito, representando o valor médio, normalmente dos preços de fechamento, em uma janela móvel de tempo.

A cada novo período, entra uma observação nova e sai uma antiga na janela de dados.

Figura 48. móvel aritmética (MMA)



Fonte: Elaboração própria

$$MM(n) = \frac{P_1 + P_2 + \dots + P_N}{n} \quad (14)$$

Na fórmula acima, P_k , com k indo de 1 a n , representa os diferentes preços, enquanto que N é a janela de tempo sobre a qual se constrói a média. O parâmetro N é muito importante quando trabalhamos com médias móveis na análise gráfica, pois é a variável que iremos ajustar para obter melhores resultados. Modificando seu valor, a média irá responder mais ou menos rapidamente às variações de preços.

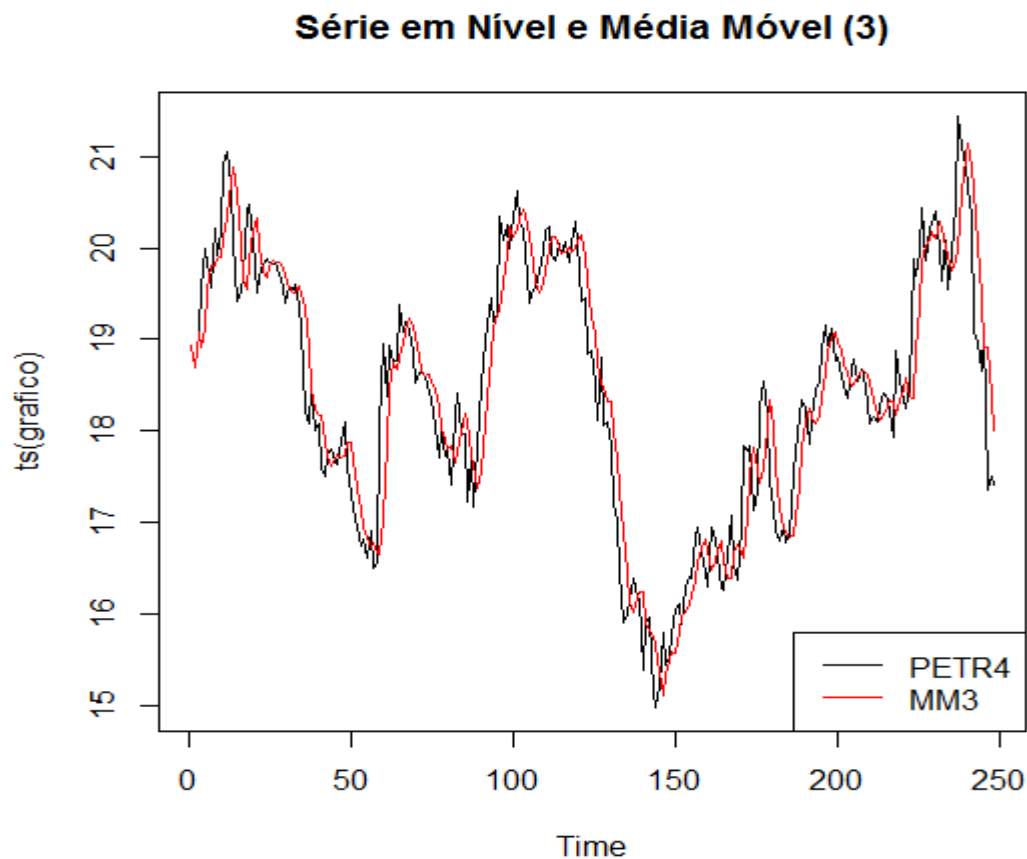
A Média Móvel Aritmética (MMA) de ordem k corresponde à média simples dos últimos k valores observados. No caso de ativos financeiros, essa técnica auxilia na identificação de movimentos de alta ou baixa de preços, reduzindo o ruído provocado por oscilações pontuais.

No exemplo a seguir, é construída uma Média Móvel de 3 períodos (MM3) para a série de preços da ação PETR4. O cálculo é feito de forma iterativa, armazenando em cada posição do vetor a média dos três preços imediatamente anteriores. Posteriormente, a série original e a respectiva média móvel são plotadas no mesmo gráfico, permitindo uma comparação visual entre os valores observados e a trajetória suavizada.

```
#Baixar os dados
port01<-read.table(file.choose(),header=T) #baixar o arquivo portfolio.txt
port01
petr4<-ts(port01$petr4)
length(petr4)
MM3<-numeric(248) #Média Móvel Aritmética de 3 períodos
MM3[1:3]<-petr4[1:3]
for(i in 4:248){
  MM3[i]<-(petr4[i-1]+petr4[i-2]+petr4[i-3])/3
}
MM3
```

```
grafico<-cbind(petr4,MM3)
plot(ts(grafico), plot.type="single", col=1:2, main="Série em Nível e Média
Móvel (3)")
legend("bottomright", legend = c("PETR4","MM3"), col = 1:2, lty = 1)
```

Figura 49. Série em nível e média móvel (3)



Fonte: Elaboração própria

7.8 ERRO MÉDIO PADRÃO DE PREVISÃO

O erro médio padrão de previsão é utilizado como medida de ajustamento de uma previsão dentro da amostra. Nas disciplinas de séries temporais, são estudados diversos modelos de previsão, como ARIMA, SARIMA, Função de Transferência etc. Um bom modelo de previsão para fora da amostra, é aquele que, em tese, consegue melhor se ajustar dentro da amostra.

Pode-se utilizar média móvel para previsão fora da amostra? Sim, em geral, os pesquisadores usam como uma espécie de benchmarking, ou seja, comparação com outros modelos de previsão.

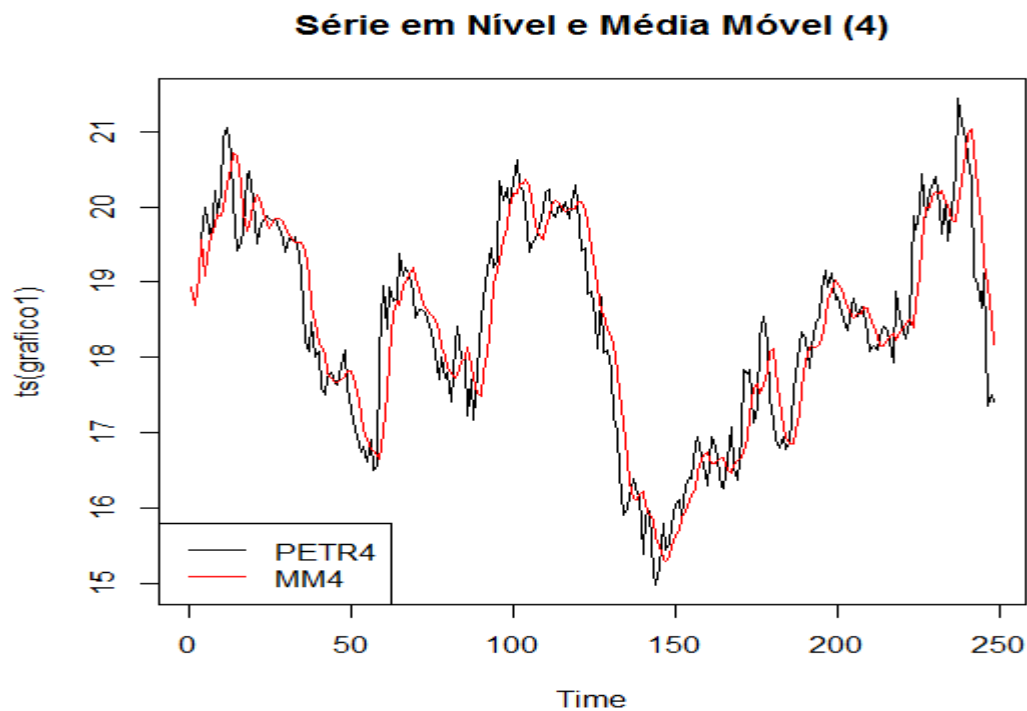
$$EMP = \sqrt{\sum \left(\frac{Y - \hat{Y}}{N} \right)^2} \quad (15)$$

```

erropadrao_MM3<-(sum((grafico[,1]-grafico[,2])^2)/248)^0.5
erropadrao_MM3
[1] 0.5107493
erropadrao_MM3<-(sum((grafico[,1]-grafico[,2])^2)/248)^0.5
erropadrao_MM3
[1] 0.5107493
MM4<-numeric(248)
MM4[1:4]<-petr4[1:4]
for(i in 5:248){
  MM4[i]<-(petr4[i-1]+petr4[i-2]+petr4[i-3]+petr4[i-4])/4
}
MM4
grafico1<-cbind(petr4,MM4)
grafico1
plot(ts(grafico1), plot.type="single", col=1:2, main="Série em Nível e
Média Móvel (4)")
legend("bottomleft", legend = c("PETR4","MM4"), col = 1:2, lty = 1)

```

Figura 50. Série em nível e média móvel (4)



Fonte: Elaboração própria

```
erropadiao_MM4<-(sum((grafico1[,1]-grafico1[,2])^2)/248)^0.5  
erropadiao_MM4  
[1] 0.5537795
```

Note que MM3 possui menor erro padrão do que MM4, logo se ajusta melhor a série.

```
length(petr4) #Visualiza a quantidade de observações  
plot(petr4)  
grafico1<-cbind(petr4,mm3,mm9,mm21)  
grafico1  
plot(ts(grafico1), plot.type="single", col=1:4, main="Série em Nível e  
Média Móvel c(3,9,21)")
```

Exercício proposto: Identificar Pontos onde MM3 > Petr4

```
cbind(MM3,petr4)  
estrategia<-numeric(248)  
for(i in 1:218){  
  if(MM3[i]>petr4[i]){  
    estrategia[i]<-1
```

```

}else{
  estrategia[i]<-0
}
}
estrategia

```

7.9 HABILITANDO PACOTE “QUANTMOD”

O pacote `quantmod` (*Quantitative Financial Modelling Framework*) é amplamente utilizado em finanças quantitativas para coleta, modelagem e visualização de séries temporais financeiras. Ele integra funções que permitem obter dados de diferentes fontes (como o Yahoo Finance), manipular objetos no formato de séries temporais e aplicar técnicas de análise técnica de maneira simplificada.

Nesta seção, será apresentado como instalar e habilitar o pacote `quantmod`, além de utilizá-lo para importar séries de preços de ações diretamente da internet. A função `getSymbols()` é o principal recurso para esse processo: ela possibilita baixar os dados de ativos financeiros, especificando o código do ativo, a fonte dos dados e o intervalo de tempo desejado.

Uma vez importadas, as séries podem ser exploradas no formato de *time series* ou *xts*, permitindo cálculos adicionais e a construção de gráficos especializados por meio da função `chartSeries()`.

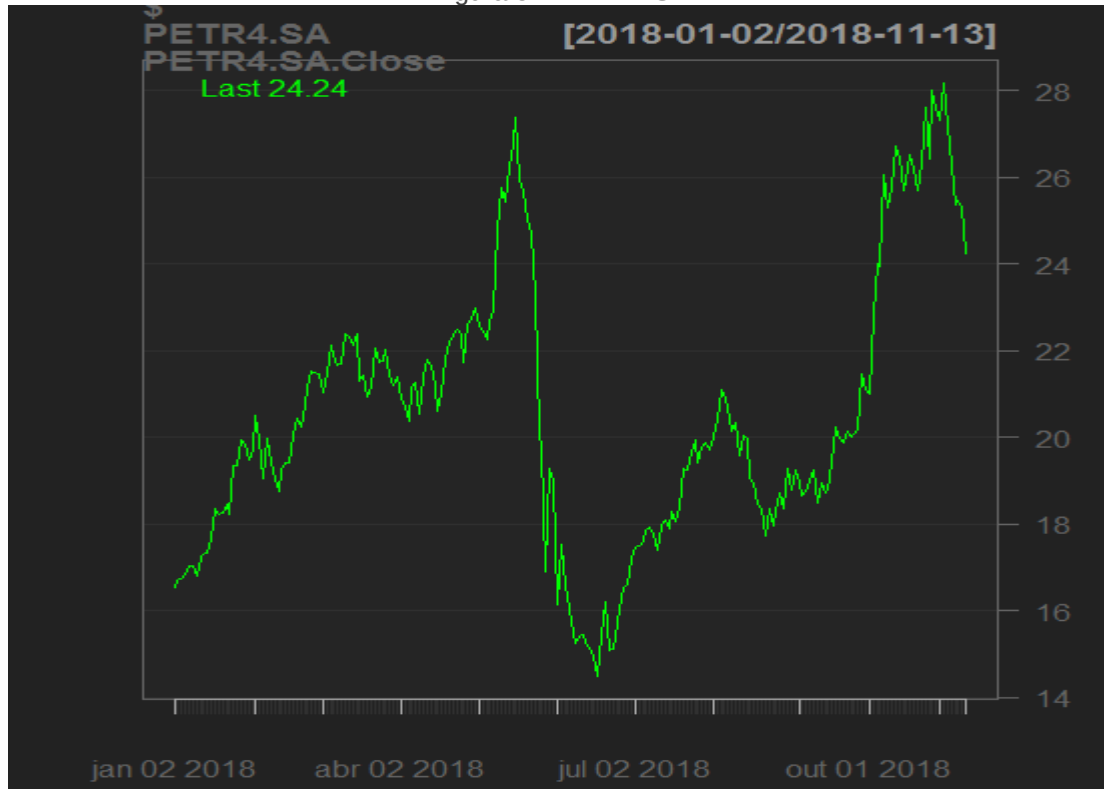
```

#Habilita o pacote quantmod
install.packages("quantmod")
library(quantmod)
getSymbols(c("PETR4.SA","BBAS3.SA"),
src="yahoo",from=as.Date("2018-01-01"),to=as.Date("2018-11-14"))
head(PETR4.SA)
head(BBAS3.SA)
#ou, para uma única série:
petr4base<-getSymbols("PETR4.SA", src="yahoo",from=as.Date("2018-
01-01"),to=as.Date("2018-11-14"), auto.assign = FALSE)
petr4base #Salva a base no objeto. Para baixar uma série, use a opção
auto.assign = FALSE

```

```
petr4<-ts(petr4base[,4]) #Cria o objeto petr4 como uma série temporal,  
com o preço de fechamento, localizado na 4ª coluna  
petr4  
length(petr4) #Visualiza a quantidade de observações  
chartSeries(PETR4.SA$PETR4.SA.Close) #ou  
chartSeries(petr4)
```

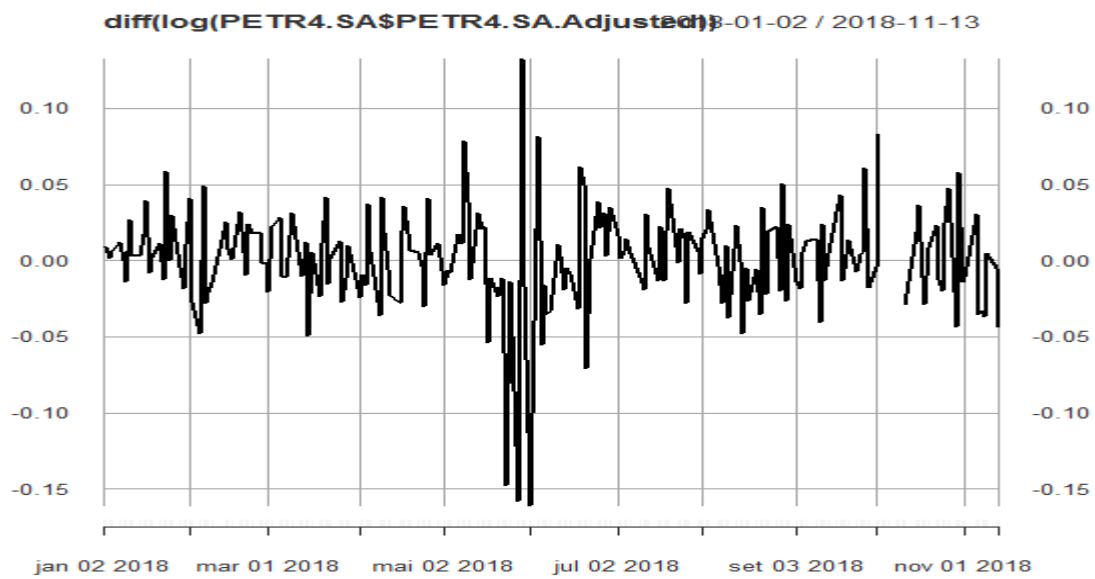
Figura 51. PETR4.SA



Fonte: Elaboração própria

```
plot(diff(log(PETR4.SA$PETR4.SA.Adjusted))) #Gráfico dos retornos dos  
preços ajustados
```

Figura 52. $\text{diff}(\log(\text{PETR4.SA}\$PETR4.SA.\text{Adjusted}))$



Fonte: Elaboração própria

```
candleChart(petr4base) #ou
chartSeries(petr4base, type = "candlesticks")
candleChart(petr4base, up.col = "green", dn.col = "red", theme="white",
name="PETR4")
addSMA(n = c(5,30), col=c(1,4))
```

Figura 53. PETR4



Fonte: Elaboração própria

7.10 IDENTIFICANDO EVENTOS NO GRÁFICO

Para identificar eventos de grande impacto no mercado acionário, é útil analisar os dias em que o Ibovespa apresentou oscilações mais intensas. Neste caso, consideramos como critério variações diárias absolutas superiores a 5%, sejam elas de alta ou de baixa. Para isso, utilizamos a função `dailyReturn()` do pacote `quantmod`, que calcula os retornos diários a partir dos preços de fechamento. Em seguida, aplicamos um filtro lógico com `abs(retorno) > 5`, de modo a selecionar apenas os dias em que a volatilidade ultrapassou esse limite. Esse procedimento nos permite isolar episódios de forte movimentação, geralmente associados a choques externos, crises políticas ou eventos macroeconômicos relevantes.

```
library(quantmod)

# 1) Baixar a série do Ibovespa (^BVSP) do Yahoo Finance
getSymbols("^BVSP", src="yahoo", from=as.Date("2015-01-01"),
to=as.Date("2022-12-31"))

# 2) Calcular retornos diários (%), removendo valores faltantes (NA)
# dailyReturn() gera a série de retornos com base no fechamento (Cl)
ret_diario <- na.omit(dailyReturn(Cl(BVSP), type="arithmetic") * 100)

# 3) Filtrar apenas os dias em que a variação foi maior que 5% (para cima
ou para baixo)
eventos_5 <- ret_diario[abs(ret_diario) > 5]

# 4) Extrair apenas as DATAS desses eventos
# index() retorna as datas de um objeto xts
datas_evt <- as.Date(index(eventos_5))

# 5) Preparar os vetores para o gráfico em base R
# x = datas da série completa
# y = valores do fechamento do Ibovespa
x <- index(Cl(BVSP)) # extrai todas as datas da série de preços
y <- as.numeric(Cl(BVSP)) # converte os preços de fechamento para
valores numéricos

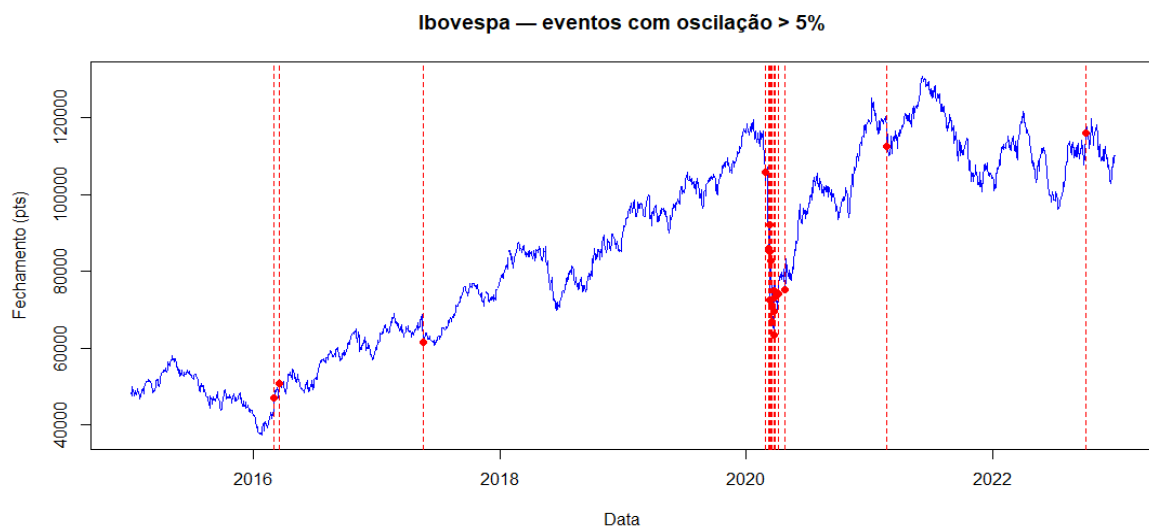
# 6) Plotar gráfico da série histórica do Ibovespa
plot(x, y, type = "l", col = "blue",
main = "Ibovespa — eventos com oscilação > 5%",
```

```

xlab = "Data", ylab = "Fechamento (pts)")
# 7) Adicionar linhas verticais vermelhas nas datas dos eventos
abline(v = datas_evt, col = "red", lty = 2, lwd = 1.5)
# 8) (Opcional) adicionar pontos vermelhos nos valores do fechamento
nessas datas
points(datas_evt, Cl(BVSP)[datas_evt], col="red", pch=19)

```

Figura 54. Ibovespa – eventos com oscilação > 5%



Fonte: Elaboração própria

Para identificar episódios de forte volatilidade no mercado, primeiro é necessário carregar o objeto `eventos_5`, que contém os dias em que o Ibovespa apresentou oscilações superiores a $\pm 5\%$. Esse objeto é obtido a partir do cálculo dos retornos diários da série de preços do índice e da aplicação de um filtro lógico ($\text{abs}(\text{retorno}) > 5$).

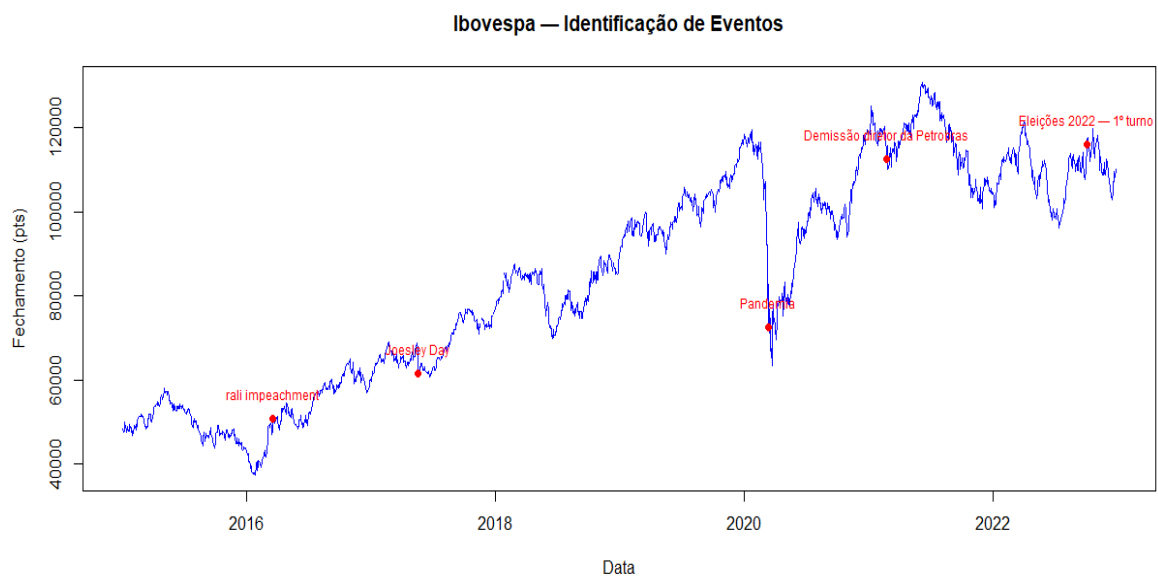
Uma vez carregado o `eventos_5`, é possível analisar manualmente quais eventos históricos estiveram associados a essas variações anormais. Assim, selecionamos datas representativas — como o rali pré-impeachment de 2016, o *Joesley Day* em 2017, os sucessivos circuit breakers da pandemia em 2020, a demissão do presidente da Petrobras em 2021 e o resultado do primeiro turno das eleições em 2022.

Para facilitar a interpretação visual, em vez de múltiplas linhas verticais que poluiriam o gráfico, optamos por destacar apenas um ponto por evento. Esses pontos são plotados próximos à curva do Ibovespa, acompanhados de

rótulos explicativos. Essa solução deixa o gráfico mais limpo e, ao mesmo tempo, mantém a identificação clara dos principais choques de mercado.

```
# Plot base R (mais amigável a abline/text)
x <- index(precos)
y <- as.numeric(precos)
plot(x, y, type = "l", col = "blue",
     main = "Ibovespa — Identificação de Eventos",
     xlab = "Data", ylab = "Fechamento (pts)")
# Eventos (uma data por evento)
evt <- data.frame(
  data = as.Date(c(
    "2016-03-17", # rali pró-impeachment (use esta como referência)
    "2017-05-18", # Joesley Day
    "2020-03-12", # Pandemia (dia mais emblemático da queda)
    "2021-02-22", # Demissão diretor da Petrobras (troca na presidência)
    "2022-10-03"  # Eleições 2022 — 1º turno
  )),
  label = c(
    paste("rali", "impeachment"),
    "Joesley Day",
    "Pandemia",
    "Demissão diretor da Petrobras",
    "Eleições 2022 — 1º turno"
  ),
  stringsAsFactors = FALSE
)
# Posição Y (preço) desses dias
px <- as.numeric(precos[evt$data])
# Plotar pontos e rótulos próximos à linha
points(evt$data, px, pch = 19, col = "red")
# pequeno deslocamento vertical para o texto não colar no ponto
y_off <- diff(range(as.numeric(precos), na.rm = TRUE)) * 0.02
text(evt$data, px + y_off, labels = evt$label,
     cex = 0.75, col = "red", pos = 3)
```


Figura 55. Ibovespa – Identificação de Eventos



Fonte: Elaboração própria



CAPÍTULO 08

ESTRATÉGIAS DE INVESTIMENTOS E BACKTESTING

A avaliação de estratégias de investimento pode ser realizada por meio de backtesting, isto é, a simulação de regras de compra e venda aplicadas sobre dados históricos de preços. Esse procedimento permite verificar como determinada estratégia teria se comportado no passado, fornecendo evidências sobre sua eficácia potencial.

Nesta seção, serão explorados três exemplos clássicos:

- **Buy and Hold:** consiste em comprar o ativo e mantê-lo durante todo o período, acumulando os retornos de forma contínua. É uma estratégia passiva, que busca capturar o crescimento de longo prazo do preço da ação.
- **Stop Loss:** estabelece um limite máximo de perda (neste caso, 5%). Sempre que o fator acumulado de retornos cai abaixo desse limite, a posição é encerrada, evitando maiores prejuízos. Essa é uma estratégia de proteção, comumente utilizada para mitigar riscos.
- **Cruzamento de Médias Móveis:** utiliza indicadores técnicos para definir pontos de entrada e saída. Quando a média móvel de curto prazo cruza a de longo prazo para cima, é gerado um sinal de compra; quando o cruzamento ocorre para baixo, é gerado um sinal de venda.

```
#Retorno Acumulado Petr4
#Buy and Hold
ra<-diff(log(petr4))
ra
fatora<-1+ra
fatora
fatorasb<-ts(na.omit(fatora)) #Excluir buracos da série
fatorasb
length(fatorasb)
prod(fatorasb) #retorno acumulado
[1] 1.132176
```

```

#Stop Loss
#Perda máxima 5%
fatoracum<-numeric(211)
fatoracum[1]<-fatorasb[1]
for(i in 2:211){
fatoracum[i]<-fatorasb[i]*fatoracum[i-1]
}
Fatoracum #verifique visualmente os períodos vendidos
stoploss<-numeric(211)
stoploss
for(i in 1:211){
if(fatoracum[i]>=0.95){
stoploss[i]<-fatorasb[i]
}else{
stoploss[i]<-1
}
}
stoploss
prod(stoploss)
[1] 1.391651

```

8.1 CRUZAMENTO DE MÉDIAS MÓVEIS

O pacote quantmod oferece funções diretas para extrair médias móveis

```

mm3 <- rollmean(petr4, 3,align = "right")
mm9 <- rollmean(petr4, 9,align = "right")
mm21 <- rollmean(petr4, 21,align = "right")
mm2 <- rollmean(petr4, 2,align = "right")
mm4 <- rollmean(petr4, 4,align = "right")
mm6 <- rollmean(petr4, 6,align = "right")
mm12 <- rollmean(petr4, 12,align = "right")
mm15 <- rollmean(petr4, 15,align = "right")
mm36 <- rollmean(petr4, 36,align = "right")
chartSeries(petr4, name="PETR4")

```

```
addSMA(n = c(5,30), col=c(7,4))
```

ATENÇÃO A ESTRATÉGIA:

Média móvel de curto prazo cruza a média móvel de longo prazo para
CIMA → COMPRA

Média móvel de curto prazo cruza a média móvel de longo prazo para
BAIXO → VENDE

A seguir, exemplos reais da estratégia

Figura 56. EMA (10)



Fonte: Elaboração própria

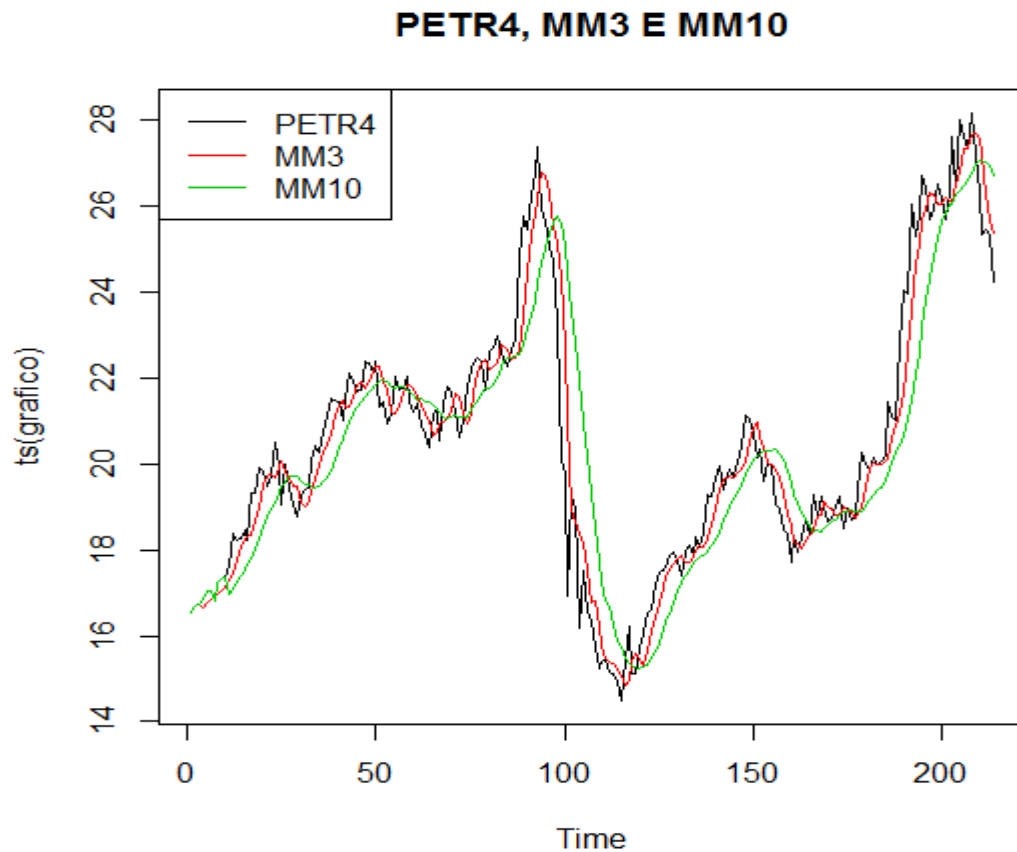
```
rm(petr4)
petr4<-ts(na.omit(petr4base[,4]))
petr4
length(petr4)
```

```

mm3<-numeric(214)
mm3
mm3[1]<-petr4[1]
mm3[2]<-petr4[2]
mm3[3]<-petr4[3]
for(i in 4:214){
mm3[i]<-(petr4[i-1]+petr4[i-2]+petr4[i-3])/3
}
mm3
plot(ts(mm3))
mm10<-numeric(214)
mm10[1:10]<-petr4[1:10]
mm10
for(i in 11:214){
mm10[i]<-(petr4[i-1]+petr4[i-2]+petr4[i-3]+petr4[i-4]+petr4[i-5]+petr4[i-
6]+petr4[i-7]+petr4[i-8]+petr4[i-9]+petr4[i-10])/10
}
mm10
grafico<-cbind(petr4,mm3,mm10)
plot(ts(grafico), plot.type="single", col=1:3)
legend("topleft", legend = c("PETR4","MM3","MM10"), col = 1:3, lty = 1)

```

Figura 57. PETR4, MM3 e MM10



Fonte: Elaboração própria

```
cruzmm<-numeric(214)
cruzmm
for(i in 1:214){
  if(mm3[i]>mm10[i]){
    cruzmm[i]<-fatorasb[i]
  }else{
    cruzmm[i]<-1
  }
}
cruzmm
prod(cruzmm)
[1] 1.524204
```

Note que a estratégia de cruzamento de médias móveis gera o maior retorno acumulado (52,42%), seguido por Stop Loss de 5% (39,16%) e por último Buy and Hold (13,21%).

8.2 PARABOLIC SAR

O Indicador SAR Parabólico foi desenvolvido por Welles Wilder com a finalidade de confirmar ou refutar a tendência, o início de uma correção, ou movimento lateral, assim como identificar possíveis pontos de saída. O princípio subjacente do indicador pode ser descrito pela frase "parada e virada".

O indicador SAR Parabólico dá os seguintes sinais:

Confirmação da tendência

- Localizada abaixo do gráfico dos preços, o indicador confirma a tendência ascendente;
- Localizado acima do gráfico dos preços, o indicador confirma a tendência descendente.



Fonte: Elaboração própria

$$P(t) = P(t-1) + AF \times (EP(t-1) - P(t-1)) \quad (16)$$

Onde:

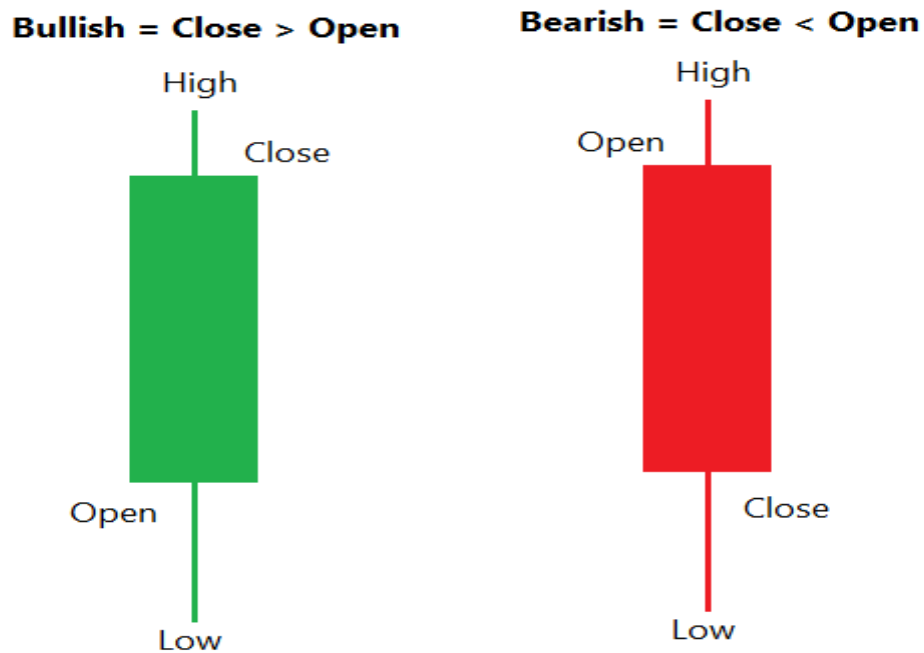
$P(t)$ – valor atual do indicador;

$P(t-1)$ – valor do indicador um período antes;

AF – fator de aceleração, aumenta em 0.02 no intervalo $[0.02; 0.2]$;

$EP(t-1)$ – valor mínimo/máximo do preço no período anterior.

Figura 59. Bullish e Bearish



Fonte: Elaboração própria

```
## Parabolic SAR
petr4base[1:5,]
close<-ts(petr4base[,4])
high<-ts(petr4base[,2])
low<-ts(petr4base[,3])
parabolic<-numeric(208)
parabolic[1]<-close[1]
parabolic
for(i in 2:208){
  if(close[i]>close[i-1]){
    parabolic[i]<- close[i-1] + 0.02*(high[i-1] - close[i-1])
  }else{
    parabolic[i]<- close[i-1]+ 0.02*(low[i-1] - close[i-1])
  }
}
parabolic
plot(ts(parabolic))
grafico1<-cbind(petr4,parabolic)
```



```

plot(ts(grafico1), plot.type="single", col=1:2)
legend("topleft", legend = c("PETR4","PARABOLIC SAR"), col = 1:2, lty =
1)

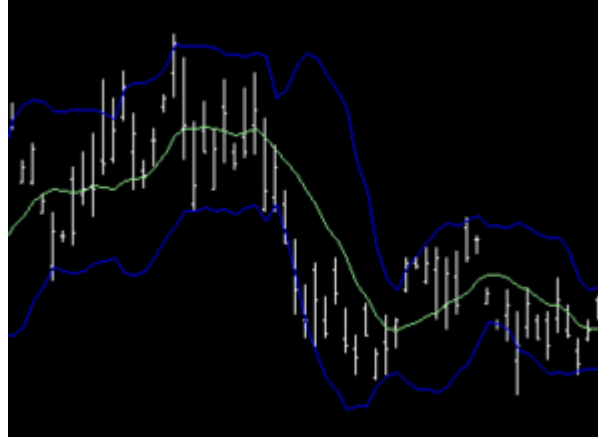
## Você só está comprado com PARABOLIC SAR POR CIMA
parabolic2<-numeric(207)
parabolic2
for(i in 1:207){
  if(parabolic[i]<close[i]){
    parabolic2[i]<-fatorasb[i]
  }else{
    parabolic2[i]<-1
  }
}
parabolic2
prod(parabolic2
1.110255

```

8.3 BANDAS DE BOLLINGER

Análise de Bollinger (também conhecida como Bandas de Bollinger) são ferramentas de análise técnica criadas por John Bollinger no início dos anos 80. Este indicador possui uma forte relação com a volatilidade, possibilitando, assim, a sua comparação com os níveis de preços num determinado período de tempo. O maior objetivo das bandas Bollinger é fornecer uma ideia relativa de alto e baixo. Por definição, os preços são altos na linha (banda) superior e baixos na linha (banda) inferior.

Figura 60. Bandas de Bollinger



Fonte: Elaboração própria

- Banda Superior = Média Móvel Simples (20 dias) + (2 x Desvio Padrão de 20 dias).
- Banda Inferior = Média Móvel Simples (20 dias) – (2 x Desvio Padrão de 20 dias).

```
candleChart(petr4base, up.col = "green", dn.col = "red", theme="white",
name="PETR4")
addSMA(n = c(5,30), col=c(1,4))
addBBands()
mm20<-rollmean(petr4, 20, align = "right")
desvio<-numeric(189)
for(i in 20:208){
desvio[i-19]<-sd(petr4[i-19:i])
}
desvio
length(desvio)
length(mm20)
banda_superior<- mm20 + 2*desvio
banda_inferior<- mm20 - 2*desvio
banda_superior<- c(rep(NA,19), mm20 + 2*desvio)
banda_inferior<- c(rep(NA,19), mm20 - 2*desvio)
grafico<-cbind(banda_superior,banda_inferior, petr4)
plot(grafico, main="Bandas de Bollinger", plot.type = "single", col
=c(2,2,1), lty = c(2,2,1) )
```

Figura 61. PETR4



Fonte: Elaboração própria

8.4 ÍNDICE DE FORÇA RELATIVA

O Índice de força relativa ou IFR mede a aceleração do movimento dos preços de determinado ativo e dá suas indicações à medida que o movimento diminui a velocidade, dentro da idéia de que é preciso desacelerar para poder mudar de direção. Para medir esta aceleração J. Welles Wilder em seu livro *New Concepts in Technical Trading Systems* (1978) desenvolveu o conceito de Força Relativa que é monitorar as mudanças nos preços de fechamento. Seu acompanhamento muitas vezes possibilita observar o enfraquecimento de uma tendência, rompimentos, suporte e resistência antes de se tornarem aparentes.

O IFR pode ser calculado como explicado pela seguinte equação:

$$I F R = 100 - 100 / (1 + (U / D)) = 100 / (1 + D / U) \quad (17)$$

Onde:

IFR= Índice de força relativa.

U= Média de todas as variações positivas no preço da ação dentro do período em estudo.

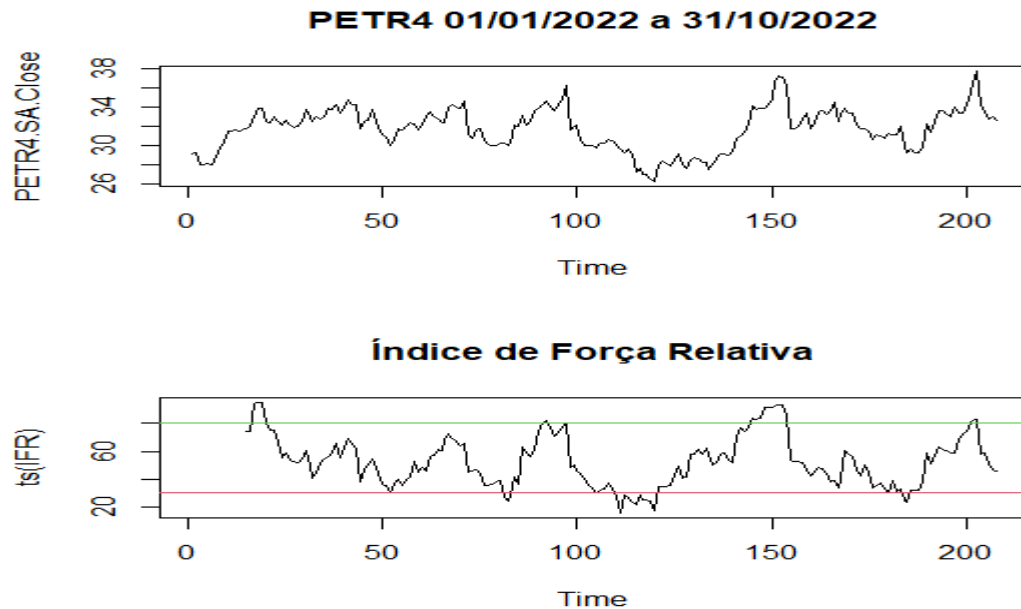
D= Média de todas as variações negativas no preço da ação dentro do período em estudo.

Obs: Usas a média de 14 períodos.

```
diff(petr4)
positivo<-numeric(207)
for(i in 1:207){
  if(diff(petr4)[i]>=0){
    positivo[i]<-diff(petr4)[i]
  }else{
    positivo[i]<-0
  }
}
positivo
negativo<-numeric(207)
for(i in 1:207){
  if(diff(petr4)[i]<0){
    negativo[i]<-abs(diff(petr4)[i])
  }else{
    negativo[i]<-0
  }
}
negativo
positivo_mm14 <- rollmean(positivo, 14, align = "right")
mean(positivo[1:14])
mean(positivo[2:15])
negativo_mm14 <- rollmean(negativo, 14, align = "right")
relacao<-positivo_mm14/negativo_mm14
formula_IFR<-100 - 100/(1+relacao)
IFR<-c(rep(NA,14),formula_IFR)
length(IFR)
par(mfrow=c(2,1))
plot(petr4, main = "PETR4 01/01/2022 a 31/10/2022")
plot(ts(IFR), main ="Índice de Força Relativa")
```

```
abline(h=30, col=2)
abline(h=80, col=3)
```

Figura 62. PETR4 01/01/2022 a 31/10/2022



Fonte: Elaboração própria

8.5 CRUZAMENTO DE TRÊS MÉDIAS

Para avaliar a viabilidade de estratégias de análise técnica, elaborou-se um procedimento de programação em R baseado no cruzamento de três médias móveis. A lógica dessa estratégia combina uma média de curto prazo (MM3), uma intermediária (MM10) e uma de longo prazo (MM21). A compra ocorre quando a média curta cruza para cima a média longa, exatamente como em estratégias de duas médias. A diferença central está na saída: em vez de esperar um novo cruzamento da média curta com a longa, utiliza-se o cruzamento com a média intermediária, o que permite realizar a venda de forma mais rápida e, em teoria, reduzir perdas em movimentos de reversão. O código apresentado a seguir implementa essa lógica, tratando valores faltantes iniciais, construindo a série temporal de posições (comprado ou fora do mercado) e calculando os retornos acumulados da estratégia.

```
library(quantmod)

getSymbols(c("PETR4.SA","BBAS3.SA"),
src="yahoo",from=as.Date("2018-01-01"),to=as.Date("2018-11-14"))
```

```

head(PETR4.SA)
head(BBAS3.SA)

petr4base<-getSymbols("PETR4.SA", src="yahoo",from=as.Date("2018-
01-01"),to=as.Date("2018-11-14"), auto.assign = FALSE)

petr4base #Salva a base no objeto. Para baixar uma série, use a opção
auto.assign = FALSE

petr4<-ts(petr4base[,4]) #Cria o objeto petr4 como uma série temporal,
com o preço de fechamento, localizado na 4ª coluna

petr4
length(petr4) #Visualiza a quantidade de observações

#Média móvel de curto prazo cruza a média móvel de longo prazo para
CIMA → COMPRA

#Média móvel curto prazo cruza a média móvel intermediária para BAIXO
→ VENDE

fech <- Cl(petr4base) #Cl() é um atalho do pacote quantmod para extrair
os preços de fechamento

#rollmean cria as médias móveis
mm3 <- rollmean(fech, 3, align = "right", fill = NA)
mm10 <- rollmean(fech, 10, align = "right", fill = NA)
mm21 <- rollmean(fech, 21, align = "right", fill = NA)

# Inicializa vetor de sinal
sinal <- rep(NA, length(fech)) #objeto vazio com NAs na quantidade n de
preços de fechamento)

sinal[1] <- 0 # primeira obserção igual a zero

#Preenchimento das demais informações
for(i in 2:length(fech)){

# Verifica se o valor atual OU o anterior são NA em alguma das séries,
evita erro nos primeiros dias pois as séries de médias moveis tem NAs no inicio.
if (is.na(mm3[i]) | is.na(mm10[i]) | is.na(mm21[i]) |
is.na(mm3[i-1]) | is.na(mm10[i-1]) | is.na(mm21[i-1])) {
sinal[i] <- sinal[i-1] # Mantém o valor anterior
} else if (mm3[i-1] < mm21[i-1] && mm3[i] > mm21[i]) {
sinal[i] <- 1 # Compra
} else if (mm3[i-1] > mm10[i-1] && mm3[i] < mm10[i]) {

```

```

sinal[i] <- 0 # Venda
} else {
sinal[i] <- sinal[i-1] # Mantém a posição anterior
}
}
sinal

sinal_xts <- xts(sinal, order.by = index(fech)) #transforma o objeto em
série temporal comando xts

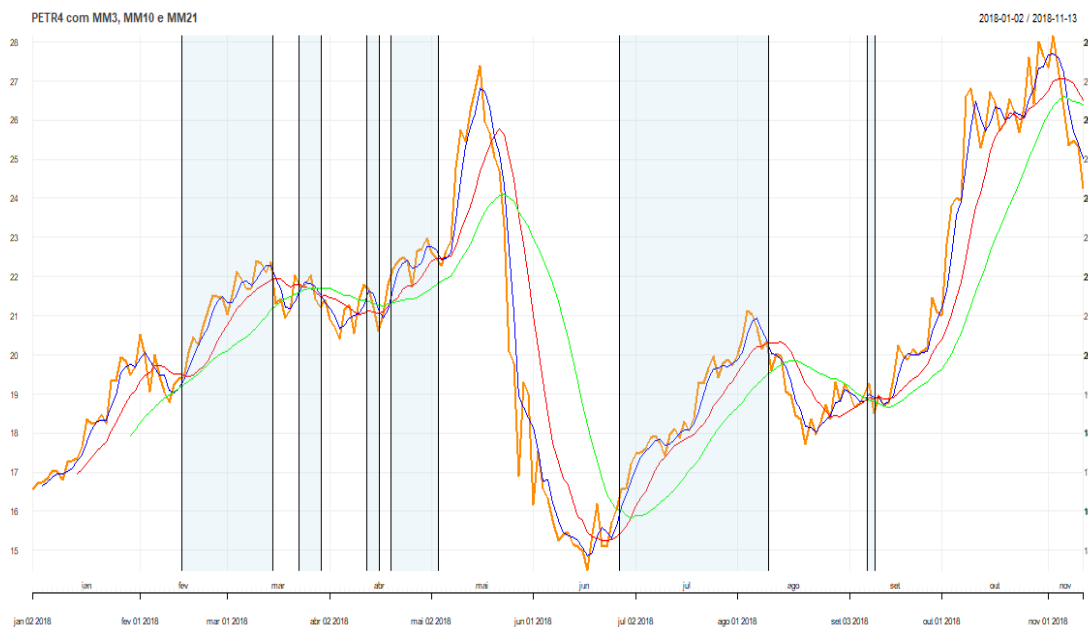
ret <- diff(log(fech))
ret <- na.omit(ret)
sinal_lag <- lag(sinal_xts, k = 1)
sinal_lag <- na.omit(sinal_lag)
ret_estrat <- ret * sinal_lag
ret_acumulado <- exp(cumsum(ret_estrat))
# Gráfico do preço + médias
chart_Series(fech, name = "PETR4 com MM3, MM10 e MM21")
add_TA(mm3, on=1, col="blue")
add_TA(mm10, on=1, col="red")
add_TA(mm21, on=1, col="green")
# Agora destacamos onde o sinal = 1
# 'sinal_xts' é tua série 0/1 já criada
areas <- sinal_xts == 1
# Adiciona sombreado transparente
add_TA(areas, on=1, type="bg", col=rgb(173,216,230, max=255,
alpha=50))
#173,216,230 = código RGB da cor lightblue
#alpha=50 → transparência (quanto menor, mais transparente; varia de 0
a 255

```

O gráfico mostra a evolução do preço da PETR4 acompanhada das três médias móveis: MM3 em azul, MM10 em vermelho e MM21 em verde. As áreas sombreadas em azul claro indicam os períodos em que a estratégia permaneceu comprada. Observa-se que os pontos de entrada (compra) ocorrem no mesmo instante em que ocorreria no modelo de duas médias (curta × longa). A diferença aparece nos pontos de saída: ao empregar uma média intermediária, a estratégia

aciona a venda mais cedo, evitando que o investidor permaneça comprado em momentos de queda prolongada. Esse recurso confere maior agilidade na gestão da posição, ao mesmo tempo em que mantém a simplicidade da lógica baseada em cruzamentos

Figura 63. PETR4 com MM3, MM10 e MM21



Fonte: Elaboração própria

Além da estratégia baseada em três médias móveis, também é possível avaliar o comportamento da versão tradicional de duas médias (curta $\times$ longa). Essa comparação é relevante porque, embora ambas utilizem o mesmo ponto de entrada, o cruzamento da média curta acima da média longa, diferem quanto ao critério de saída. Enquanto a estratégia simples de duas médias permanece comprada até que ocorra novamente o cruzamento da média curta para baixo da longa, a versão com três médias introduz uma média intermediária para antecipar o sinal de venda. Essa modificação busca reduzir a exposição a movimentos de queda, proporcionando maior agilidade na gestão da posição.

#duas médias versus três médias

--- Estratégia de 2 médias (MM3 x MM21) ---

```
senal2 <- rep(NA, length(fech))
```

```
senal2[1] <- 0
```

```
for(i in 2:length(fech)){
```

```
  if (is.na(mm3[i]) | is.na(mm21[i]) | is.na(mm3[i-1]) | is.na(mm21[i-1])) {
```



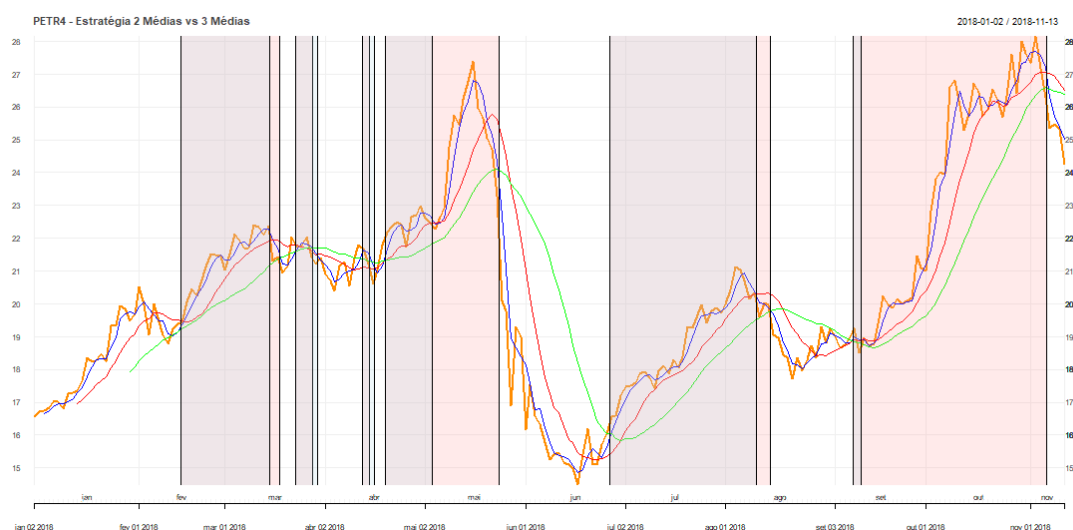
```

sinal2[i] <- sinal2[i-1]
} else if (mm3[i-1] < mm21[i-1] && mm3[i] > mm21[i]) {
sinal2[i] <- 1 # Compra
} else if (mm3[i-1] > mm21[i-1] && mm3[i] < mm21[i]) {
sinal2[i] <- 0 # Venda
} else {
sinal2[i] <- sinal2[i-1]
}
}

sinal2_xts <- xts(sinal2, order.by = index(fech))
# --- Gráfico comparando ---
chart_Series(fech, name = "PETR4 - Estratégia 2 Médias vs 3 Médias")
add_TA(mm3, on=1, col="blue")
add_TA(mm10, on=1, col="red")
add_TA(mm21, on=1, col="green")
# Áreas de compra (3 médias = azul claro, 2 médias = vermelho claro)
areas3 <- sinal_xts == 1
areas2 <- sinal2_xts == 1
add_TA(areas2, on=1, type="bg", col=rgb(255,150,150, max=255,
alpha=50)) # vermelho claro
add_TA(areas3, on=1, type="bg", col=rgb(173,216,230, max=255,
alpha=50)) # azul claro

```

Figura 64. Estratégia 2 Médias vs 3 Médias



Fonte: Elaboração própria

O gráfico apresenta a trajetória da ação PETR4 com as médias móveis de 3, 10 e 21 períodos, acompanhadas de duas áreas sombreadas: em vermelho claro, os intervalos em que a estratégia tradicional de duas médias estaria comprada; em azul claro, os períodos em que a estratégia de três médias se manteve posicionada. A sobreposição das áreas evidencia que os pontos de compra coincidem nas duas abordagens, mas a saída ocorre mais cedo na estratégia de três médias. Assim, visualiza-se a principal vantagem dessa adaptação: ao reduzir a duração da posição em momentos de reversão, a estratégia busca mitigar perdas potenciais sem alterar o critério de entrada.



CAPÍTULO 09

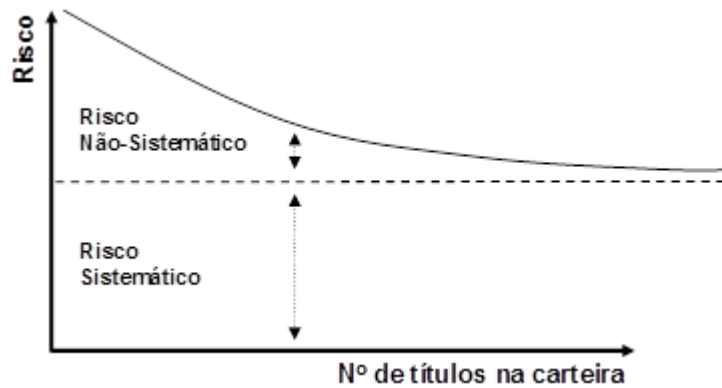
INTRODUÇÃO A TEORIA DA CARTEIRA

A Teoria da Carteira, desenvolvida por Harry Markowitz (1952), constitui um dos marcos fundamentais das Finanças Modernas. Seu objetivo central é demonstrar como os investidores podem combinar ativos de forma eficiente, de modo a maximizar o retorno esperado para um dado nível de risco ou, inversamente, minimizar o risco para um determinado nível de retorno.

DOIS ATIVOS DE RISCO

- Harry Markowitz desenvolveu a Teoria Moderna da Carteira em 1952, sua obra foi premiada com o Nobel de economia em 1990.
- A teoria é desenvolvida na análise Risco x Retorno. As métricas utilizadas são desvio padrão e retorno esperado, respectivamente.
- A teoria tem como premissa básica que a diversificação de ativos de uma carteira reduz o risco.
- Ativos negativamente correlacionados anulam os ganhos e perdas, reduzindo a volatilidade da carteira. Métrica utilizada é o coeficiente de correlação.
- Os ativos precisam atender o princípio da dominância: Ativos com retornos mais elevados exigem um risco maior.
- A teoria é desenvolvida em um cenário de Hipótese de Mercados Eficientes (HME), ou seja, não há assimetria de informação, nenhum agente é grande suficiente para influenciar o preço de mercado e todas as informações disponíveis são imediatamente incorporadas aos preços.
- Há dois riscos considerados, o risco de toda a economia chamado de risco de sistêmico e portanto não diversificável; e o risco de um setor da economia, chamado de risco específico e portanto diversificável.

Figura 65. Risco de sistêmico



Fonte: Elaboração própria

Ex: O investidor pode anular sua exposição ao risco em petróleo ao incluir na composição do portfólio dois ativos que reagem inversamente proporcionais as oscilações da commodities, como petrolíferas e setor aéreo.

9.1 MÉDIA E VARIÂNCIA DA CARTEIRA

Considere dois Ativos de risco, A e B:

1) O Retorno esperado do Portfólio P é dado por:

$$E(R_p) = w_A E(R_A) + w_B E(R_B), \quad \text{onde } w_A + w_B = 1 \quad (18)$$

2) A variância dos retornos do Portfólio P é dada por:

$$V(R_p) = \sigma_p^2 = (w_A \sigma_A)^2 + (w_B \sigma_A)^2 + 2w_A w_B \text{Cov}(A, B) \quad (19)$$

Onde:

R_p = Retorno do Portfólio P composto por ativos A e B.

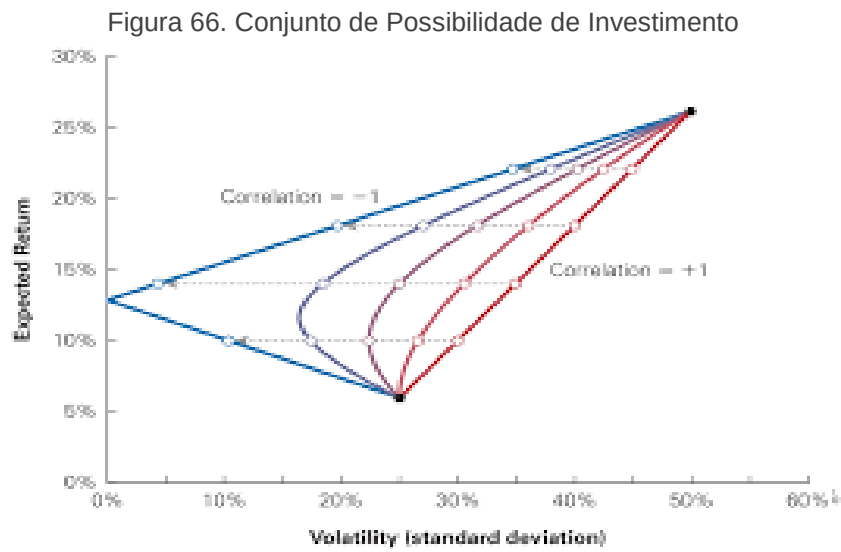
$E(R_A)$ e $E(R_B)$ = Retornos esperados dos ativos A e B, respectivamente. Resultado da média ponderada entre os cenários (recessão, normal e crescimento rápido) para cada Ativo individual.

w_A e w_B = pesos dos ativos A e B na carteira P.

$$\text{Cov}(A, B) = \sigma_A \sigma_A \rho_{AB}$$

9.1.1 Conjunto de possibilidade de investimento

O Conjunto de possibilidade de investimento representa todas as combinações possíveis, com proporção w_A do ativo A e w_B do ativo B, com $w_A + w_B = 1$, que formam uma carteira arriscada de dois ativos A e B, posicionados nos extremos do conjunto. Se o coeficiente de correlação for +1 o conjunto é uma reta com a combinação linear dos dois ativos A e B, à medida que a correlação vai se aproximando de -1 haverá ganhos na diversificação, com a carteira P reduzindo o risco em relação aos ativos individuais.



Problema do Investidor (Minimizar o Risco)

$$\min_{w_A} (w_A \sigma_A)^2 + (w_B \sigma_A)^2 + 2w_A w_B \text{Cov}(A, B) \quad (20)$$

$$\min_{w_A} (w_A \sigma_A)^2 + (1 - w_A)^2 \sigma_A^2 + 2w_A(1 - w_A) \text{Cov}(A, B) \quad (21)$$

$$\min_{w_A} (w_A \sigma_A)^2 + (1 - w_A)^2 \sigma_A^2 + 2w_A \text{Cov}(A, B) - 2w_A^2 \text{Cov}(A, B) \quad (22)$$

$$\frac{\partial}{\partial w_A} = 2w_A \sigma_A^2 - 2(1 - w_A) \sigma_A^2 + 2 \text{Cov}(A, B) - 4w_A \text{Cov}(A, B) = 0 \quad (23)$$

$$2w_A \sigma_A^2 - 2(1 - w_A) \sigma_A^2 + 2 \text{Cov}(A, B) - 4w_A \text{Cov}(A, B) = 0 \quad (24)$$

$$2w_A\sigma_A^2 - 2\sigma_A^2 + 2w_A\sigma_A^2 + 2Cov(A,B) - 4w_A Cov(A,B) = 0 \quad (25)$$

$$w_A\sigma_A^2 + w_A\sigma_A^2 - 2w_A Cov(A,B) = \sigma_A^2 - Cov(A,B) \quad (26)$$

$$w_A(\sigma_A^2 + \sigma_A^2 - 2Cov(A,B)) = \sigma_A^2 - Cov(A,B) \quad (27)$$

$$w_A^* = \frac{\sigma_B^2 - Cov(A,B)}{\sigma_A^2 + \sigma_B^2 - 2Cov(A,B)} \quad (28)$$

w_A^* = peso no ativo A na carteira de variância mínima.

$$w_B^* = 1 - w_A^* \quad (29)$$

Tabela 4. Exemplo

Cenário	Probabilidade	Taxa de Retorno	
		Ativo A	Ativo B
Recessão	1/3	-7%	+17%
Normal	1/3	+12%	+7%
Crescimento Rápido	1/3	+28%	-3%

Fonte: Elaboração própria

Tabela 5. Exemplo

Cenário	Ativo A			Ativo B		
	Taxa de Retorno	Desvio do Retorno Esperado	Desvio ao Quadrado	Taxa de Retorno	Desvio do Retorno Esperado	Desvio ao Quadrado
Recessão	-7%	-18%	324	17%	10%	100
Norma	12%	1%	1	7%	0%	0
Cresc Rápido	28%	17%	289	-3%	-10%	100
Retorno Esperado	1/3 x (-7% + 12% + 28%) = 11%			1/3 x (17% + 7% - 3%) = 7%		
Variância	1/3 x (324 + 1 + 289) = 204,7			1/3 x (100 + 0 + 100) = 66,7		
Desvio Padrão *	14,3%			8,2%		

Fonte: Elaboração própria

Tabela 6. Exemplo

Cenário	Ativo A		Ativo B		Produto dos Desvios
	Taxa de Retorno	Desvios do Retorno Esperado	Taxa de Retorno	Desvios do Retorno Esperado	
Recessão	-7%	-18%	17%	10%	-180
Normal	12%	1%	7%	0%	0
Cresc. Rápido	28%	17%	-3%	-10%	170
Covariância	=Média do Produto dos Desvio = $1/3 \times (-180 + 0 - 170) = -116,7$				

Fonte: Elaboração própria

9.1.2 Carteira equal wage (pesos iguais)

50% ativo A e 50% ativo B

1) O Retorno esperado do Portfólio P é dado por:

$$E(R_p) = 0,5 \times 11\% + 0,5 \times 7\% = 9\% \quad (30)$$

2) A variância dos retornos do Portfólio P é dada por:

$$V(R_p) = (0,5 \times 14,3)^2 + (0,5 \times 8,2)^2 + 2 \times 0,5 \times 0,5 \times (-116,7) = 9,5825$$

$$\text{Risco } (\sigma_p) = \sqrt{9,5825} = 3,0955 \% \quad (31)$$

IMPORTANTE: Observe que o Risco da carteira é menor do que os riscos individuais.

9.1.3 Carteira variância mínima

$$w_A^* = \frac{14,3^2 - (-116,7)}{14,3^2 + 8,2^2 - 2 \times (-116,7)} = 0,3637 \text{ ou } 36,37\%$$

$$w_B^* = 1 - w_A^* = 0,6363 \text{ ou } 63,63\% \quad (32)$$

36,37% ativo A e 63,63% ativo B

1) O Retorno esperado do Portfólio P é dado por:

$$E(R_p) = 36,37\% \times 11\% + 63,63\% \times 7\% = 8,4548 \% \quad (33)$$

2) A variância dos retornos do Portfólio P é dada por:

$$\begin{aligned} V(R_p) &= (0,3637 \times 14,3)^2 + (0,6363 \times 8,2)^2 + \\ &2 \times 0,3637 \times 0,6363 \times (-116,7) = 0,2594 \\ \text{Risco } (\sigma_p) &= \sqrt{0,2594} = 0,5093\% \end{aligned} \quad (34)$$

Nesta seção, iniciaremos a aplicação prática dessa teoria no R, utilizando séries históricas de preços de ações negociadas na B3. Primeiramente, serão importados os preços de fechamento de seis ativos, PETR4, BBAS3, LAME4, GOAU4, CMIG3 e NATU3, diretamente do Yahoo Finance, por meio do pacote quantmod. Como os calendários de negociação entre Brasil e Estados Unidos podem divergir (devido a feriados locais), será necessário utilizar a função `na.omit()` para remover eventuais lacunas nas séries.

Em seguida, procederemos à transformação dos preços em retornos logarítmicos, uma medida amplamente utilizada em finanças pela sua simetria temporal e pela possibilidade de agregação ao longo do tempo. A partir dos retornos, será possível calcular a matriz de correlação, ferramenta essencial para avaliar o grau de associação linear entre os ativos. Essa matriz é um insumo fundamental da Teoria da Carteira, pois evidencia como a diversificação pode reduzir o risco total de uma carteira, especialmente quando os ativos apresentam correlação imperfeita.

```
#Habilitar o pacote quantmod
install.packages("quantmod")
library(quantmod)
getSymbols(c("PETR4.SA","BBAS3.SA","LAME4.SA","GOAU4.SA","CMIG3.SA","NATU3.SA"),
           src="yahoo",from=as.Date("2018-01-01"),to=as.Date("2018-11-19"))
head(PETR4.SA)
head(BBAS3.SA) #Visualizar se carregou corretamente as séries
head(LAME4.SA)
head(GOAU4.SA)
head(CMIG3.SA)
head(NATU3.SA)
```


Extrair o preço de fechamento das séries, sem buraco (na.omit()). Os buracos referem-se a dias que o yahoo finance considera útil, por seguir o calendário americano, porém é feriado no Brasil

```
petr4<-na.omit(PETR4.SA$PETR4.SA.Close)
petr4
bbas3<-na.omit(BBAS3.SA$BBAS3.SA.Close)
bbas3
lame4<-na.omit(LAME4.SA$LAME4.SA.Close)
lame4
goau4<-na.omit(GOAU4.SA$GOAU4.SA.Close)
goau4
cmig3<-na.omit(CMIG3.SA$CMIG3.SA.Close)
cmig3
natu3<-na.omit(NATU3.SA$NATU3.SA.Close)
natu3
plot(cbind(petr4,bbas3,lame4,goau4,cmig3,natu3)) #Gráfico das Séries
```

A seguir, extrair o retorno das séries

```
ra<-diff(log(petr4))
rb<-diff(log(natu3))
rc<-diff(log(lame4))
rd<-diff(log(goau4))
re<-diff(log(cmig3))
rf<-diff(log(bbas3))
```

A matriz de correlação será construída da seguinte forma:

```
cbind(ra,rb,rc,rd,re,rf)[-1,] #Excluir a primeira linha, pois ficou NA na
construção do retorno
cor(cbind(ra,rb,rc,rd,re,rf)[-1,])
```

	PETR4.SA.Close	NATU3.SA.Close	LAME4.SA.Close	GOAU4.SA.Close	CMIG3.SA.Close
PETR4.SA.Close	1.0000000	0.2111895	0.4169367	0.4621079	0.4699629
NATU3.SA.Close	0.2111895	1.0000000	0.4237214	0.3001567	0.2724284
LAME4.SA.Close	0.4169367	0.4237214	1.0000000	0.3795038	

0.4933360

GOAU4.SA.Close	0.4621079	0.3001567	0.3795038	1.0000000
-----------------------	------------------	------------------	------------------	------------------

0.2561209

CMIG3.SA.Close	0.4699629	0.2724284	0.4933360	0.2561209
-----------------------	------------------	------------------	------------------	------------------

1.0000000

BBAS3.SA.Close	0.6185652	0.3775059	0.6501852	0.4688377
-----------------------	------------------	------------------	------------------	------------------

0.6030805

BBAS3.SA.Close

PETR4.SA.Close 0.6185652

NATU3.SA.Close 0.3775059

LAME4.SA.Close 0.6501852

GOAU4.SA.Close 0.4688377

CMIG3.SA.Close 0.6030805

BBAS3.SA.Close 1.0000000

Observe que a menor correlação entre dois ativos é entre os ativos a e b, $cor=0.2111895$. Quanto mais próximo de -1, maior será os ganhos da diversificação de uma carteira de ativos, pois haverá um efeito compensação nas oscilações opostas de preço, o que minimizará o risco da carteira, medido pelo desvio padrão.

A Teoria da Carteira eficiente de ativos foi desenvolvida por Harry Markowitz, que foi merecedor do prêmio Nobel de economia em 1990 com seu artigo produzido em 1952. A ideia central consiste na diversificação, impactando na redução da variabilidade de retornos (risco) da carteira (Bodie, 2000).

O Modelo de Markowitz permite que se calcule o risco de uma carteira de investimentos. Permite construir carteiras de investimento em que o risco da carteira é inferior ao risco do ativo de menor risco da carteira. Isto é, imagine uma carteira com PETR4 (risco de 3% ao dia) e NATU3 (risco de 4% ao dia), seria possível que o risco da carteira diversificada com estes dois ativos fosse inferior ao ativo de menor risco, ou seja, menor que 3% ao dia.

O conjunto de possibilidade de investimento representa um gráfico RISCO-RETORNO de todas as possíveis carteiras de investimento formada por dois ativos de risco, para tanto, precisamos calcular os retornos e os desvios-padrão de todas as combinações de carteiras.

A taxa esperada de retorno sobre a carteira é a média ponderada dos retornos esperados:

$$E[r_p] = w_a E[r_a] + w_b E[r_b] \quad (35)$$

A variância da taxa de retorno da carteira de dois ativos de risco:

$$\sigma_p^2 = (w_a \sigma_a)^2 + (w_b \sigma_b)^2 + 2(w_a \sigma_a)(w_b \sigma_b) \rho_{ab} \quad (36)$$

Onde $w_a + w_b = 1$ são pesos atribuídos as carteiras e, σ_a , σ_b e ρ_{ab} representam o desvio padrão de r_a , desvio padrão de r_b e coeficiente de correlação entre r_a e r_b .

Médias individuais:

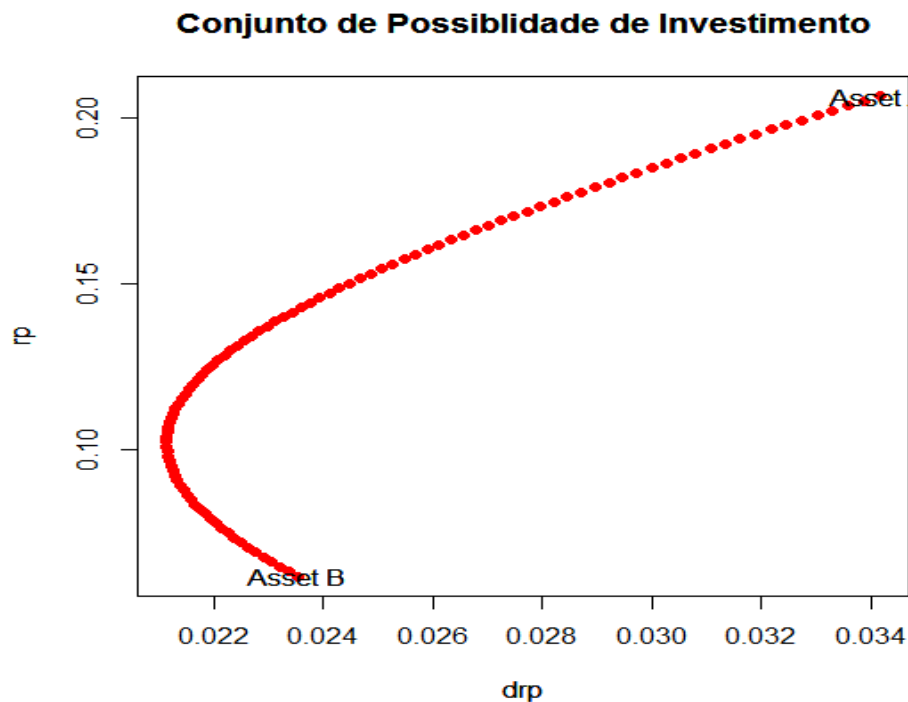
```
mra=mean(ra[-1,])*100 #Lembre-se de excluir a primeira linha, está NA
mrb=mean(rb[-1,])*100
mra
mrb
#Desvios-padrão individuais
sig.a<-sd(ra[-1,])
sig.b<-sd(rb[-1,])
sig.a
sig.b
cov(ra[-1,],rb[-1,])
pho<-cov(ra[-1,],rb[-1,])/(sd(ra[-1,])*sd(rb[-1,]))
pho
#Construindo carteiras
a<-0:100
b<-100:0
wa<-a/100
wa
wb<-1-wa
wb
wa+wb==1
```

Média de retorno e desvio padrão (risco) de todas as combinações de

carteiras

```
rp=wa*mra+wb*mrB
rp
vrp=(wa*sig.a)^2+(wb*sig.b)^2+2*wa*wb*sig.a*sig.b*pho
drp=sqrt(vrp)
drp
plot(drp,rp, type="b", pch=16, col=2, main="Conjunto de Possibilidade de
Investimento")
text(x=sig.a, y=mra, "Asset A")
text(x=sig.b, y=mrB, "Asset B")
```

Figura 67. Conjunto de possibilidade de investimento

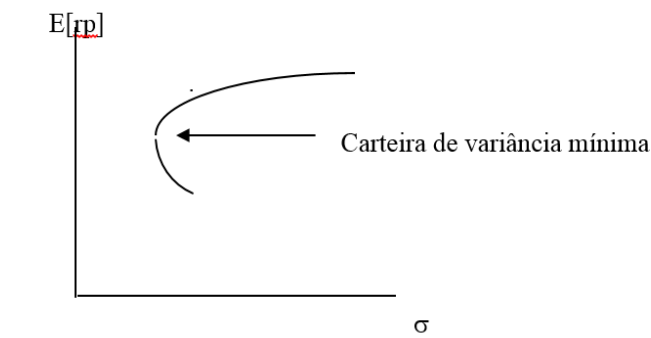


Fonte: Elaboração própria

9.2 CARTEIRA DE VARIÂNCIA MÍNIMA E CARTEIRA EFICIENTE

Ao calcular todas as combinações de w_a e w_b encontraremos um conjunto de carteiras com proporções diferentes de do ativo A e do ativo B, cada uma com uma variância e um retorno diferente. As combinações de todas estas variâncias e os retornos irá formar o conjunto de possibilidade de investimento, dado abaixo:

Figura 68. Carteira de Variância Mínima



Fonte: Elaboração própria

O investidor sempre escolherá carteiras com maior retorno e menor risco, assim, a carteira de variância mínima é preferível às carteiras que estão ao seu sudeste, desta forma, basta saber qual a carteira que o investidor escolherá dentre as carteiras restantes.

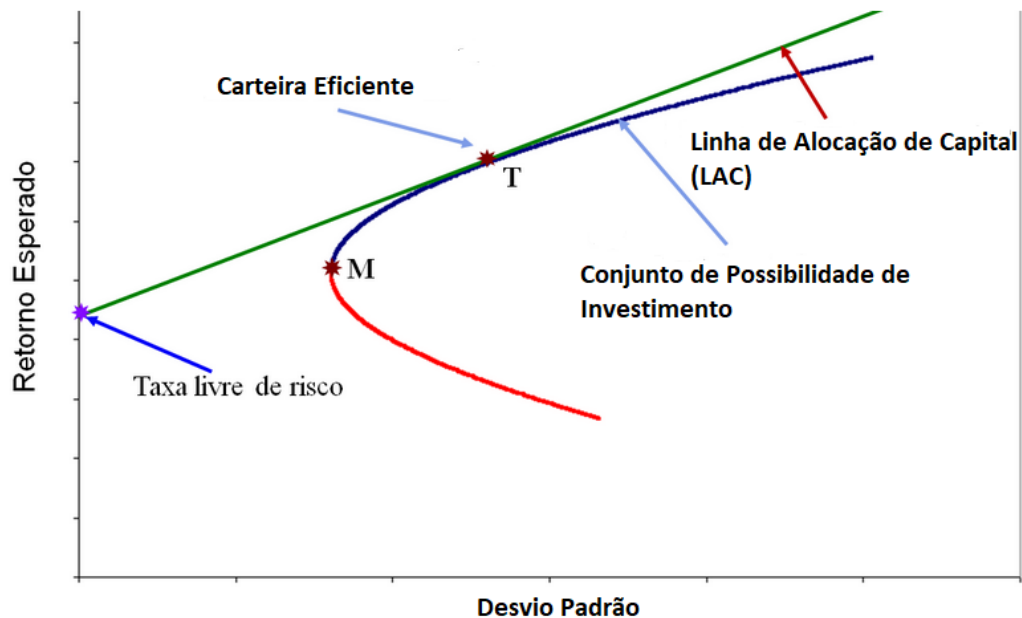
Note que até agora discutimos apenas sobre os ativos de risco, entretanto, um investidor pode investir apenas um percentual de sua riqueza nos ativos de risco, e o restante aplicar em um ativo livre de risco, com uma taxa de retorno r_f . Desta forma, surge a linha de alocação de capital (LAC), que é a reta de combinações entre ativos livres de risco e a carteira de risco (com dois ativos de risco). Veja o gráfico abaixo:

9.2.1 Dois ativos de risco e um ativo livre de risco

- Considere dois Ativos de Risco (A e B) e um ativo livre de risco (r_f)
- O Prêmio de risco é a diferença entre o retorno esperado de uma carteira e o ativo livre de risco: $E(R_C) - r_f$
- O Investidor escolhe entre um ativo livre de risco, com proporção y , e uma carteira arriscada (dentro do conjunto de possibilidade de investimento), com proporção $1-y$. Essas possibilidades são descritas pela Linha de Alocação de Capital (LAC).
- O Retorno Total da Carteira, com um ativo livre de risco e uma carteira arriscada P, será $E(R_C) = yr_f + (1 - y)E(R_P)$
- A inclinação na LAC é dada pelo coeficiente de recompensa $(E(R_P) - r_f)/\sigma_P$, que mede o prêmio de risco por risco assumido.

- A carteira eficiente será, dentre todas as carteiras possíveis dentro do conjunto de possibilidade de investimento, aquela que possui o maior coeficiente de recompensa. Ou seja, com maior Índice de Sharpe.

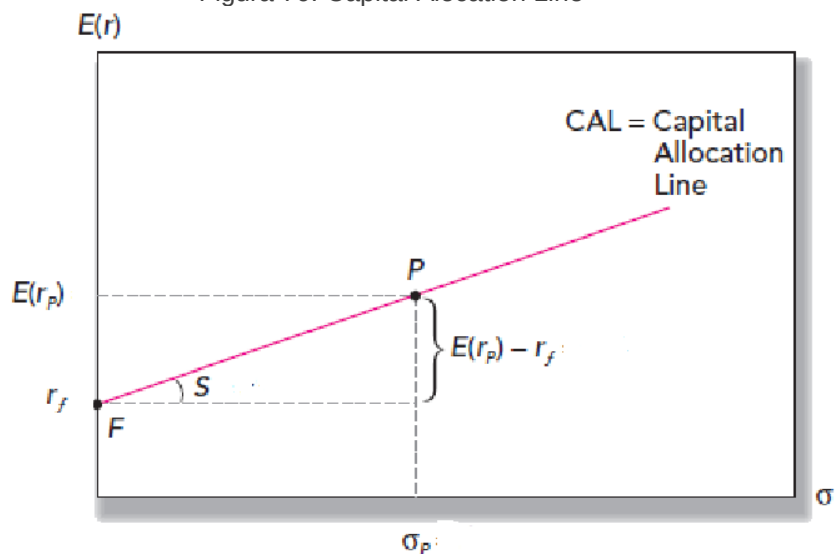
Figura 69. Retorno Esperado



Fonte: Elaboração própria

- A inclinação da Linha de Alocação de Capital (LAC), chamada de coeficiente de recompensa S , é o coeficiente angular da reta, ou seja, cateto oposto dividido por cateto adjacente.

Figura 70. Capital Allocation Line



Fonte: Elaboração própria

Se o investidor resolver colocar toda sua riqueza em um ativo livre de risco ele terá um retorno r_f que representa o intercepto da LAC. Por outro lado, se ele quiser somente investir em ativo arriscado ele investirá na carteira de variância mínima no exemplo do gráfico acima.

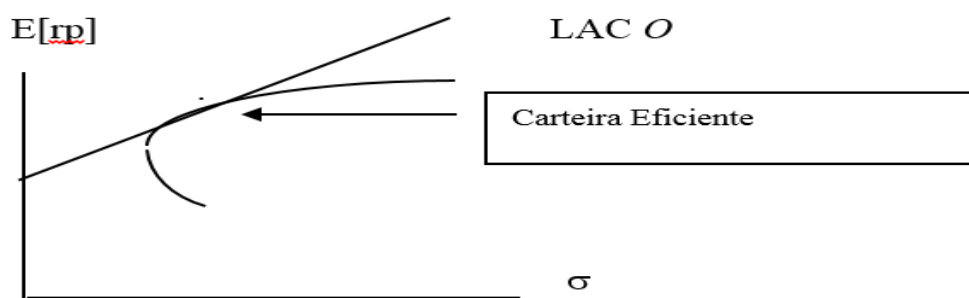
A reta entre r_f e a carteira de variância mínima é as combinações de investimento entre uma carteira de risco e livre de risco, essa combinação formará a carteira completa que terá um retorno dado abaixo:

$$E[r_c] = yE[r_p] + (1-y)E[r_f] \quad (37)$$

Note que as combinações possuem uma relação linear, esta é a LAC. Se chamarmos y da proporção investida na carteira de risco, temos que o investidor estará no ponto, digamos A, da LAC que tem um retorno $E[r_a]$ e um risco σ_a , o prêmio de risco desta suposta carteira A é dada por $E[r_a] - r_f$.

Entretanto, mostramos acima que o investidor está optando seu investimento entre um ativo livre de risco e a carteira de risco de variância mínima, todavia, será que esta é a decisão correta? Não. O investidor deve buscar a LAC mais inclinada dentro do seu conjunto de possibilidade de investimento, pois a inclinação da LAC mede o quociente de recompensa pela variabilidade, ou seja, dado um pequeno aumento do risco, o quanto aumenta de retorno, desta forma, quanto mais inclinada a LAC melhor, pois uma pequena variação daria uma maior recompensa. A tangencia da LAC de maior inclinação com o conjunto de possibilidade de investimento será a carteira eficiente, a inclinação da LAC é dada por: $(E[r_p] - r_f)/SD$

Figura 71. LAC O



Fonte: Elaboração própria

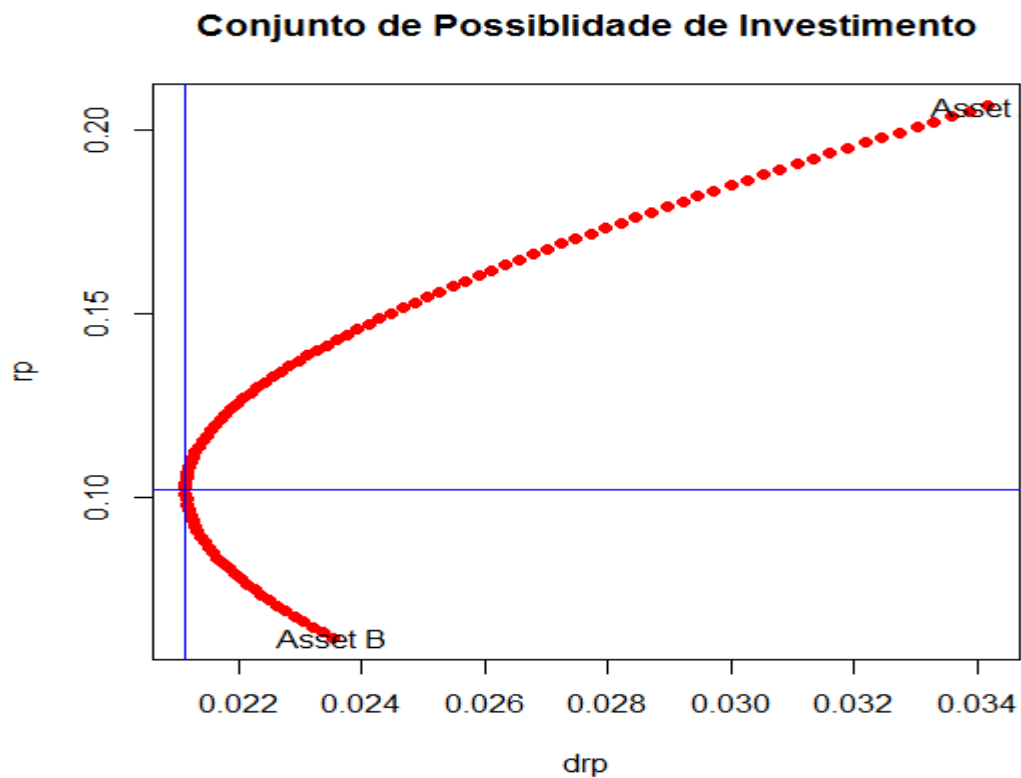
Chamaremos esta carteira eficiente de carteira O , ela terá uma combinação de pesos w_d e w_b (ótimos) entre os ativos de risco. A decisão do investidor será, o quanto investir nesta carteira e quanto investir em ativos livres de risco? Esta decisão estará sobre a LAC O, e somente a aversão ao risco do investidor poderá determinar com precisão onde estará a decisão de investimento. Esta decisão de investimento é a carteira completa, que é a combinação entre ativos livres de risco e a carteira ótima. A teoria calcula a carteira eficiente, mas, o quanto investir nela dependerá da aversão ao risco do investidor, ou seja, cada investidor montará sua carteira completa de acordo com sua aversão ao risco.

```
rf<-0.1
coef<-(rp-rf)/drp
coef
cbind(wa,wb,rp,drp,coef)
```

Carteira de Variância Mínima será:

```
min(drp)
wao=(var(rb[-1,])-cov(ra[-1,],rb[-1,]))/(var(ra[-1,])+var(rb[-1,])-2*cov(ra[-1,],rb[-1,]))
wao
wbo=1-wao
wbo
rcvm=wao*mra+wbo*mrB
rcvm
abline(h=rcvm, v=min(drp), col=4)
```


Figura 72. Conjunto de possibilidades de Investimentos



Fonte: Elaboração própria

A carteira eficiente será:

```

xta=((mra-rf)*var(rb)-(mrb-rf)*cov(ra,rb))/((mra-rf)*var(rb)+(mrb-rf)*var(ra)-
(mra+mrb-2*rf)*cov(ra,rb))
xtb=1-xta
xta
xtb
rcef=xta*mra+xtb*mrb
rcef
vcef=((xta*sig.a)^2+(xtb*sig.b)^2+2*xta*xtb*sig.a*sig.b*pho)^0.5
vcef
abline(h=rcef, v=vcef, col=3)

```

Exercício: Qual o retorno da cateira equal wage? $w_a=0.5$ e $w_b=0.5$

```

RetornoC<-function(x,y,wa,wb){
ra<-mean(x)
rb<-mean(y)
matrix(c(wa,wb),1,2)%*%matrix(c(ra,rb),2,1)

```

```
}
RetornoC(ra[-1,],rb[-1,],0.5,0.5)
```

9.3 ANALISE DO PORTFÓLIO PARA N ATIVOS

9.3.1 Modelo matricial: portfólio com N Ativos

A análise Risco x Retorno do modelo de Markowitz, para dois ativos arriscados A e B, sugere a construção de variâncias e retornos esperados de portfólios diversificados com proporções w_A e w_B , respectivamente, onde $w_A + w_B = 1$, através das seguintes fórmulas:

Considere dois Ativos de risco, A e B:

1. O Retorno esperado do Portfólio P é dado por:

$$E(R_p) = w_A E(R_A) + w_B E(R_B), \quad \text{onde } w_A + w_B = 1 \quad (38)$$

2. A variância dos retornos do Portfólio P é dada por:

$$V(R_p) = \sigma_p^2 = (w_A \sigma_A)^2 + (w_B \sigma_B)^2 + 2w_A w_B \text{Cov}(A, B) \quad (39)$$

Onde:

R_p = Retorno do Portfólio P composto por ativos A e B.

$E(R_A)$ e $E(R_B)$ = Retornos esperados dos ativos A e B, respectivamente. Resultado da média ponderada entre os cenários (recessão, normal e crescimento rápido) para cada Ativo individual.

w_A e w_B = pesos dos ativos A e B na carteira P.

$\text{Cov}(A, B) = \sigma_A \sigma_B \rho_{AB}$

9.3.2 Para N ativos

O Retorno esperado do Portfólio P é dado por:

$$E(R_p) = w_A E(R_A) + w_B E(R_B) + \dots + w_N E(R_N), \quad \text{onde} \quad w_A + w_B + \dots + w_N = 1 \quad (40)$$

A equação pode ser descrita matricialmente como:

$$E(R_p) = [w_A \quad w_B \quad \dots \quad w_N] \begin{bmatrix} E(R_A) \\ E(R_B) \\ \vdots \\ E(R_N) \end{bmatrix} \quad (41)$$

A notação matricial da equação (2) pode ser descrita como $E(R_p) = wR$

Onde w é uma matriz $1 \times N$ e R uma matriz $N \times 1$.

Note que a multiplicação das matrizes $1 \times N \times N \times 1$ da equação (2) se transformam na equação (1).

A variância dos retornos do Portfólio P é dada por:

$$V(R_p) = \sigma_p^2 = (w_A \sigma_A)^2 + (w_B \sigma_B)^2 + 2w_A w_B \text{Cov}(A, B) \quad (42)$$

A equação pode ser descrita matricialmente como:

$$V(R_p) = [w_A \quad w_B] \begin{bmatrix} \sigma_A^2 & \text{Cov}(A, B) \\ \text{Cov}(B, A) & \sigma_B^2 \end{bmatrix} \begin{bmatrix} w_A \\ w_B \end{bmatrix} \quad (43)$$

Note que a multiplicação das matrizes $1 \times 2N \times 2 \times 2 \times 2 \times 1$ da equação (4) se transformam na equação (3). Generalizando para N ativos:

$$V(R_p) = [w_A \quad w_B \quad \dots \quad w_N] \begin{bmatrix} \sigma_A^2 & \text{Cov}(A, B) & \dots & \text{Cov}(A, N) \\ \text{Cov}(B, A) & \sigma_B^2 & \dots & \text{Cov}(B, N) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(N, A) & \text{Cov}(N, B) & \dots & \sigma_N^2 \end{bmatrix} \begin{bmatrix} w_A \\ w_B \\ \vdots \\ w_N \end{bmatrix} \quad (44)$$

A notação matricial da equação (5) pode ser descrita como $E(R_p) = w \Sigma w'$

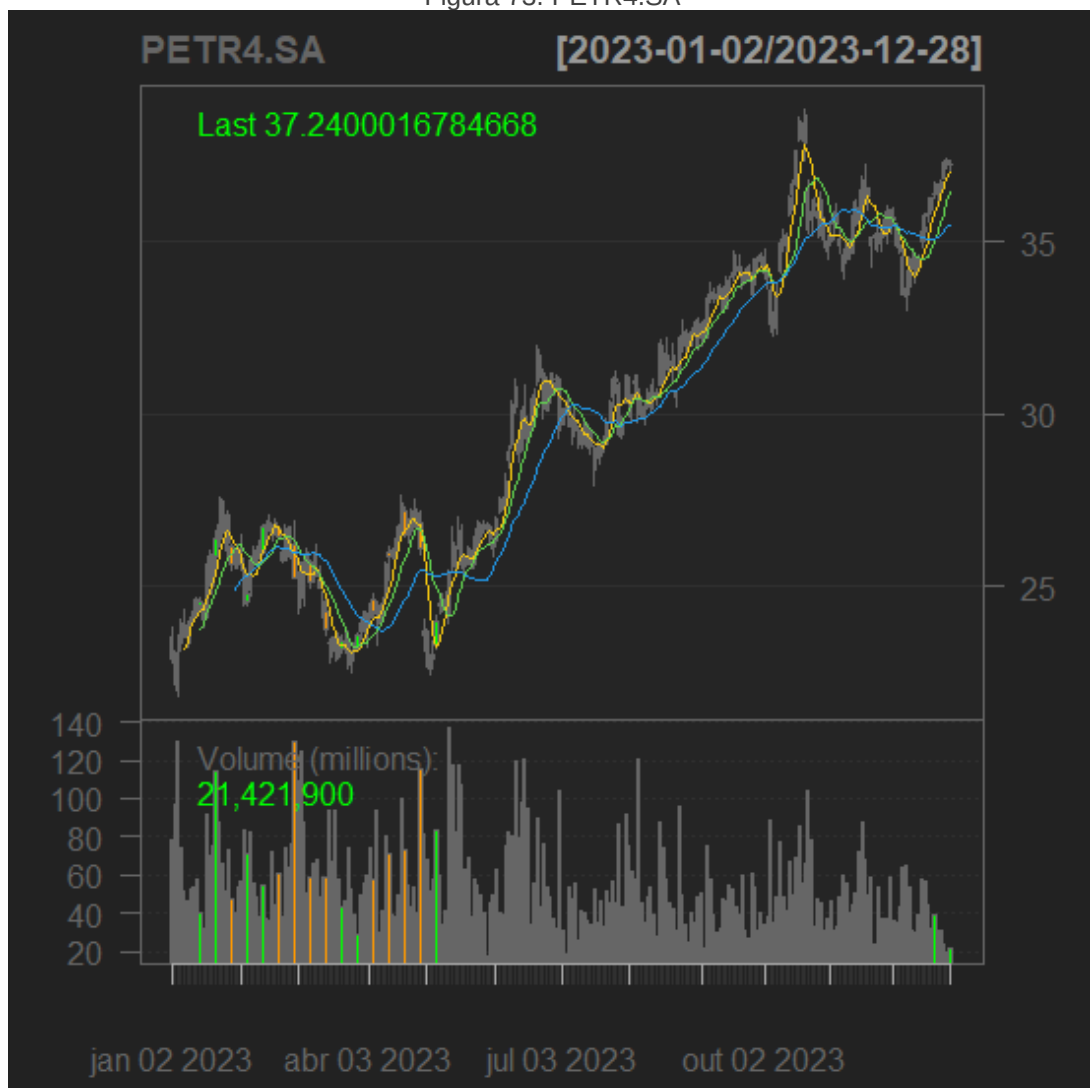
Onde w é uma matriz $1 \times N$, Σ é a matriz de variância covariância $N \times N$ e w' é a transposta de w , com dimensão $N \times 1$.

```

#Habilitar o pacote quantmod
install.packages("quantmod")
library(quantmod)
getSymbols(c("PETR4.SA","BBAS3.SA","VALE3.SA","ABEV3.SA","^BVSP"), src="yahoo",from=as.Date("2023-01-01"),to=as.Date("2023-12-31"))
head(PETR4.SA)
head(BBAS3.SA) #Visualizar se carregou corretamente as series
head(VALE3.SA)
head(ABEV3.SA)
head(BVSP)
chartSeries(PETR4.SA)
addSMA(n = c(5,10,21), col=c(7,3,4))

```

Figura 73. PETR4.SA



Fonte: Elaboração própria

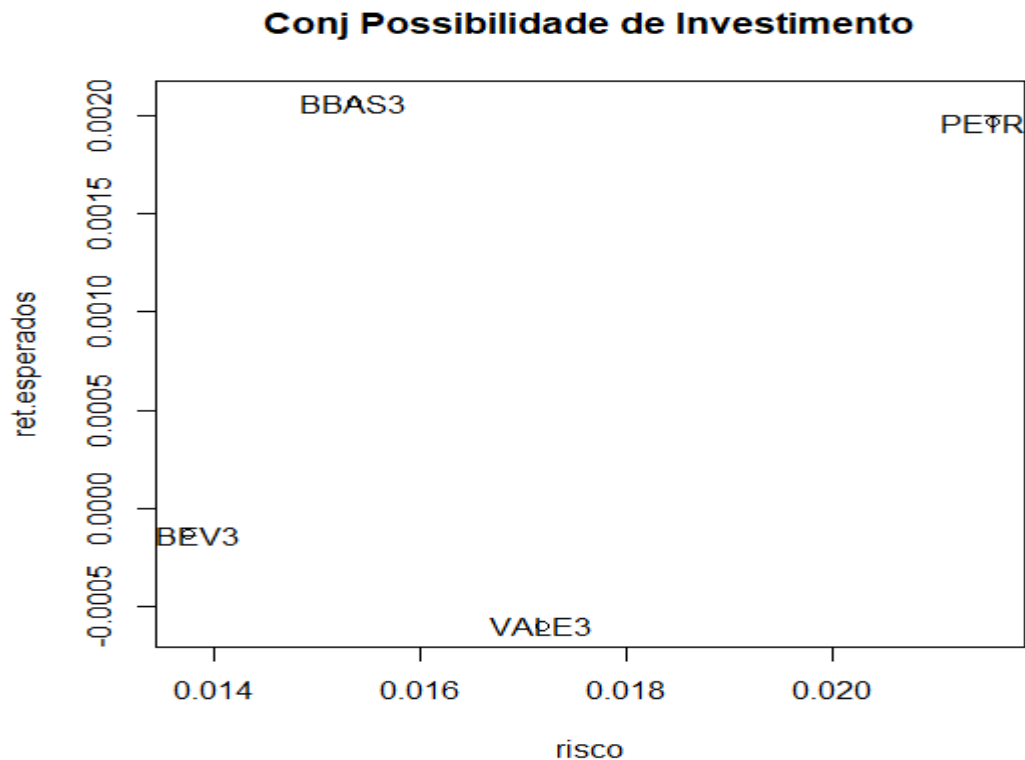
```

# Criar os retornos
rpetr4<-diff(log(PETR4.SA$PETR4.SA.Close))
rbbas3<-diff(log(BBAS3.SA$BBAS3.SA.Close))
rvale3<-diff(log(VALE3.SA$VALE3.SA.Close))
rabev3<-diff(log(ABEV3.SA$ABEV3.SA.Close))
ribov<-diff(log(BVSP$BVSP.Close))
base1<-na.omit(cbind(rpetr4,rbbas3,rvale3,rabev3,ribov))
colnames(base1)<-c("PETR4","BBAS3","VALE3","ABEV3","IBOV")
base1
length(base1[,1])
##INSTALAR E CARREGAR OS PACOTES
install.packages("fPortfolio")
library(fPortfolio)
install.packages("timeSeries")
library(timeSeries)
base2<- base1[,1:4] #exclui ibov
##conversao no tipo de dados "timeSeries"
base2<-as.timeSeries(base2)
#####
##ESTATISTICAS DOS DADOS
##RETORNOS ESPERADOS
ret.esperados = colMeans(base2)
ret.esperados
#MATRIZ DE COVARIANCIAS
mat.cov = cov(base2)
mat.cov
r1<-sqrt(mat.cov[1,1])
r2<-sqrt(mat.cov[2,2])
r3<-sqrt(mat.cov[3,3])
r4<-sqrt(mat.cov[4,4])
risco<-c(r1,r2,r3,r4)
risco
plot(risco,ret.esperados,      main="Conj      Possibilidade      de
Investimento")

```

```
text(risco,ret.esperados, c("PETR4","BBAS3","VALE3","ABEV3"))
#####
```

Figura 74. Conj Possibilidades de investimento



Fonte: Elaboração própria

```
##OTIMIZACAO
##pesos dos ativos
##retorna o portfolio com a maior relacao retorno/risco na fronteira
eficiente
p1 = tangencyPortfolio(base2, spec = portfolioSpec(), constraints =
"LongOnly")
p1
##retorna o portfolio com o risco minimo na fronteira eficiente
p2 = minvariancePortfolio(base2, spec = portfolioSpec(), constraints
= "LongOnly")
p2
## calculos para obtencao da fronteira eficiente
Frontier = portfolioFrontier(base2)
```

```

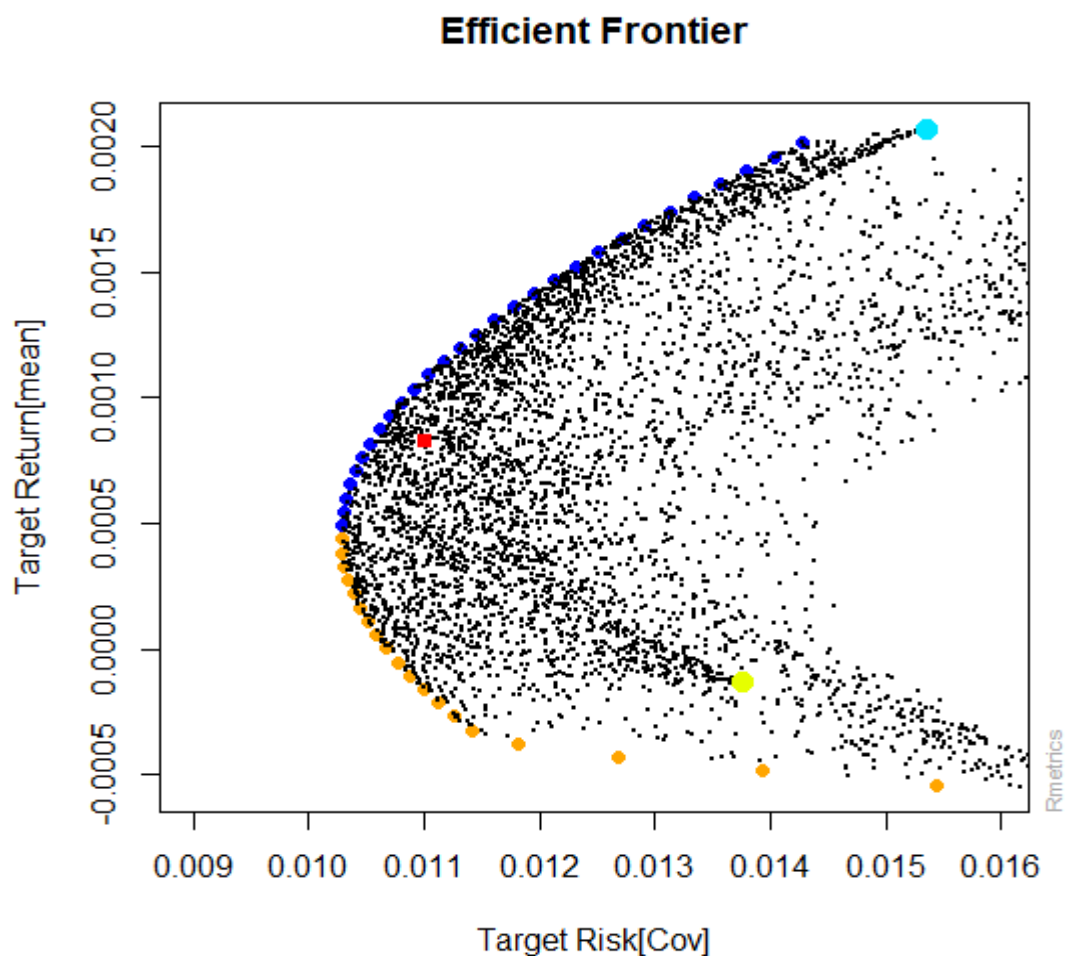
## Plotagem da fronteira eficiente no grafico
frontierPlot(Frontier, col = c("blue", "orange"), pch = 19)

## Adicinando informacoes ao grafico, pontos associados a
possiveis carteiras
p3 = monteCarloPoints(Frontier, mcSteps = 5000, cex = 0.25, pch =
19)

p3
## mostrando o local da carteira com proporcoes iguais em cada
ativo
equalWeightsPoints(Frontier, pch = 15, col = "red")
## mostrando os pontos relativos a cada ativo individualmente
singleAssetPoints(Frontier, pch = 19, cex = 1.5, col = topo.colors(6))

```

Figura 75. Efficient Frontier



Fonte: Elaboração própria

```

#RETORNOS ACUMULADOS
#Retorno acumulado carteira eficiente
peso_efic<-matrix(getPortfolio(p1)$weights,4,1)
dim(peso_efic)
ret_ef<-base2%*%peso_efic
fator_carteira_ef<-ret_ef+1
retorno_acumulado_ef<-numeric(247)
retorno_acumulado_ef[1]<-1
for(i in 2:247){
  retorno_acumulado_ef[i]<-
fator_carteira_ef[i]*retorno_acumulado_ef[i-1]}
  retorno_acumulado_ef-1
  g1<-plot(ts(retorno_acumulado_ef-1),col=1)
#Retorno acumulado carteira variancia minima
peso_cvm<-matrix(getPortfolio(p2)$weights,4,1)
dim(peso_cvm)
ret_cvm<-base2%*%peso_cvm
fator_carteira_cvm<-ret_cvm+1
retorno_acumulado_cvm<-numeric(247)
retorno_acumulado_cvm[1]<-1
for(i in 2:247){
  retorno_acumulado_cvm[i]<-
fator_carteira_cvm[i]*retorno_acumulado_cvm[i-1]}
  retorno_acumulado_cvm-1
  g2<-plot(ts(retorno_acumulado_cvm-1),col=2)
#Retorno acumulado carteira equal wage
peso_ew<-matrix(rep(1,4)/4,4,1)
dim(peso_ew)
ret_ew<-base2%*%peso_ew
length(ret_ew)
fator_carteira_ew<-ret_ew+1
retorno_acumulado_ew<-numeric(247)
retorno_acumulado_ew[1]<-1
for(i in 2:247){

```

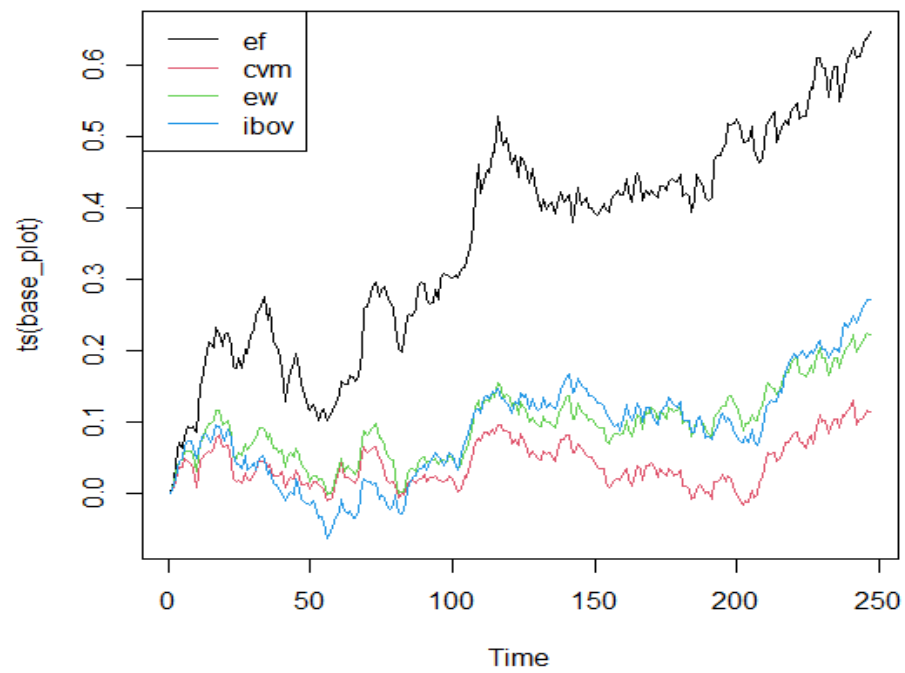


```

    retorno_acumulado_ew[i]<-
fator_carteira_ew[i]*retorno_acumulado_ew[i-1]}
    retorno_acumulado_ew-1
    g3<-plot(ts(retorno_acumulado_ew-1),col=3)
    #Retorno acumulado IBOV
    length((diff(log(BVSP$BVSP.Close))))
    ribov<-na.omit(matrix(diff(log(BVSP$BVSP.Close)),248,1))
    length(ribov)
    fator_carteira_ibov<-ribov+1
    retorno_acumulado_ibov<-numeric(247)
    retorno_acumulado_ibov[1]<-1
    for(i in 2:247){
    retorno_acumulado_ibov[i]<-
fator_carteira_ibov[i]*retorno_acumulado_ibov[i-1]}
    retorno_acumulado_ibov-1
    g4<-plot(ts(retorno_acumulado_ibov-1),col=4)
    base_plot<-cbind((retorno_acumulado_ef-
1),(retorno_acumulado_cvm-1),(retorno_acumulado_ew-
1),(retorno_acumulado_ibov-1))
    colnames(base_plot)<-c("ef","cvm","ew","ibov")
    plot(ts(base_plot),plot.type="single", col=1:4)
    legend("topleft", legend = colnames(base_plot), col = 1:4, lty = 1)

```

Figura 76. TS(base_plot)



Fonte: Elaboração própria



CAPÍTULO 10

MODELO DE PRECIFICAÇÃO DE ATIVOS DE CAPITAL - CAPM

O conceito de risco e retorno, eixo central da Moderna Teoria de Finanças, encontra sua fundamentação no Capital Asset Pricing Model (CAPM), desenvolvido de forma independente por Sharpe (1966). O modelo estabelece a relação entre o risco de um ativo e seu retorno esperado, a partir de um conjunto de hipóteses que caracterizam um mercado perfeito:

- a) os investidores preocupam-se apenas com o retorno esperado e a variância (ou desvio padrão) dos retornos;
- b) preferem maior retorno e menor risco;
- c) desejam formar carteiras eficientes, ou seja, aquelas que oferecem o máximo retorno esperado para um dado nível de risco ou o mínimo risco para um dado retorno esperado;
- d) existe um ativo livre de risco, permitindo que investidores possam emprestar ou tomar emprestado a uma taxa equivalente ao retorno desse ativo;
- e) nenhum investidor individual consegue afetar os preços de mercado, e todos planejam seus investimentos em horizontes temporais idênticos.

Essas hipóteses permitem a aceitação da relação risco-retorno como princípio fundamental e dão suporte ao uso do CAPM para determinar a taxa de retorno teórica adequada de um ativo em comparação a uma carteira de mercado perfeitamente diversificada. O modelo considera a sensibilidade do ativo ao risco não diversificável — também chamado risco sistemático ou risco de mercado — representado pelo coeficiente beta (β).

A equação do CAPM é expressa como:

$$E(R_i) = r_f + \beta[E(R_M) - r_f] \quad (45)$$

onde:

- $E(R_i)$: retorno esperado do ativo iii;

- rf: taxa livre de risco;
- $E(R_m)$: retorno esperado do mercado;
- β : medida da sensibilidade do ativo em relação ao mercado.

No CAPM, β representa o impacto do risco de mercado sobre o retorno de um ativo individual. Sua estimação é feita por meio de uma regressão linear simples, em que:

- variável dependente: retorno do ativo (R_i);
- variável independente: retorno do mercado (R_m).

Assim, o coeficiente angular da regressão corresponde ao beta (β), indicando a variação esperada no retorno do ativo para cada unidade de variação no retorno do mercado.

Matematicamente:

$$\beta = \frac{Cov(R_i; R_m)}{Var(R_m)} \quad (46)$$

Um $\beta > 1$ indica que o ativo é mais volátil que o mercado (agressivo), enquanto um $\beta < 1$ sugere menor volatilidade em relação ao mercado (defensivo). Já $\beta = 1$ representa um ativo com risco igual ao risco médio do mercado.

Note que o Beta, por medir o impacto do mercado, ele é uma medida de risco sistêmico, que pode ser visualizado da seguinte forma:

$$R_i = \alpha + \beta R_m + e_i \quad (47)$$

$$VAR(R_i) = \beta^2 VAR(R_m) + VAR(e_i) \quad (48)$$

Note que a Variância do ativo é composta por um risco sistêmico, impactado pelo Beta, e um risco específico, mensurado pelos os termos de erro.

10.1 LINHA DE MERCADO DE TÍTULOS

Uma forma de apresentar a relação entre os retornos de um ativo e os retornos da carteira do mercado (ou do índice de mercado) pode ser sintetizada pela Linha de Mercado do Ativo (*Security Market Line* - *SML*). A equação da

Linha de Mercado do Ativo - SML - pode ser expressa da seguinte forma:

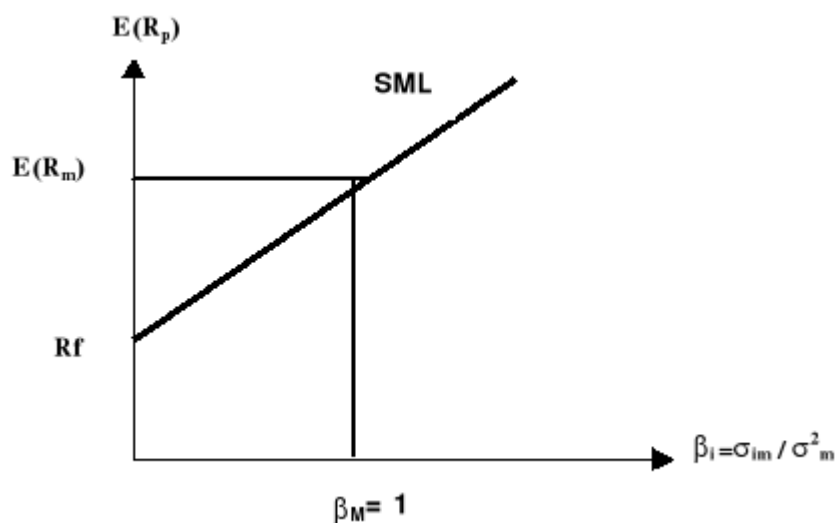
$$E(R_p) = R_f + [E(R_m) - R_f] \beta_p \quad (49)$$

Suas premissas básicas são as seguintes:

- a) a taxa de juros livre de risco é igual a R_f ;
- b) o prêmio de risco de mercado é igual a $E(R_m) - R_f$,ou seja, a diferença entre o valor esperado da rentabilidade da carteira de mercado $E(R_m)$ e a taxa de juros livre de risco R_f ;
- c) $E(R_p)$ retorno esperado da carteira p;
- d) β_p é o risco sistemático ou risco não diversificável da carteira p, com relação à carteira de mercado.

Em equilíbrio, todos os ativos devem cair ao longo de uma linha reta, conhecida como a reta de mercado de ativos. Portanto, um beta $\beta = 1,0$ é considerado *neutro*. À medida que o mercado como um todo sobe de x%, o ativo tende a subir x%. À medida que o mercado como um todo cai x%, o ativo tende também a cair x%. O ativo *agressivo* que tem um $\beta > 1,0$ significa que, quando o mercado estiver em alta de x%, o ativo terá o seu preço elevado em mais de x%. Um ativo com $\beta < 1,0$ é considerado um título *defensivo*, ou seja, há ativos cuja evolução dos índices de rentabilidade é mais lento que os do mercado.

Figura 77. Linha de mercado de títulos



Fonte: Elaboração própria

A linha de mercado de títulos reflete um preço justo para o ativo dado o seu nível de risco sistêmico (beta). Ativos acima da SML estão subprecificados, pois remuneram acima do preço justo, já os ativos abaixo da SML estão sobreprecificados, pois remuneram abaixo da remuneração justa dado aquele nível de risco.

Exercício

Suponha que os retornos de títulos sejam gerados pelo modelo CAPM, onde R_i é o retorno do título i , R_m o retorno do mercado e r_f a taxa livre de risco. Suponha que existam 5 títulos (A, B, C, D e E) caracterizados pelos seguintes dados:

Tabela 7. Exercício

Título	Beta	E(R)	$\sigma^2(e)$
A	0,85	10	5
B	1,0	12	1
C	1,25	14	10
D	1,10	14	12
E	0,92	11	3

Fonte: Elaboração própria

Se $\sigma^2 M = 4$, calcule as variâncias dos retornos dos títulos A, B, C, D e E. Quais as ações mais arriscadas? Quem tem maior risco sistêmico e maior risco específico?

Trace a Linha de Mercado de Títulos e comente sua dispersão, existe oportunidade na precificação? Considere a taxa livre de risco 5%.

$$VAR(R_i) = \beta^2 VAR(R_m) + VAR(e_i) \quad (50)$$

Ativo A:

$$VAR(R_i) = 0,85^2 \times 4 + 5 = 2,89 + 5 = 7,89 \quad (51)$$

Ativo B:

$$VAR(R_i) = 1^2 \times 4 + 5 = 4 + 1 = 5 \quad (52)$$

Ativo C:

$$VAR(R_i) = 1,25^2 \times 4 + 10 = 6,25 + 10 = 16,25 \quad (53)$$

Ativo D:

$$VAR(R_i) = 1,10^2 \times 4 + 12 = 4,84 + 12 = 16,84 \quad (54)$$

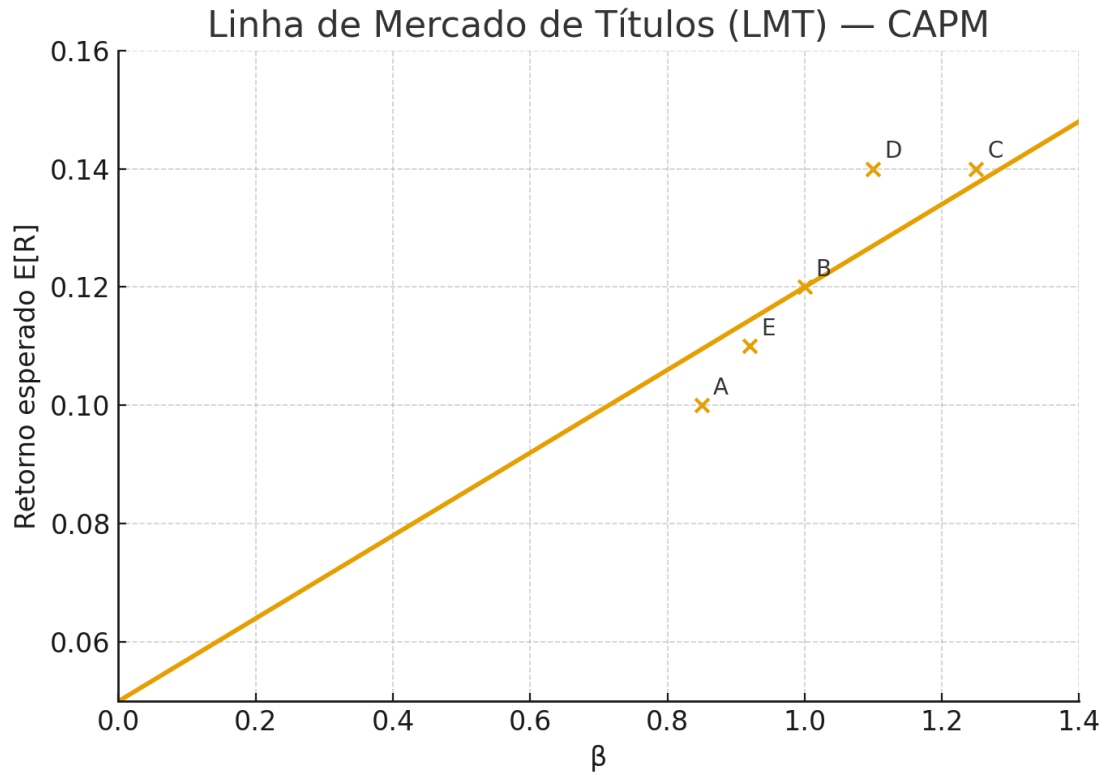
Ativo E:

$$VAR(R_i) = 0,92^2 \times 4 + 3 = 3,38 + 3 = 6,38 \quad (55)$$

Quem é mais arriscada?

- **Maior variância total:** D (16,8416) → mais arriscada no total.
- **Maior risco sistêmico ($\beta^2 \cdot \text{Var}(R_m)$):** C (6,256).
- **Maior risco específico ($\sigma^2(e)$):** D (12).

Figura 78. Linha de mercado de Título (LMT) - CAPM



Fonte: Elaboração própria

Retornos Justamente Precificados:

$$E(R_i) = r_f + \beta[E(R_m) - r_f] \quad (56)$$

Ativo A:

$$E(R_i) = 5\% + 0,85[12 - 5] = 5\% + 5,95\% = 10,95\% \quad (57)$$

(10% < 10,95%) → abaixo da LMT → **sobreprecificada**.

Ativo B:

-> Beta igual a 1, representa o mercado, justamente precificado sobre a LMT (12% = 12%) → na LMT → **justamente precificada**.

Ativo C:

$$E(R_i) = 5\% + 1,25[12 - 5] = 5\% + 8,75\% = 13,75\% \quad (58)$$

(14% > 13,75%) → acima da LMT → **subprecificada**.

Ativo D:

$$E(R_i) = 5\% + 1,10[12 - 5] = 5\% + 7,7\% = 12,7\% \quad (59)$$

(14% > 12,70%) → acima da LMT → **subprecificada**

Ativo E:

$$E(R_i) = 5\% + 0,92[12 - 5] = 5\% + 6,44\% = 11,44\% \quad (60)$$

(11% < 11,44%) → abaixo da LMT → **sobreprecificada**

Note que, o ativo A tem um retorno observado de 10%, e seu retorno justo

é de 10,95%, ou seja, remunera menos do que deveria, estando abaixo da LMT, logo sobreprecificado. O ativo B, com beta igual a 1, representa o mercado, estando sobre a LMT. O ativo C, com retorno observado de 14% e retorno justo de 13,75%, remunera mais do que justo, estando acima da LMT e, consequentemente subprecificado. Seguindo essa lógica, os ativos D e E estão subprecificados e sobreprecificados respectivamente.

A diferença entre o retorno observado e o retorno justo, calculado pelo CAPM, é chamado de alfa de Jensen. Graficamente, é a distância de cada ativo até a LMT, logo, ativos subprecificados possuem alfa de Jensen positivo, e quanto maior essa medida melhor, servindo de medida de desempenho.

10.2 ESTIMAÇÃO DOS BETAS INDIVIDUAIS

Agora vamos calcular o beta individuais, para um ativo, por trimestre, uma vez que o beta é interpretado como uma medida de risco. Inicialmente, um exemplo com o beta individual da PETRR, posteriormente dos portfólios. Imprescindível carregar os códigos da seção 9.3.

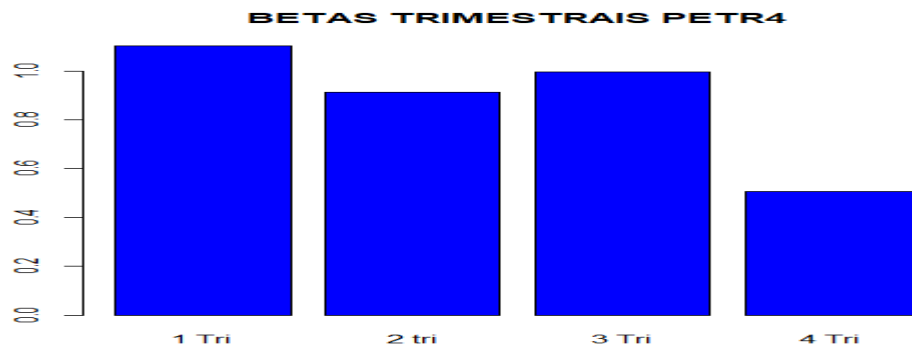
```
##### CAPM E PRECIFICAÇÃO INDIVIDUAL #####
library(quantmod)
getSymbols(c("PETR4.SA","^BVSP"), src="yahoo",from=as.Date("2023-
01-01"),to=as.Date("2023-12-31"))
#Retornos Petrobrás e IBOV
rpetr4<-diff(log(PETR4.SA$PETR4.SA.Close))
bov<-BVSP$BVSP.Close
rbov<-diff(log(BVSP$BVSP.Close))
install.packages("lubridate") #Pacote para habilitar funções de datas
library(lubridate)
month(PETR4.SA[,1])
quarter(PETR4.SA[,1])
trimestre<-quarter(PETR4.SA[,1])
#Regressão Período Total
reg_total<-lm(rpetr4~rbov)
summary(reg_total)
#Montar base por Trimestre
```

```

base3<-cbind(rpetr4,rbov,trimestre)
base3[which(base3[,3]==1),] #recorta a base 1T
# Separação dos dados por trimestre
rpetr4_1T <- base3[base3[,3] == 1, 1] # Retorno PETR4 - 1º trimestre
rpetr4_2T <- base3[base3[,3] == 2, 1] # Retorno PETR4 - 2º trimestre
rpetr4_3T <- base3[base3[,3] == 3, 1] # Retorno PETR4 - 3º trimestre
rpetr4_4T <- base3[base3[,3] == 4, 1] # Retorno PETR4 - 4º trimestre
ribov_1T <- base3[base3[,3] == 1, 2] # Retorno IBOV - 1º trimestre
ribov_2T <- base3[base3[,3] == 2, 2] # Retorno IBOV - 2º trimestre
ribov_3T <- base3[base3[,3] == 3, 2] # Retorno IBOV - 3º trimestre
ribov_4T <- base3[base3[,3] == 4, 2] # Retorno IBOV - 4º trimestre
# Regressões por trimestre (modelo de CAPM)
reg1T_petr4 <- lm(rpetr4_1T ~ ribov_1T)
summary(reg1T_petr4)
reg2T_petr4 <- lm(rpetr4_2T ~ ribov_2T)
summary(reg2T_petr4)
reg3T_petr4 <- lm(rpetr4_3T ~ ribov_3T)
summary(reg3T_petr4)
reg4T_petr4 <- lm(rpetr4_4T ~ ribov_4T)
summary(reg4T_petr4)
#Coeficientes Beta por Trimestre
beta1T_petr4<-coef(reg1T_petr4)[2]
beta2T_petr4<-coef(reg2T_petr4)[2]
beta3T_petr4<-coef(reg3T_petr4)[2]
beta4T_petr4<-coef(reg4T_petr4)[2]
#Base de Betas
betas_petr4<-c(beta1T_petr4,beta2T_petr4,beta3T_petr4,beta4T_petr4)
names(betas_petr4)<-c("1 Tri", "2 tri", "3 Tri", "4 Tri")
barplot(betas_petr4,col="blue", main="BETAS TRIMESTRAIS PETR4")

```

Figura 79. Betas Trimestrais PETR4



Fonte: Elaboração própria

A análise dos coeficientes β evidencia que a ação PETR4 apresentou comportamento agressivo apenas no primeiro trimestre, quando o valor do beta foi superior a 1, indicando maior sensibilidade às variações do mercado. A partir do segundo trimestre, observa-se um movimento de redução progressiva do risco sistemático, com valores de β inferiores a 1, o que caracteriza um perfil defensivo. No quarto trimestre, essa tendência se intensifica, com o beta atingindo patamar significativamente abaixo da unidade. Esse comportamento sugere que, ao longo do período, a influência dos fatores de mercado sobre o retorno da ação diminuiu, possivelmente em razão do aumento da relevância de fatores específicos da empresa e do setor.

```
#Retornos Justos (CAMP)
#CAPM = rf + beta*(E(Rm) - rf)
rf<- 0.125/252

CAPM_PETR4_1T <- rf + as.numeric(beta1T_petr4) *
(mean(na.omit(ribov_1T) - rf))
CAPM_PETR4_2T <- rf + as.numeric(beta2T_petr4) *
(mean(na.omit(ribov_2T)) - rf)
CAPM_PETR4_3T <- rf + as.numeric(beta3T_petr4) *
(mean(na.omit(ribov_3T)) - rf)
CAPM_PETR4_4T <- rf + as.numeric(beta4T_petr4) *
(mean(na.omit(ribov_4T)) - rf)

CAPM_PETR4_1T
CAPM_PETR4_2T
CAPM_PETR4_3T
```

```

CAPM_PETR4_4T
#Precificação - Se o retorno observado for maior que o retorno justo,
o ativo esta subprecificado, caso contrário sobreprecificado
mean_rpetr4_1T <- mean(na.omit(rpetr4_1T))
mean_rpetr4_2T <- mean(na.omit(rpetr4_2T))
mean_rpetr4_3T <- mean(na.omit(rpetr4_3T))
mean_rpetr4_4T <- mean(na.omit(rpetr4_4T))
CAPM_PETR4<-
c(CAPM_PETR4_1T,CAPM_PETR4_2T,CAPM_PETR4_3T,CAPM_PETR4_4
T)

Medias_Observadas_PETR4 <-
c(mean_rpetr4_1T,mean_rpetr4_2T,mean_rpetr4_3T,mean_rpetr4_4T)
Status_PETR4 = ifelse(c(mean_rpetr4_1T, mean_rpetr4_2T,
mean_rpetr4_3T, mean_rpetr4_4T) > c(CAPM_PETR4_1T,
CAPM_PETR4_2T, CAPM_PETR4_3T, CAPM_PETR4_4T),"Subprecificado",
"Sobreprecificado")
resultado_PETR4<-
data.frame(matrix(c(CAPM_PETR4,Medias_Observadas_PETR4,Status_PETR4),4,3))
colnames(resultado_PETR4) <- c("Retorno_CAPM",
"Retorno_Observado", "Status")
rownames(resultado_PETR4) <- paste0(1:4, "T")
resultado_PETR4
# Dados do gráfico
valores_PETR4 <- c(Medias_Observadas_PETR4[1],
CAPM_PETR4[1],
Medias_Observadas_PETR4[2], CAPM_PETR4[2],
Medias_Observadas_PETR4[3], CAPM_PETR4[3],
Medias_Observadas_PETR4[4], CAPM_PETR4[4])
nomes_barras <- rep(c("Observado", "Justo"), 4)
espacos <- c(0, 0.5, 1.5, 0.5, 1.5, 0.5, 1.5, 0.5)
cores <- rep(c("darkgreen", "darkblue"), 4)
# Gera o gráfico e captura as posições das barras
posicoes <- barplot(

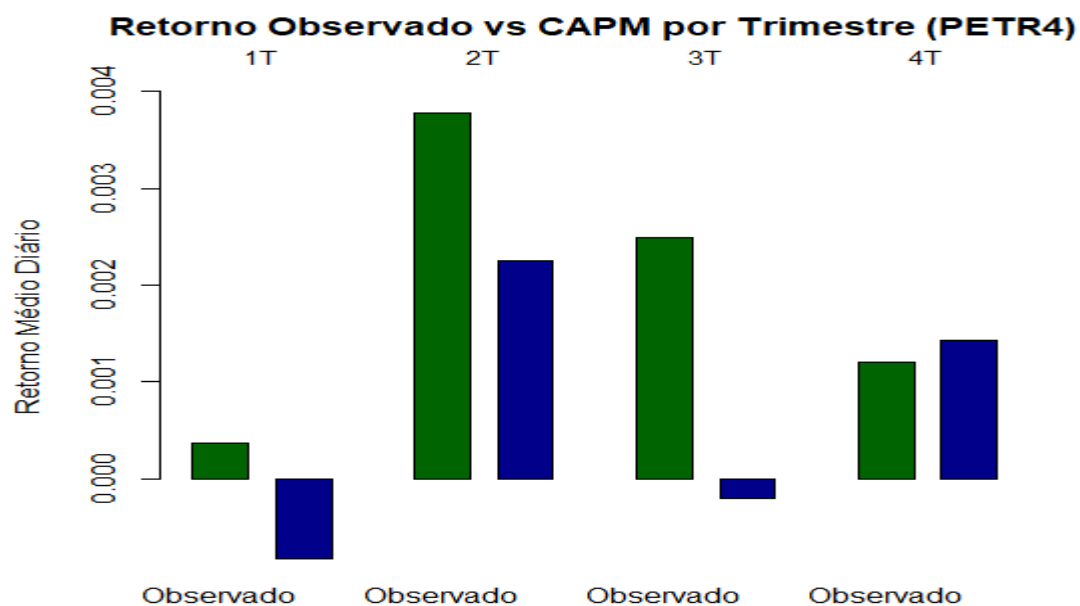
```

```

valores_PETR4,
beside = TRUE,
col = cores,
space = espacos,
ylim = c(min(valores_PETR4), max(valores_PETR4) * 1.1),
names.arg = nomes_barras,
main = "Retorno Observado vs CAPM por Trimestre (PETR4)",
ylab = "Retorno Médio Diário"
)
# Calcula os centros de cada par (trimestre)
centros_trimestres <- colMeans(matrix(posicoes, nrow = 2))
# Adiciona os rótulos centrados
text(x = centros_trimestres, y = par("usr")[4] * 1.05,
labels = c("1T", "2T", "3T", "4T"), xpd = TRUE, cex = 0.9)

```

Figura 80. Retorno Observado vs CAPM por Trimestre (PETR4)



Fonte: Elaboração própria

10.3 ESTIMAÇÃO DA LINHA DE MERCADO DE TÍTULOS (LMT)

Nesta seção, aplicaremos o Modelo de Precificação de Ativos Financeiros (CAPM) para estimar o risco sistemático de alguns ativos representativos da B3

e analisar sua relação com a Linha de Mercado de Títulos (LMT). O objetivo é verificar como os retornos observados dos ativos se posicionam em relação aos retornos esperados de acordo com o modelo teórico, permitindo identificar se estão subprecificados (acima da LMT) ou sobreprecificados (abaixo da LMT).

Para tanto, utilizaremos dados diários de empresas listadas na bolsa brasileira, como Petrobras (PETR4), Banco do Brasil (BBAS3), Vale (VALE3) e Ambev (ABEV3), no ano de referência, tendo o índice Ibovespa (IBOV) como proxy para o mercado. A metodologia envolve três etapas principais:

1. Cálculo dos retornos logarítmicos diários dos ativos e do índice de mercado.
2. Estimação dos coeficientes beta por meio de regressões lineares, a fim de mensurar a sensibilidade de cada ação em relação às variações do mercado.
3. Construção da Linha de Mercado de Títulos (LMT), comparando os retornos médios observados com os retornos “justos” sugeridos pelo CAPM.

Além disso, o código foi estruturado de forma genérica, permitindo sua aplicação a qualquer quantidade de ativos. A única exigência é que o último ticker informado na lista corresponda ao índice de mercado (por exemplo, o Ibovespa), enquanto os demais são tratados como ativos individuais. Assim, basta ajustar a lista de tickers, sejam 4, 10 ou mais ativos, para que a rotina se adapte automaticamente, entregando tanto a tabela-resumo com betas e retornos quanto o gráfico da LMT.

Esse exercício ilustra, de forma prática, como o CAPM pode ser utilizado não apenas para fins de ensino, mas também para análises aplicadas de precificação e avaliação de risco em diferentes contextos de investimento, combinando rigor teórico e flexibilidade computacional.

```
##### LINHA DE MERCADO DE TÍTULOS — CÓDIGO GENÉRICO
#####

library(quantmod)
# ----- PARÂMETROS -----
# O ÚLTIMO ticker é o ÍNDICE DE MERCADO (ex.: "^BVSP")
tickers                                     <-
c("PETR4.SA","BBAS3.SA","VALE3.SA","ABEV3.SA","^BVSP")
```

```

from <- as.Date("2023-01-01")
to <- as.Date("2023-12-31")
Rf_annual <- 0.05 # taxa livre de risco (5% a.a.)
ERm_annual <- 0.12 # retorno esperado do mercado (12% a.a.)
premium <- ERm_annual - Rf_annual
# ----- FUNÇÕES AUXILIARES -----
ret_log <- function(x) diff(log(Cl(x))) # retornos log diários
ann_mean <- function(x) mean(x) * 252 # anualização por
média*252
ann_log <- function(x) exp(sum(x)) - 1 # alternativa: retorno log
acumulado anual
# ----- DOWNLOAD E CÁLCULO DE RETORNOS -----
# Baixa cada série sem criar objetos no ambiente e calcula os
retornos
rets_list <- lapply(tickers, function(tk){
  x <- getSymbols(tk, src="yahoo", from=from, to=to,
auto.assign=FALSE)
  ret_log(x)
})
# Nomes limpos para colunas (remove sufixo .SA e o ^ de índices)
colnames_list <- gsub("\\.SA$", "", gsub("^\\^", "", tickers))
names(rets_list) <- colnames_list
# Merge e remove NAs em qualquer série
R <- na.omit(do.call(merge, rets_list))
colnames(R) <- colnames_list
# Define ativos e mercado (o último é o índice)
ativos <- colnames(R)[-ncol(R)]
mercado <- colnames(R)[ ncol(R)]
# ----- ESTIMAÇÃO DOS BETAS (Market Model:  $r_{i,t} \sim r_{m,t}$ ) -----
betas <- sapply(ativos, function(a){
  mod <- lm(coredata(R[,a]) ~ coredata(R[,mercado]))
  coef(mod)[2]
})
# (Opcional) Intervalos de confiança 95% dos betas

```

```

beta_ci <- t(sapply(ativos, function(a){
  mod <- lm(coredata(R[,a]) ~ coredata(R[,mercado]))
  confint(mod, level=0.95)[2,] # intervalo do coeficiente de r_m
}))
colnames(beta_ci) <- c("CI_low","CI_high")
# ----- RETORNOS OBSERVADOS (ANUALIZADOS) -----
ER_obs <- sapply(ativos, function(a) ann_mean(coredata(R[,a])))
# Alternativa (comente a linha acima e descomente a seguinte, se
preferir):
# ER_obs <- sapply(ativos, function(a) ann_log(coredata(R[,a])))
# ----- RETORNOS JUSTOS PELO CAPM -----
ER_capm <- Rf_annual + betas * premium
# ----- CLASSIFICAÇÃO -----
status <- ifelse(ER_obs > ER_capm, "Subprecificado",
"Sobreprecificado")
# ----- TABELA-RESUMO -----
tab <- data.frame(
  Ativo = ativos,
  Beta = round(betas, 3),
  Beta_CI_low = round(beta_ci[,1], 3),
  Beta_CI_high = round(beta_ci[,2], 3),
  `E[R] observado (%)` = round(ER_obs*100, 2),
  `E[R] CAPM (%)` = round(ER_capm*100, 2),
  Status = status,
  row.names = NULL
)
print(tab)
# ----- GRÁFICO DA LMT COM PONTOS -----
beta_seq <- seq(0, max(1.4, 1.1*max(betas, na.rm=TRUE)),
length.out=200)
LMT_seq <- Rf_annual + beta_seq * premium
# Limites do eixo Y considerando reta + pontos + Rf + ERm
all_y <- c(Rf_annual, ERm_annual, LMT_seq, ER_obs)
ylim_use <- c(min(all_y, na.rm=TRUE)*0.95, max(all_y,

```



```

na.rm=TRUE)*1.05)

plot(beta_seq, LMT_seq, type="l", lwd=2,
main="Linha de Mercado de Títulos (LMT) — CAPM",
xlab=expression(beta), ylab="Retorno esperado E[R]",
ylim=ylim_use)
cols <- ifelse(status=="Subprecificado", "darkgreen", "firebrick")
points(as.numeric(betas), as.numeric(ER_obs), pch=19, cex=1.25,
col=cols)

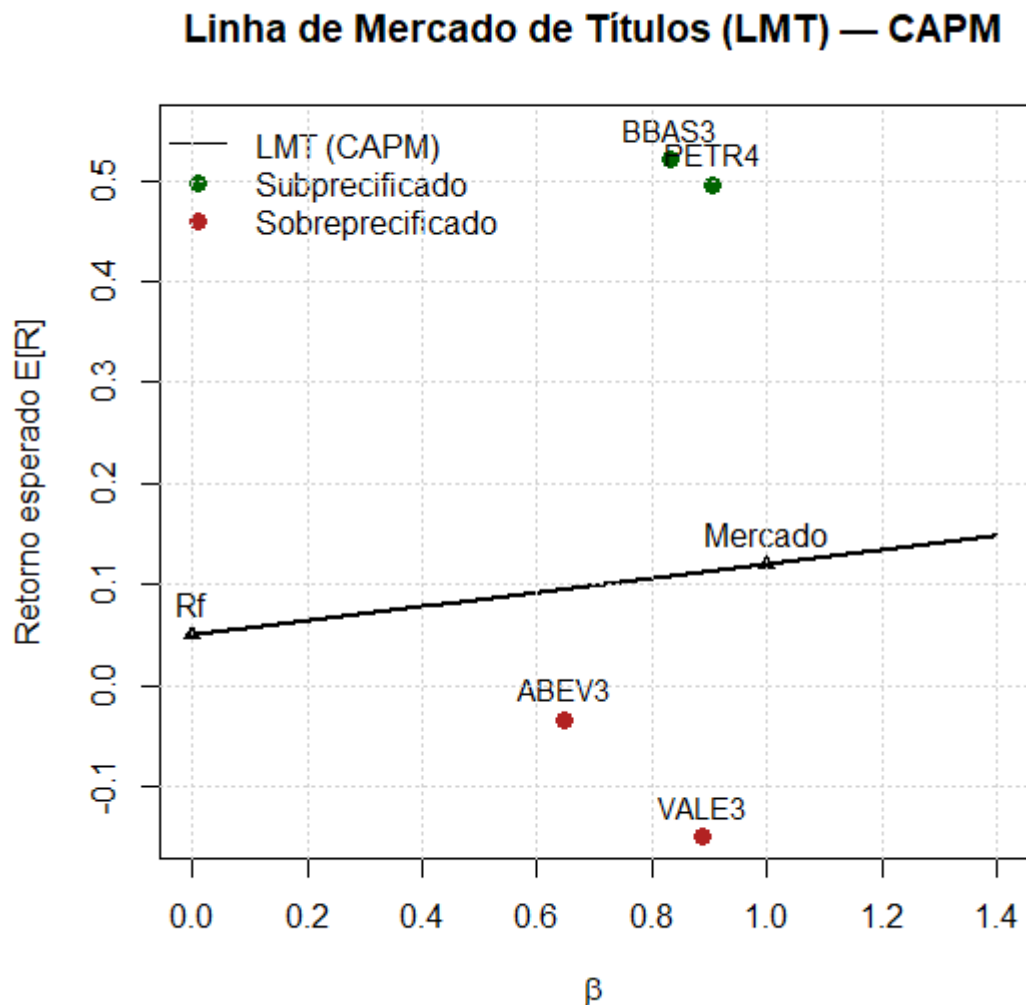
text(as.numeric(betas), as.numeric(ER_obs), labels=ativos, pos=3,
cex=0.9)

# Referências Rf ( $\beta=0$ ) e Mercado ( $\beta=1$ )
points(0, Rf_annual, pch=17); text(0, Rf_annual, "Rf", pos=3)
points(1, ERm_annual, pch=17); text(1, ERm_annual, "Mercado",
pos=3)

legend("topleft",
legend=c("LMT (CAPM)", "Subprecificado", "Sobrepresificado"),
lty=c(1, NA, NA), pch=c(NA,19,19),
col=c("black","darkgreen","firebrick"), pt.cex=1.2, bty="n")
grid()

```

Figura 81. Linha de Mercado de Títulos (LMT) - CAPM



Fonte: Elaboração própria

10.4 AVALIANDO PORTFÓLIOS COM CAPM

Uma vez obtidos os retornos das carteiras de variância mínima e carteira eficiente, é possível avaliar o seu desempenho à luz do Capital Asset Pricing Model (CAPM). O objetivo aqui é analisar se tais carteiras apresentam perfil defensivo ($\beta < 1$) ou agressivo ($\beta > 1$), além de verificar a evolução do risco sistemático ao longo do tempo.

A análise é conduzida de forma trimestral, permitindo observar a dinâmica dos coeficientes β em diferentes períodos e identificar eventuais tendências crescentes ou decrescentes no risco das carteiras. Em seguida, confrontamos os retornos médios observados com os retornos justos previstos pelo CAPM, de modo a avaliar se as carteiras estariam subprecificadas (retorno acima do justo)

ou sobreprecificadas (retorno abaixo do justo).

Essa abordagem complementa a construção da fronteira eficiente realizada anteriormente, oferecendo uma perspectiva adicional sobre a relação entre risco e retorno. Enquanto a fronteira eficiente privilegia a diversificação e a minimização da variância, a análise via CAPM permite verificar como as carteiras se posicionam em relação ao mercado, ampliando a compreensão sobre seu comportamento e atratividade para o investidor.

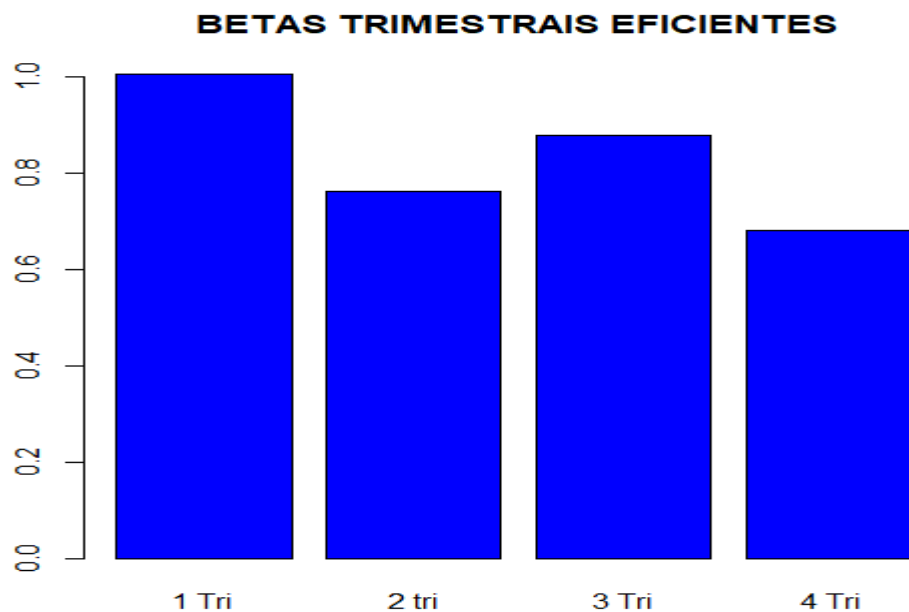
```
##### CAPM E PRECIFICAÇÃO DA CARTEIRA #####  
#Retorno da Carteira de Variância Mínima  
ret_cvm  
#Retorno da Carteira de Equal Wage  
ret_ew  
#Retorno da Carteira de Eficiente  
ret_ef  
##### CARTEIRA EFICIENTE #####  
#Montar base por Trimestre  
base4<-cbind(ret_ef,ribov,trimestre[-1]) #ja existem objetos com  
nome base1, 2 e 3  
base4[which(base4[,3]==1),] #recorta a base 1T  
# Separação dos dados por trimestre  
ret_ef_1T <- base4[base4[,3] == 1, 1] # Retorno EFICIENTE - 1º  
trimestre  
ret_ef_2T <- base4[base4[,3] == 2, 1] # Retorno EFICIENTE - 2º  
trimestre  
ret_ef_3T <- base4[base4[,3] == 3, 1] # Retorno EFICIENTE - 3º  
trimestre  
ret_ef_4T <- base4[base4[,3] == 4, 1] # Retorno EFICIENTE - 4º  
trimestre  
ribov_1T <- base4[base4[,3] == 1, 2] # Retorno IBOV - 1º trimestre  
ribov_2T <- base4[base4[,3] == 2, 2] # Retorno IBOV - 2º trimestre  
ribov_3T <- base4[base4[,3] == 3, 2] # Retorno IBOV - 3º trimestre  
ribov_4T <- base4[base4[,3] == 4, 2] # Retorno IBOV - 4º trimestre  
# Regressões por trimestre (modelo de CAPM)  
reg1T_ef <- lm(ret_ef_1T ~ ribov_1T)
```

```

summary(reg1T_ef)
reg2T_ef <- lm(ret_ef_2T ~ ribov_2T)
summary(reg2T_ef)
reg3T_ef <- lm(ret_ef_3T ~ ribov_3T)
summary(reg3T_ef)
reg4T_ef <- lm(ret_ef_4T ~ ribov_4T)
summary(reg4T_ef)
#Coeficientes Beta por Trimestre
beta1T_ef<-coef(reg1T_ef)[2]
beta2T_ef<-coef(reg2T_ef)[2]
beta3T_ef<-coef(reg3T_ef)[2]
beta4T_ef<-coef(reg4T_ef)[2]
#Base de Betas
betas_ef<-c(beta1T_ef,beta2T_ef,beta3T_ef,beta4T_ef)
names(betas_ef)<-c("1 Tri", "2 tri", "3 Tri", "4 Tri")
barplot(betas_ef,col="blue",      main="BETAS      TRIMESTRAIS
EFICIENTES")

```

Figura 82. Betas Trimestrais Eficientes



Fonte: Elaboração própria

```

#Retornos Justos (CAMP)
#CAPM = rf + beta*(E(Rm) - rf)
rf<- 0.125/252
CAPM_1T_ef <- rf + as.numeric(beta1T_ef) * (mean(na.omit(ribov_1T) -
rf))
CAPM_2T_ef <- rf + as.numeric(beta2T_ef) * (mean(na.omit(ribov_2T)) -
rf)
CAPM_3T_ef <- rf + as.numeric(beta3T_ef) * (mean(na.omit(ribov_3T)) -
rf)
CAPM_4T_ef <- rf + as.numeric(beta4T_ef) * (mean(na.omit(ribov_4T)) -
rf)

CAPM_1T_ef
CAPM_2T_ef
CAPM_3T_ef
CAPM_4T_ef

#Precificação - Se o retorno observado for maior que o retorno justo, o
ativo esta subprecificado, caso contrário sobreprecificado
mean_ref_1T <- mean(na.omit(ret_ef_1T))
mean_ref_2T <- mean(na.omit(ret_ef_2T))
mean_ref_3T <- mean(na.omit(ret_ef_3T))
mean_ref_4T <- mean(na.omit(ret_ef_4T))
CAPM_ef<-c(CAPM_1T_ef,CAPM_2T_ef,CAPM_3T_ef,CAPM_4T_ef)
Medias_Observadas_ef                                     <-
c(mean_ref_1T,mean_ref_2T,mean_ref_3T,mean_ref_4T)
Status_ef  =  ifelse(c(mean_ref_1T,  mean_ref_2T,  mean_ref_3T,
mean_ref_4T)  >  c(CAPM_1T_ef,  CAPM_2T_ef,  CAPM_3T_ef,
CAPM_4T_ef),"Subprecificado", "Sobreprecificado")
resultado_ef<-
data.frame(matrix(c(CAPM_ef,Medias_Observadas_ef,Status_ef),4,3))
colnames(resultado_ef) <- c("Retorno_CAPM", "Retorno_Observado",
"Status")
rownames(resultado_ef) <- paste0(1:4, "T")
resultado_ef
# Dados do gráfico

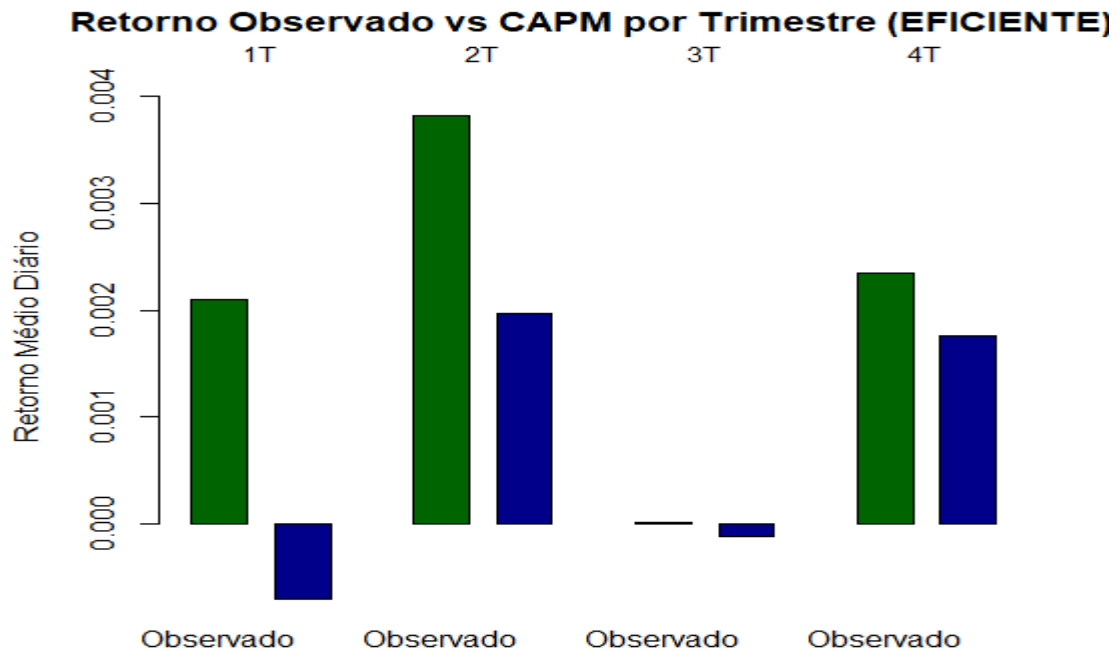
```

```

valores_ef <- c(Medias_Observadas_ef[1], CAPM_ef[1],
Medias_Observadas_ef[2], CAPM_ef[2],
Medias_Observadas_ef[3], CAPM_ef[3],
Medias_Observadas_ef[4], CAPM_ef[4])
nomes_barras <- rep(c("Observado", "Justo"), 4)
espacos <- c(0, 0.5, 1.5, 0.5, 1.5, 0.5, 1.5, 0.5)
cores <- rep(c("darkgreen", "darkblue"), 4)
# Gera o gráfico e captura as posições das barras
posicoes <- barplot(
valores_ef,
beside = TRUE,
col = cores,
space = espacos,
ylim = c(min(valores_ef), max(valores_ef) * 1.1),
names.arg = nomes_barras,
main = "Retorno Observado vs CAPM por Trimestre (EFICIENTE)",
ylab = "Retorno Médio Diário"
)
# Calcula os centros de cada par (trimestre)
centros_trimestres <- colMeans(matrix(posicoes, nrow = 2))
# Adiciona os rótulos centrados
text(x = centros_trimestres, y = par("usr")[4] * 1.05,
labels = c("1T", "2T", "3T", "4T"), xpd = TRUE, cex = 0.9)

```

Figura 83. Retorno Obersevado vs CAPM por trimestre (EFICIENTE)



Fonte: Elaboração própria

```
##### CARTEIRA VARIÂNCIA MÍNIMA #####
#Montar base por Trimestre
base5<-cbind(ret_cvm,ribov,trimestre[-1]) #ja existem objetos com nome
base1, 2, 3 e 4
base5[which(base5[,3]==1),] #recorta a base 1T
# Separação dos dados por trimestre
ret_cvm_1T <- base5[base5[,3] == 1, 1] # Retorno VARIÂNCIA MÍNIMA -
1º trimestre
ret_cvm_2T <- base5[base5[,3] == 2, 1] # Retorno VARIÂNCIA MÍNIMA-
2º trimestre
ret_cvm_3T <- base5[base5[,3] == 3, 1] # Retorno VARIÂNCIA MÍNIMA-
3º trimestre
ret_cvm_4T <- base5[base5[,3] == 4, 1] # Retorno VARIÂNCIA MÍNIMA-
4º trimestre
ribov_1T <- base5[base5[,3] == 1, 2] # Retorno IBOV - 1º trimestre
ribov_2T <- base5[base5[,3] == 2, 2] # Retorno IBOV - 2º trimestre
ribov_3T <- base5[base5[,3] == 3, 2] # Retorno IBOV - 3º trimestre
ribov_4T <- base5[base5[,3] == 4, 2] # Retorno IBOV - 4º trimestre
```

```

# Regressões por trimestre (modelo de CAPM)
reg1T_cvm<- lm(ret_cvm_1T ~ ribov_1T)
summary(reg1T_cvm)
reg2T_cvm<- lm(ret_cvm_2T ~ ribov_2T)
summary(reg2T_cvm)
reg3T_cvm<- lm(ret_cvm_3T ~ ribov_3T)
summary(reg3T_cvm)
reg4T_cvm<- lm(ret_cvm_4T ~ ribov_4T)
summary(reg4T_cvm)

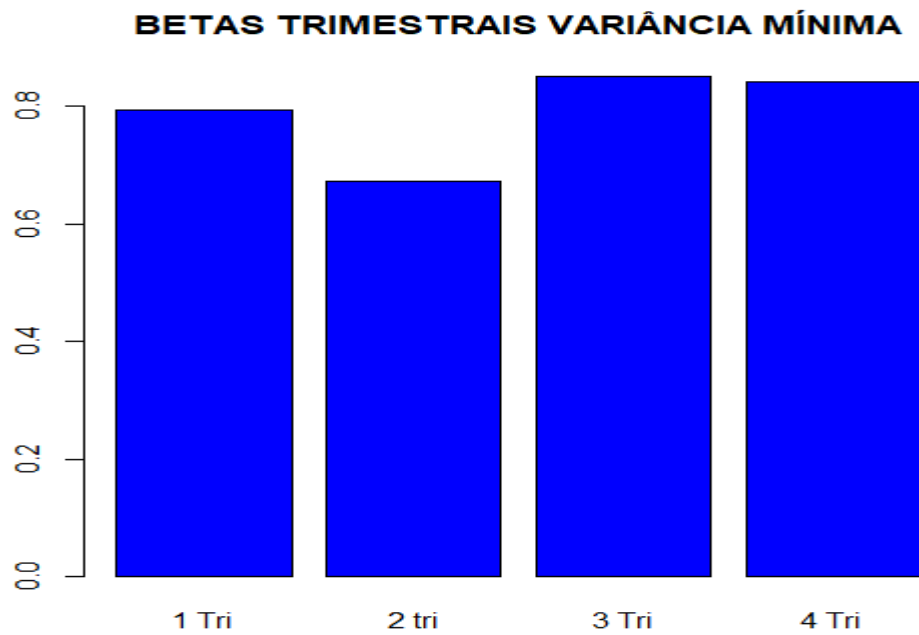
#Coeficientes Beta por Trimestre
beta1T_cvm<-coef(reg1T_cvm)[2]
beta2T_cvm<-coef(reg2T_cvm)[2]
beta3T_cvm<-coef(reg3T_cvm)[2]
beta4T_cvm<-coef(reg4T_cvm)[2]

#Base de Betas
betas_cvm<-c(beta1T_cvm,beta2T_cvm,beta3T_cvm,beta4T_cvm)
names(betas_cvm)<-c("1 Tri", "2 tri", "3 Tri", "4 Tri")

barplot(betas_cvm,col="blue", main="BETAS TRIMESTRAIS VARIÂNCIA
MÍNIMA")

```


Figura 84. Betas Trimestrais Variância Mínima



Fonte: Elaboração própria

```
#Retornos Justos (CAMP)
#CAPM = rf + beta*(E(Rm) - rf)
rf<- 0.125/252
CAPM_1T_cvm<- rf + as.numeric(beta1T_cvm) *
(mean(na.omit(ribov_1T) - rf))
CAPM_2T_cvm<- rf + as.numeric(beta2T_cvm) *
(mean(na.omit(ribov_2T)) - rf)
CAPM_3T_cvm<- rf + as.numeric(beta3T_cvm) *
(mean(na.omit(ribov_3T)) - rf)
CAPM_4T_cvm<- rf + as.numeric(beta4T_cvm) *
(mean(na.omit(ribov_4T)) - rf)
CAPM_1T_cvm
CAPM_2T_cvm
CAPM_3T_cvm
CAPM_4T_cvm
#Precificação - Se o retorno observado for maior que o retorno justo, o
ativo esta subprecificado, caso contrário sobreprecificado
mean_rcvm_1T <- mean(na.omit(ret_cvm_1T))
```

```

mean_rcvm_2T <- mean(na.omit(ret_cvm_2T))
mean_rcvm_3T <- mean(na.omit(ret_cvm_3T))
mean_rcvm_4T <- mean(na.omit(ret_cvm_4T))
CAPM_cvm<-
c(CAPM_1T_cvm,CAPM_2T_cvm,CAPM_3T_cvm,CAPM_4T_cvm)
Medias_Observadas_cvm<-
c(mean_rcvm_1T,mean_rcvm_2T,mean_rcvm_3T,mean_rcvm_4T)
Status_cvm = ifelse(c(mean_rcvm_1T, mean_rcvm_2T, mean_rcvm_3T,
mean_rcvm_4T) > c(CAPM_1T_cvm, CAPM_2T_cvm, CAPM_3T_cvm,
CAPM_4T_cvm),"Subprecificado", "Sobrepresificado")
resultado_cvm<-
data.frame(matrix(c(CAPM_cvm,Medias_Observadas_cvm,Status_cvm),4,3))
colnames(resultado_cvm) <- c("Retorno_CAPM", "Retorno_Observado",
"Status")
rownames(resultado_cvm) <- paste0(1:4, "T")
resultado_cvm
# Dados do gráfico
valores_cvm<- c(Medias_Observadas_cvm[1], CAPM_cvm[1],
Medias_Observadas_cvm[2], CAPM_cvm[2],
Medias_Observadas_cvm[3], CAPM_cvm[3],
Medias_Observadas_cvm[4], CAPM_cvm[4])
nomes_barras <- rep(c("Observado", "Justo"), 4)
espacos <- c(0, 0.5, 1.5, 0.5, 1.5, 0.5, 1.5, 0.5)
cores <- rep(c("darkgreen", "darkblue"), 4)
# Gera o gráfico e captura as posições das barras
posicoes <- barplot(
valores_cvm,
beside = TRUE,
col = cores,
space = espacos,
ylim = c(min(valores_cvm), max(valores_cvm) * 1.1),
names.arg = nomes_barras,
main = "Retorno Observado vs CAPM por Trimestre (VARIÂNCIA
MÍNIMA)",

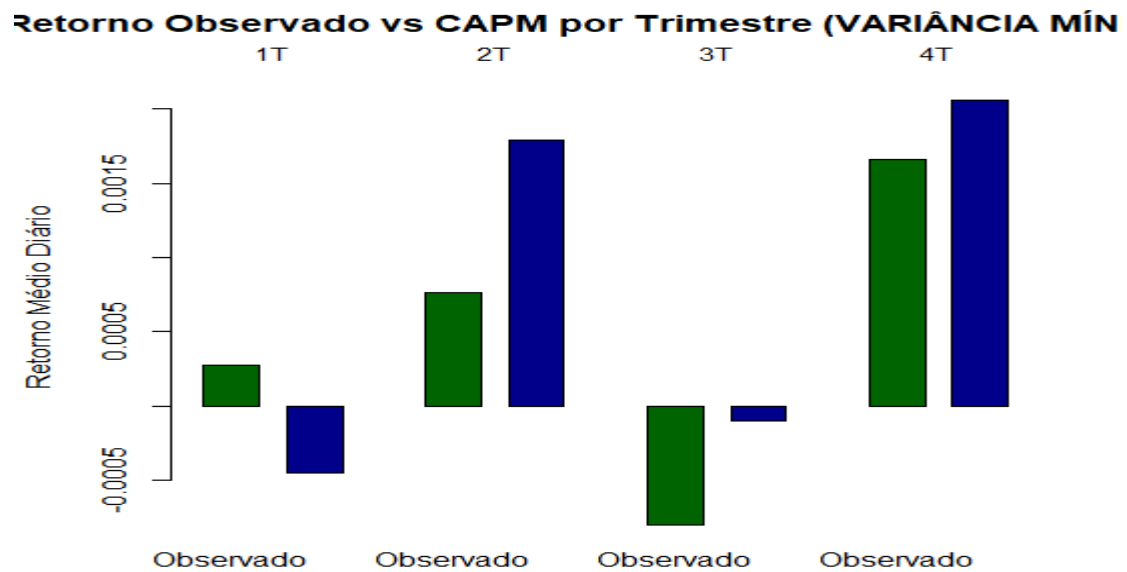
```

```

ylab = "Retorno Médio Diário"
)
# Calcula os centros de cada par (trimestre)
centros_trimestres <- colMeans(matrix(posicoes, nrow = 2))
# Adiciona os rótulos centrados
text(x = centros_trimestres, y = par("usr")[4] * 1.05,
labels = c("1T", "2T", "3T", "4T"), xpd = TRUE, cex = 0.9)

```

Figura 85. Retorno Observado vs CAPM por Trimestre (VARIÂNCIA MÍN)



Fonte: Elaboração própria

REFERÊNCIAS

- BARRETO, Alexandre Serra. Modelos de Regressão: Teoria e Aplicação com o programa estatístico R. ISBN: 8591172809. Edição do próprio autor, 2011.
- BODIE, Zvi; KANE, Alex; MARCUS, Alan J. Fundamentos de Investimentos. 3.ed. Porto Alegre: Bookman, 2000.
- DATASHENANIGAN. *A gentle introduction to finance using R: Efficient Frontier and CAPM – Part 1*. 2016. Disponível em: <https://datashenanigan.wordpress.com/2016/05/24/a-gentle-introduction-to-finance-using-r-efficient-frontier-and-capm-part-1/>
- GIAMBIAGI, Fábio; ALÉM, Ana Cláudia; GARSON, Sol (colab.). *Finanças públicas: teoria e prática no Brasil*. 5. ed. rev. atual. Rio de Janeiro: Elsevier, 2016. 539 p.
- LANDEIRO, Victor Lemes. Introdução ao uso do programa R. Instituto Nacional de Pesquisas da Amazônia: Programa de Pós Graduação em Ecologia, Manaus-AM, 2011. Disponível em: <http://ppbio.inpa.gov.br/Port/public/disciplinas>
- LUÍS, Torgo. Introdução à Programação em R. Faculdade de Economia: Universidade do Porto, 2006. Disponível em: <https://cran.r-project.org/doc/contrib/Torgo-ProgrammingIntro.pdf>
- KAHNEMAN, D.; TVERSKY, A. Prospect theory: an analysis of decisions under risk. *Econometrica*, v. 47, n. 2, p. 263-291, march. 1979
- MARTINS, Paola da Silva, LUCIENE, Alcoforado. Treinando habilidades de elaboração de gráficos com o software R. Universidade Federal Fluminense. Disponível em: <http://www.professores.uff.br/luciane/images/stories/Arquivos/Rgraficos.pdf>
- PONTES, Felipe. Passo a Passo para Otimização de uma Carteira usando o R: Tutorial. Canal ContabilidadeMQ, 2018.. disponível em: https://www.youtube.com/watch?v=BwDz_xiKdXw

SUGESTÃO DE TRABALHO PARA AVALIAÇÃO

Baixe as séries de preço das ações de sua equipe utilizado o pacote “quantmod”, utilize dados a partir de dd/mm/aaaa e siga as instruções para a resolução das questões 1 a 12

1. Fazer estatísticas descritivas dos preços das ações (média, mediana, Q1, Q3, valor máximo, valor mínimo). Coloque essas informações em gráfico de barras.
2. Utilize a função aggregate para fazer a média dos preços por mês e por trimestre. Apresente as informações em gráfico de barras.
3. Faça gráficos de Dispersão dois a dois entre as séries. Inclua reta de regressão e retas sobre o valor médio de cada ação.
4. Encontre a média de preço dessas ações quando todas elas operaram acima e abaixo de suas próprias médias. Apresente o resultado em gráfico de barras agrupando as médias acima e abaixo, das três ações, no mesmo gráfico.

5. Fazer Gráfico das Séries temporais individuais e juntas. Utilize séries em nível, em primeira diferença e das séries dos retornos. Inclua linhas horizontais sobre o valor médio das séries.
6. Fazer Gráfico das Series temporais criando uma janela (comando mfrow) para a série em nível e outra para a série de retorno, posicione a série de retorno abaixo da série em nível.
7. Faça análise técnica: identifique tendências, linhas de suporte e resistência sobre as tendências e sobre a série global.
8. Faça modelagem de médias móveis aritméticas para dois, três e quatro períodos. Qual seria o melhor modelo de previsão?
9. Mostre o retorno das estratégias *buy and hold*, *stop loss* e cruzamento de médias móveis (use MM3 e MM10).
10. Construa Bandas de Bollinger e gráfico de IFR.
11. Construa um conjunto de possibilidade de investimento para dois ativos. Selecione aqueles com menor correlação.
12. Calcule o Beta do CAPM do seu ativo por trimestre, apresente os betas trimestrais em gráfico de barras.

Agência Brasileira ISBN
ISBN: 978-65-83190-19-2